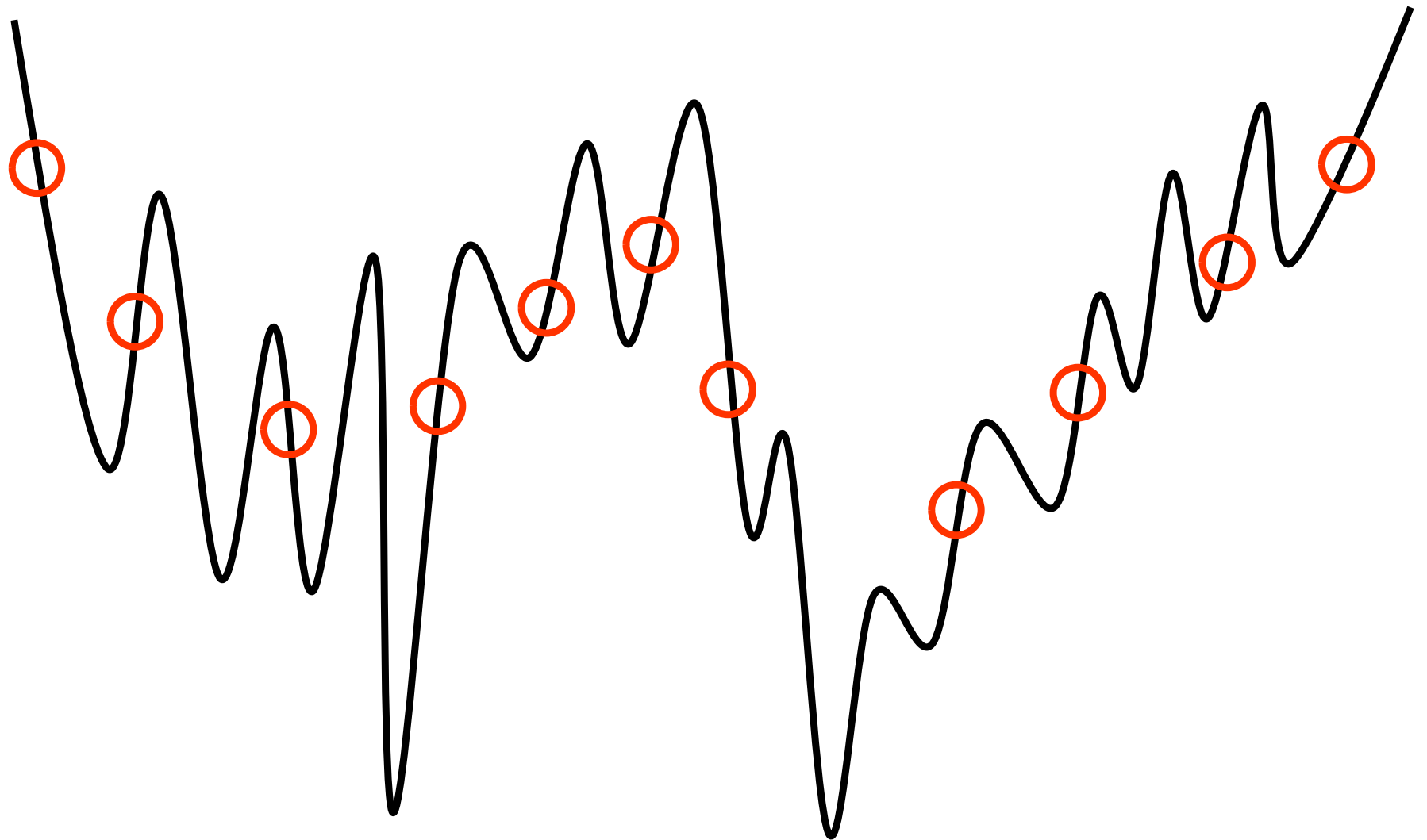


Evolutionary Algorithms: Population-based search (multi-point search)



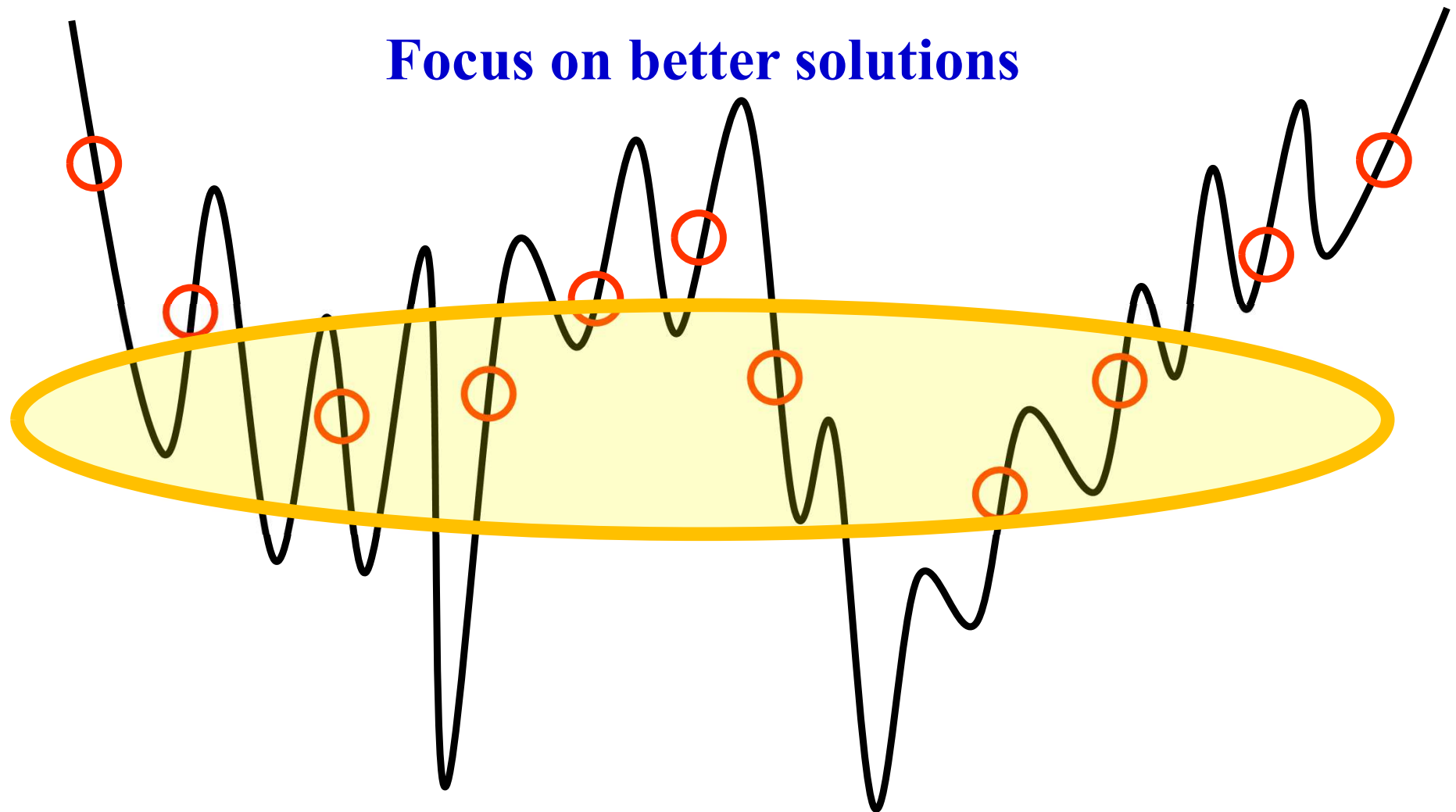
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



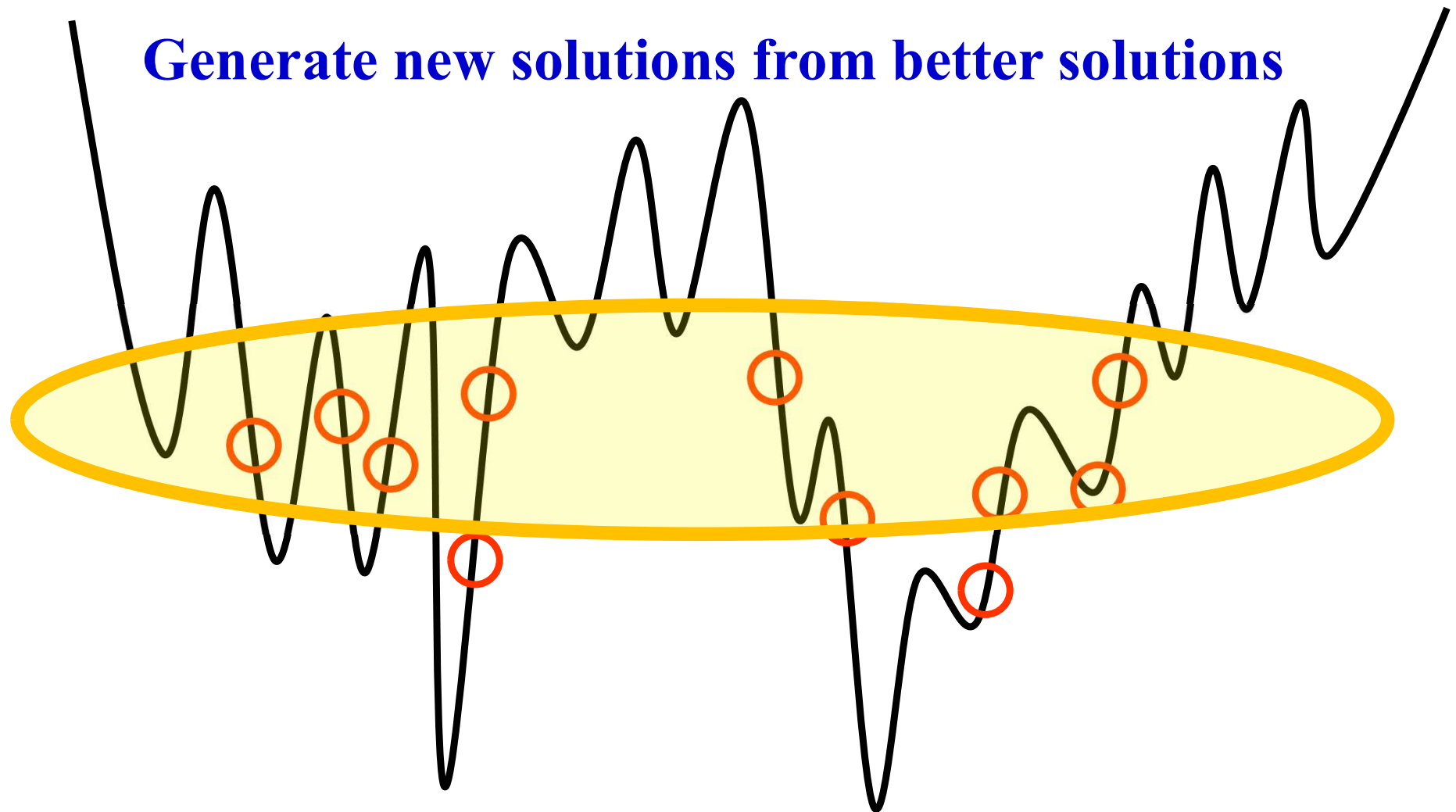
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



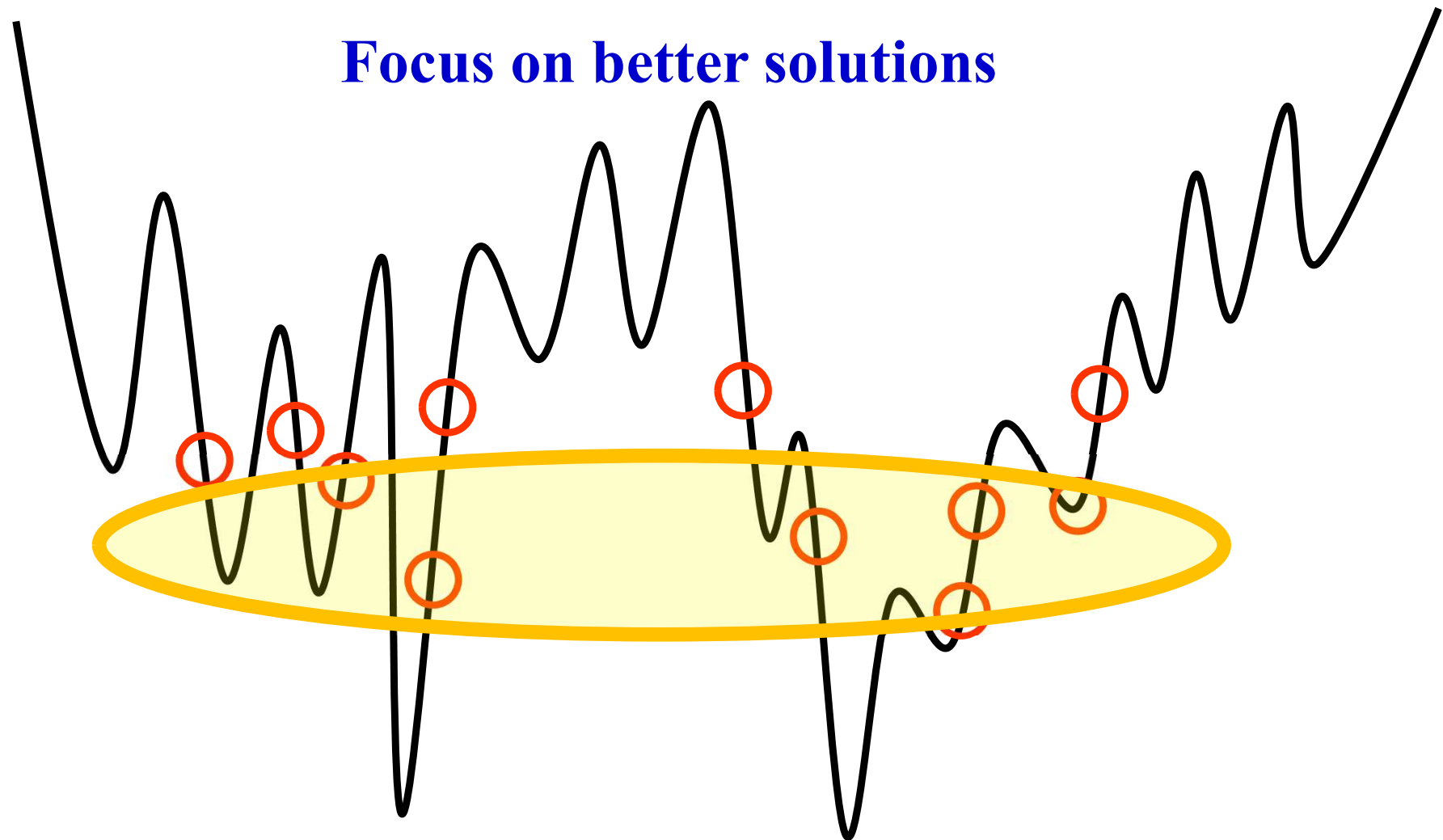
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



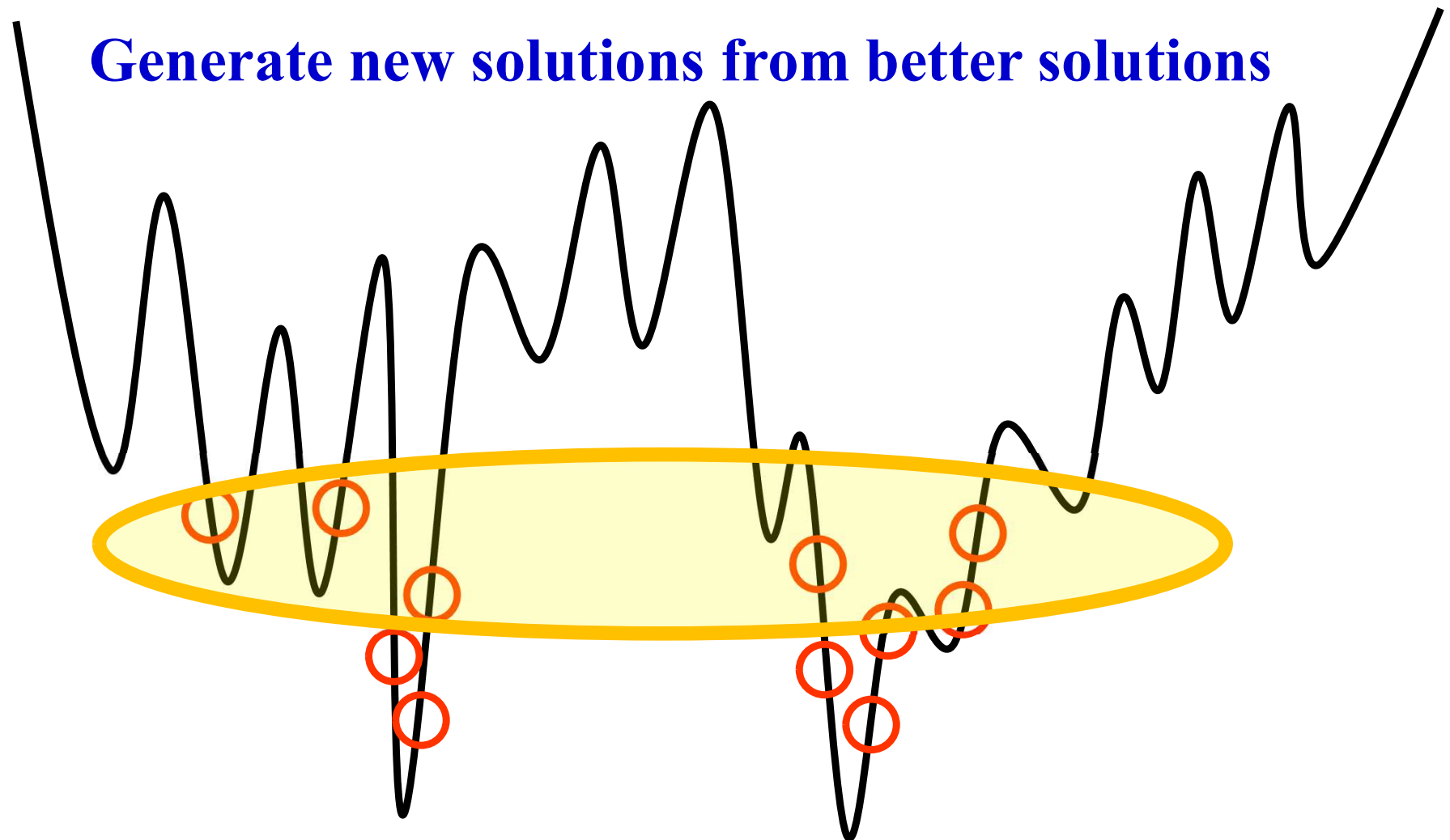
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



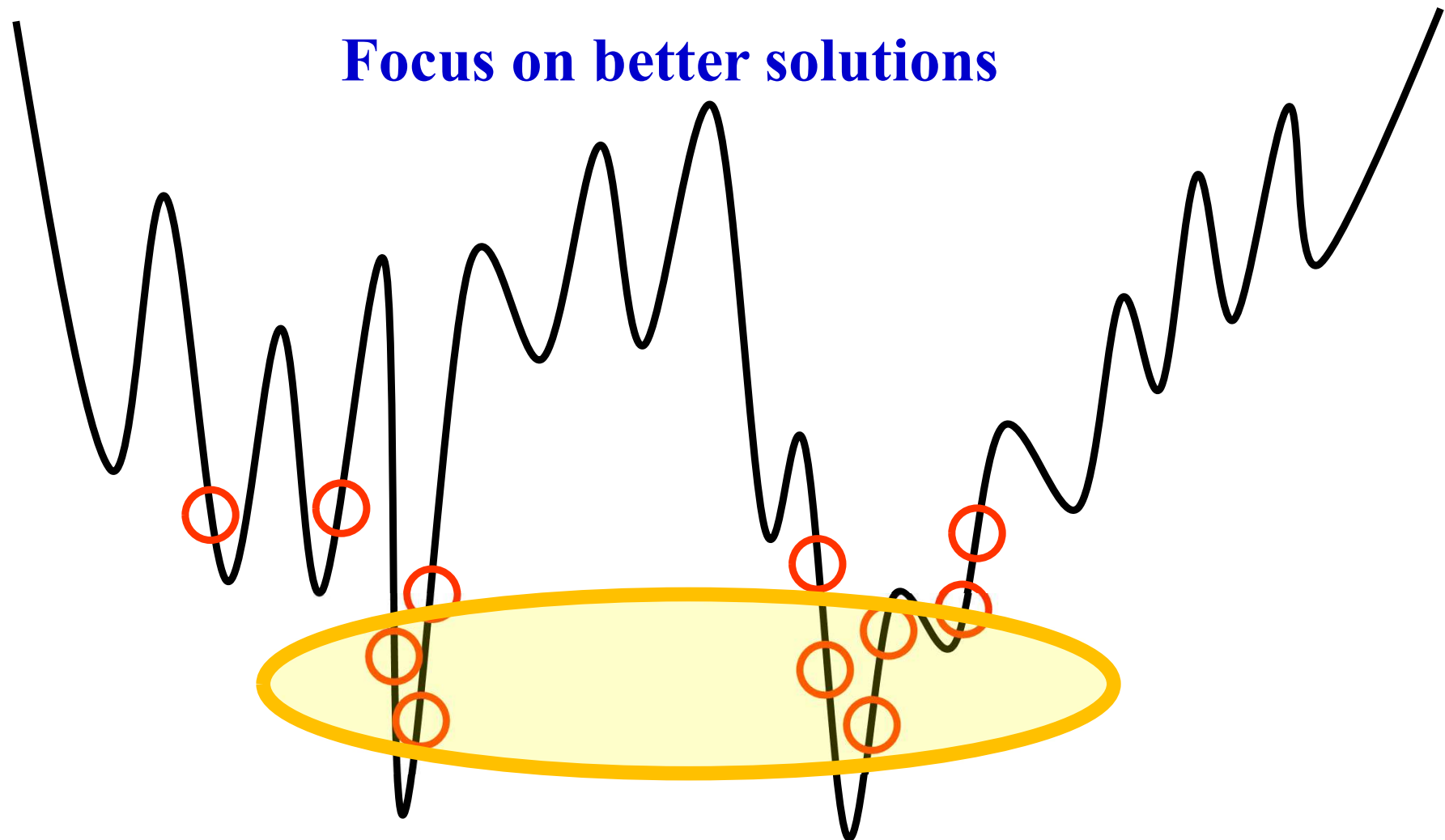
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



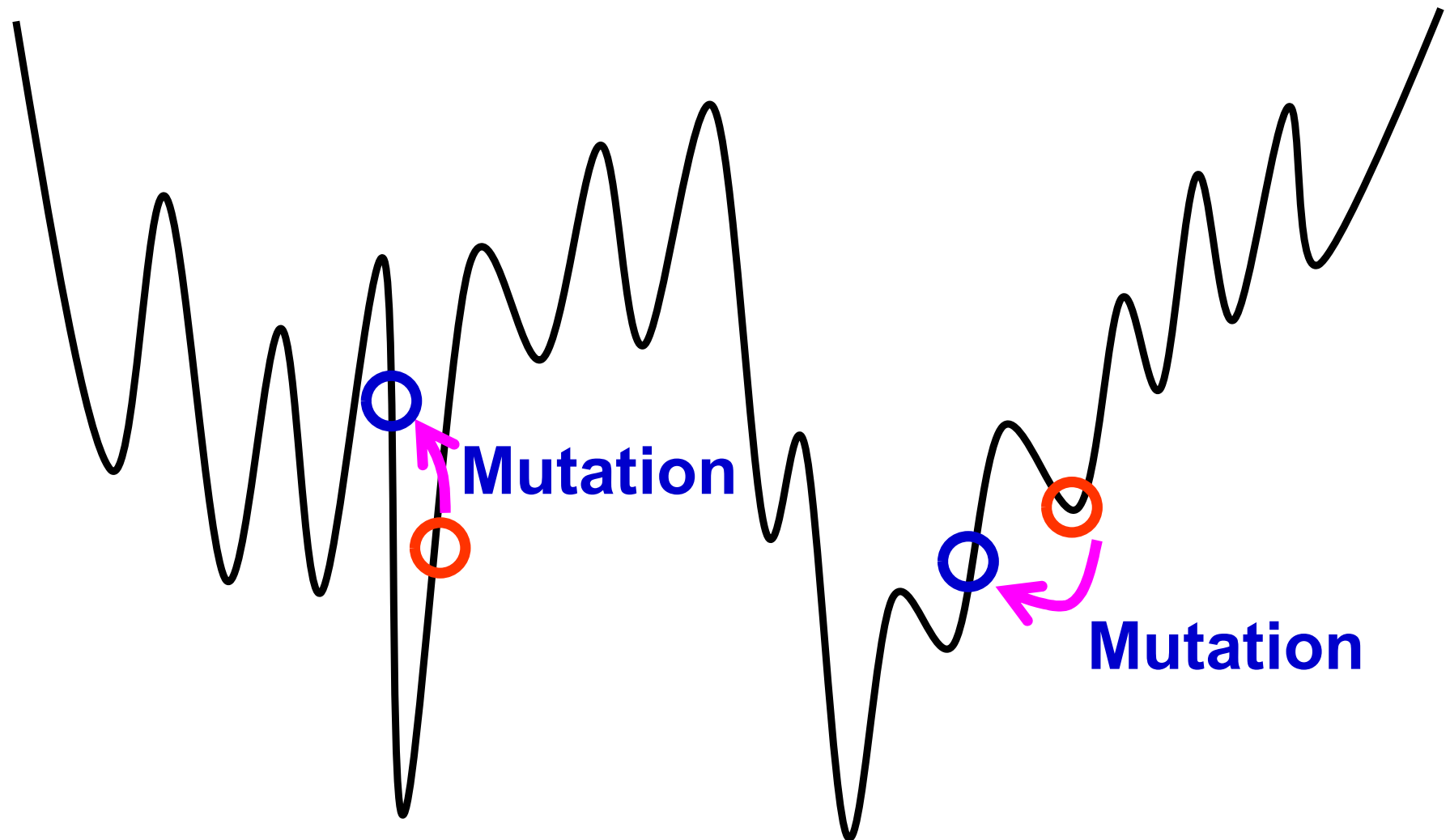
Evolutionary Algorithms (e.g., Genetic Algorithms)

- Population-based search algorithms
- Multi-point search algorithms



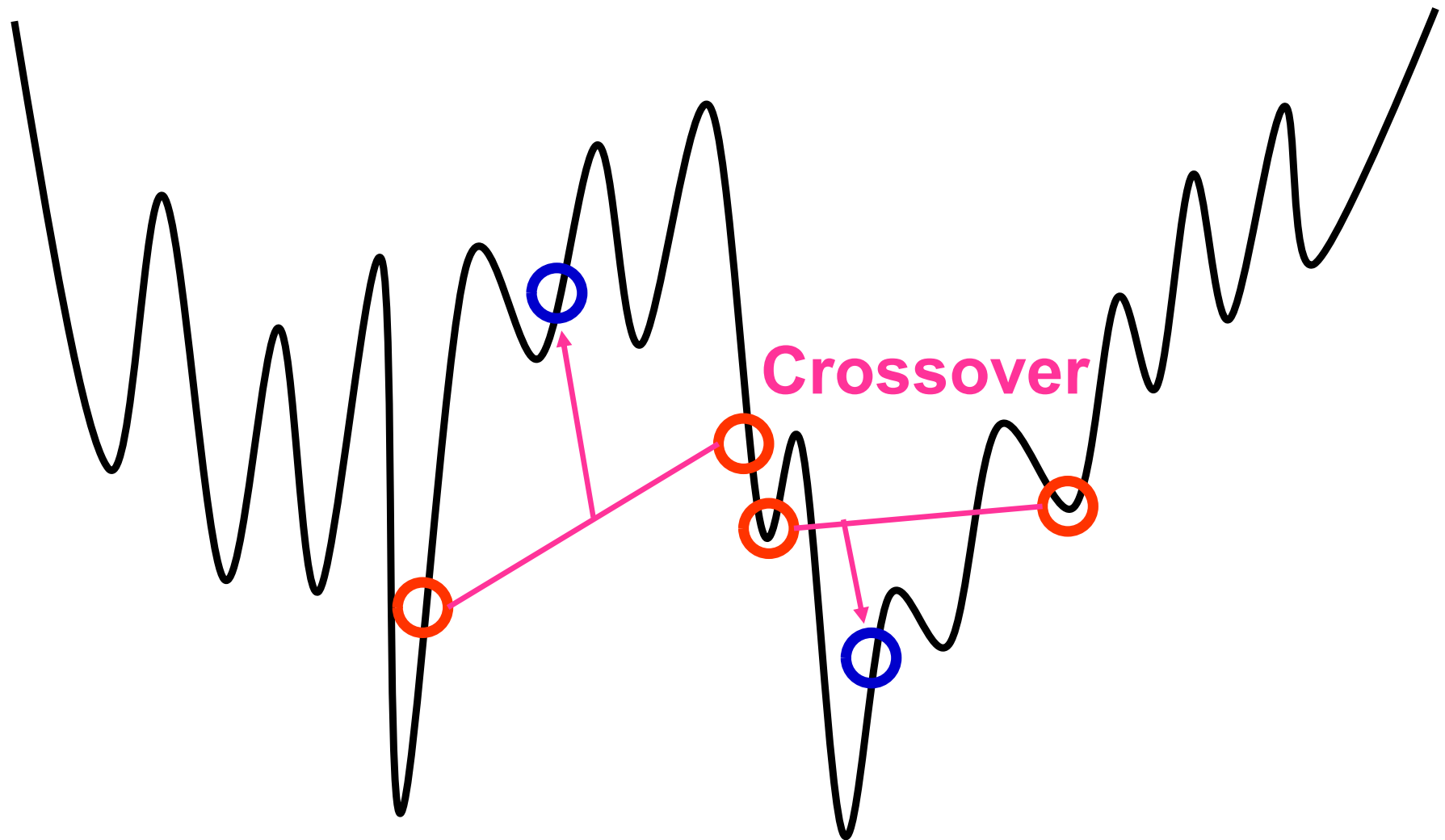
Two Operators for New Solution Generation

- (1) **Mutation** (Random choice of a neighbor)
- (2) **Crossover** (Recombination of two parents)



Two Operators for New Solution Generation

- (1) Mutation (Random choice of a neighbor)
- (2) Crossover (Recombination of two parents)



History of Evolutionary Computation

IEEE TRANSACTIONS ON EVOLUTIONARY COMPUTATION, VOL. 1, NO. 1, APRIL 1997

3

Evolutionary Computation: Comments on the History and Current State

Thomas Bäck, Ulrich Hammel, and Hans-Paul Schwefel

Studies in 1950s:

No computers, no clear advantages, not well-known even now

- [1] H. J. Bremermann, "Optimization through evolution and recombination," in *Self-Organizing Systems*, M. C. Yovits *et al.*, Eds. Washington, DC: Spartan, 1962.
- [2] R. M. Friedberg, "A learning machine: Part I," *IBM J.*, vol. 2, no. 1, pp. 2–13, Jan. 1958.
- [3] R. M. Friedberg, B. Dunham, and J. H. North, "A learning machine: Part II," *IBM J.*, vol. 3, no. 7, pp. 282–287, July 1959.
- [4] G. E. P. Box, "Evolutionary operation: A method for increasing industrial productivity," *Appl. Statistics*, vol. VI, no. 2, pp. 81–101, 1957.

History of Evolutionary Computation

Studies in 1960s:

Very weak computer power, some clear advantages,
not well-known until 1990s, currently known as origins of EC.

- [6] J. H. Holland, “Outline for a logical theory of adaptive systems,” *J. Assoc. Comput. Mach.*, vol. 3, pp. 297–314, 1962.
- [7] I. Rechenberg, “Cybernetic solution path of an experimental problem,” Royal Aircraft Establishment, Library translation No. 1122, Farnborough, Hants., U.K., Aug. 1965.
- [8] H.-P. Schwefel, “Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I,” Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968.
- [9] L. J. Fogel, “Autonomous automata,” *Ind. Res.*, vol. 4, pp. 14–19, 1962.

History of Evolutionary Computation

Studies in 1960s:

Very weak computer power, some clear advantages,
not well-known until 1990s, currently known as origins of EC.

- [6] J. H. Holland, “Outline for a logical theory of adaptive systems,” *J. Assoc. Comput. Mach.*, vol. 3, pp. 297–314, 1962. **GA**
- [7] I. Rechenberg, “Cybernetic solution path of an experimental problem,” Royal Aircraft Establishment, Library translation No. 1122, Farnborough, Hants., U.K., Aug. 1965. **ES (Evolution Strategies)**
- [8] H.-P. Schwefel, “Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I,” Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968. **ES**
- [9] L. J. Fogel, “Autonomous automata,” *Ind. Res.*, vol. 4, pp. 14–19, 1962. **EP (Evolutionary Programming)**

- [8] H.-P. Schwefel, "Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I," Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968.

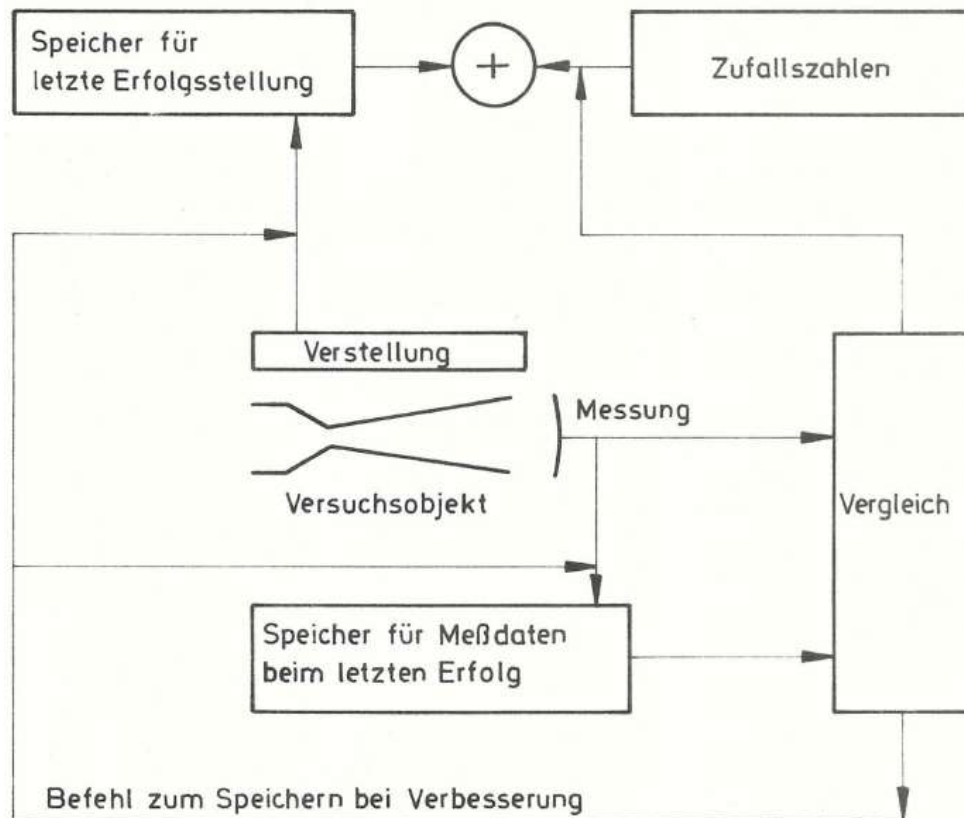


Bild 14 : Verstellbare Düse



Bild 15 : Heißwasserstrahl

- [8] H.-P. Schwefel, "Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I," Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968.

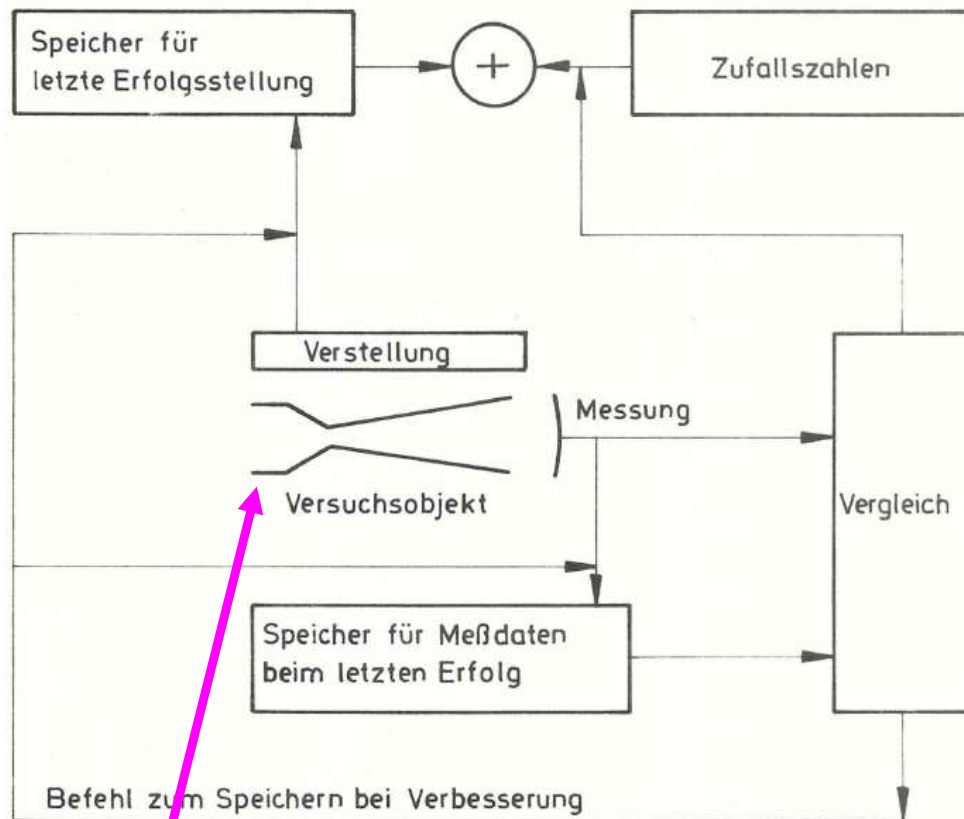


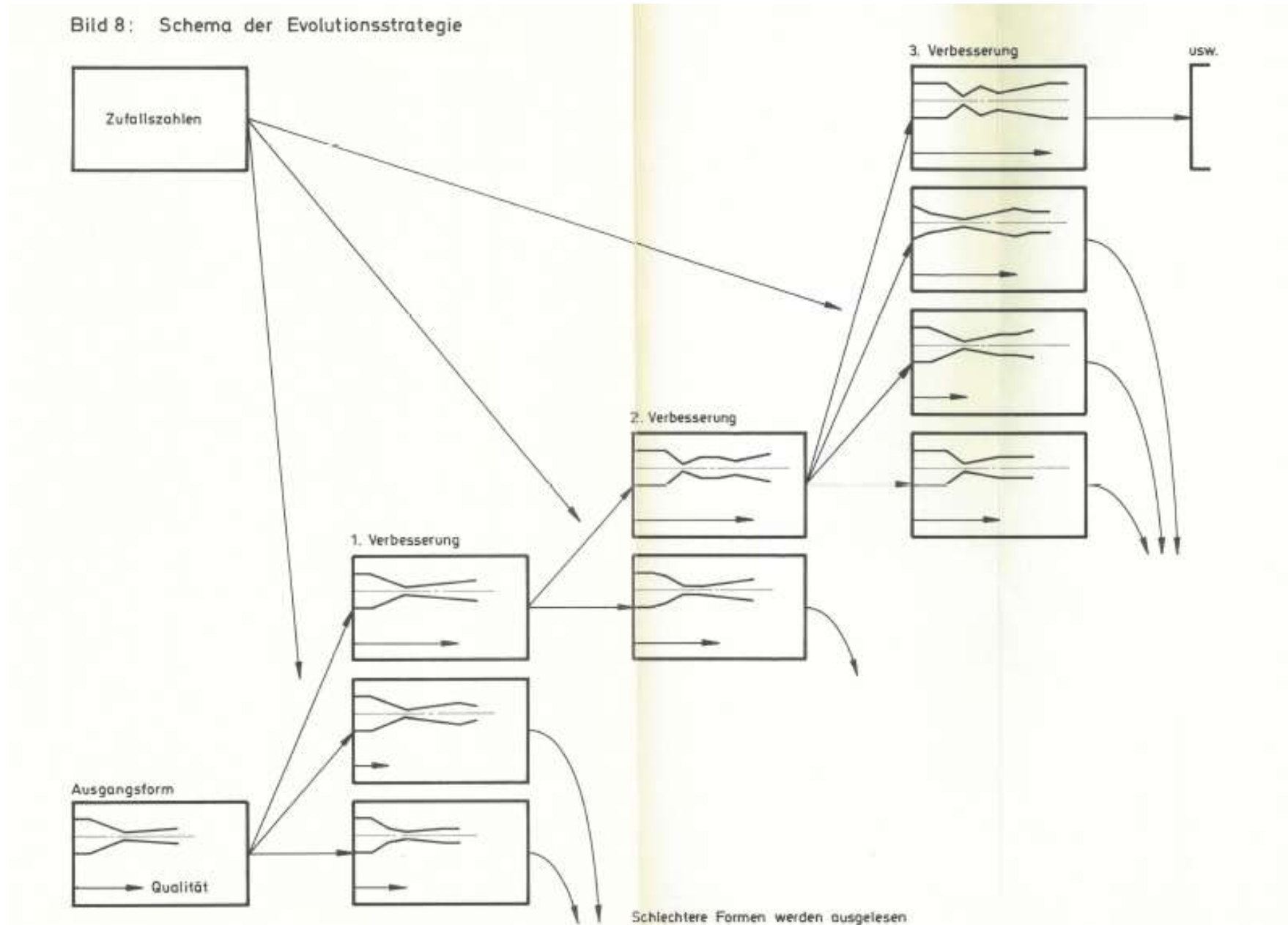
Bild 14 : Verstellbare Düse



Bild 15 : Heißwasserstrahl

Optimization of this shape (a part of a machine)

- [8] H.-P. Schwefel, "Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I," Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968.



- [8] H.-P. Schwefel, "Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I," Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968.

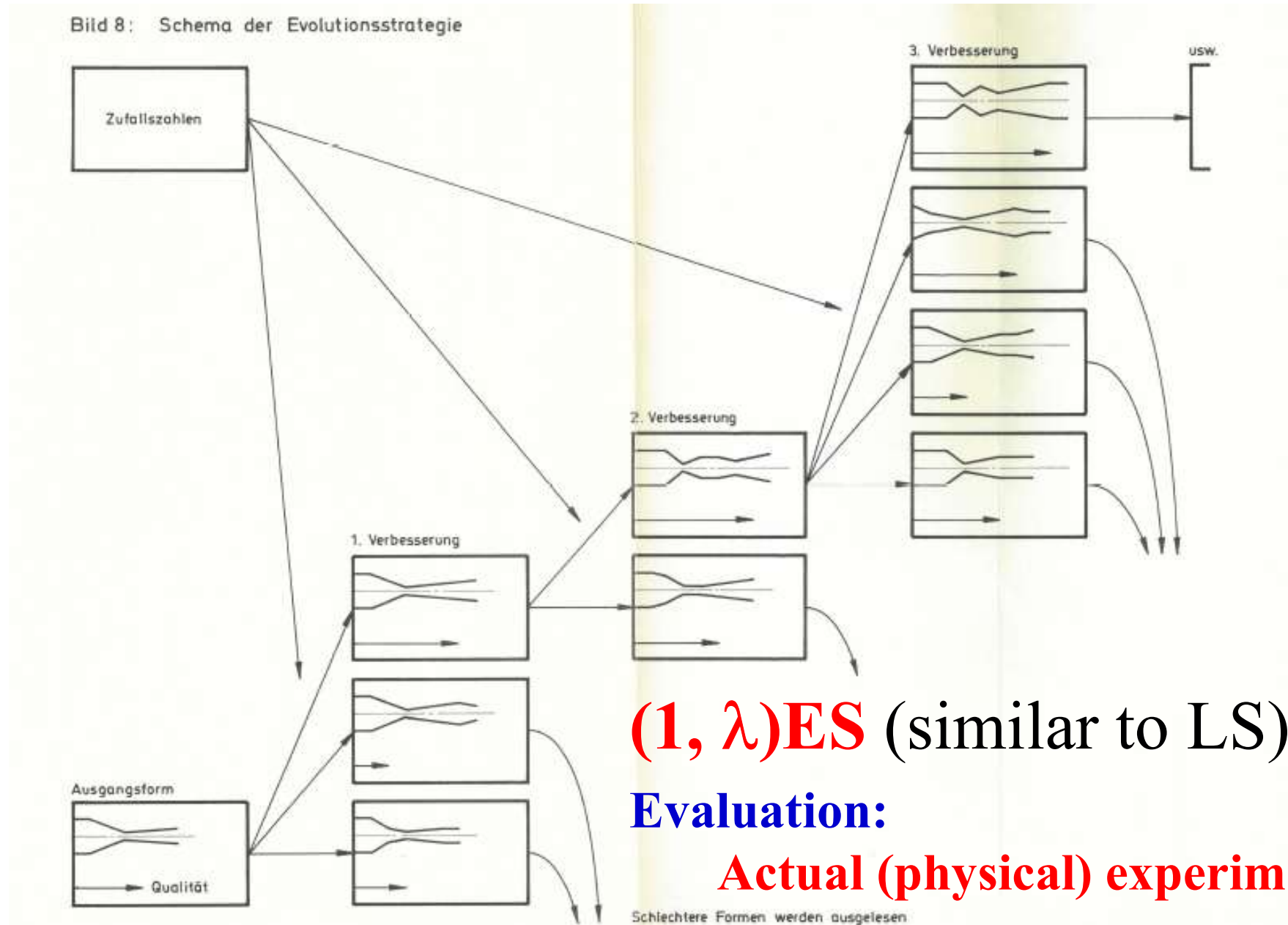
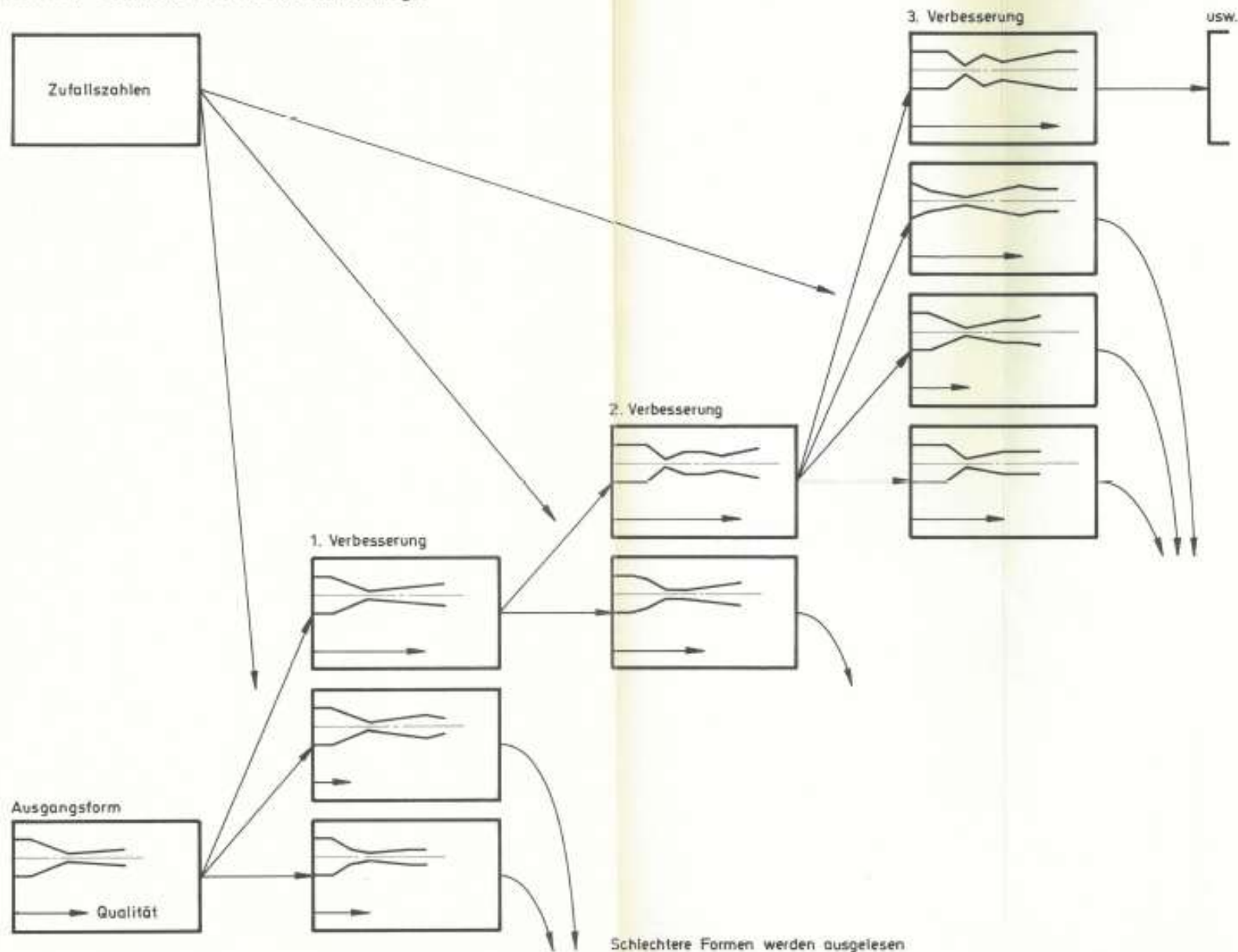


Bild 8: Schema der Evolutionsstrategie



Even now, similar optimization is still actively studied under the name of expensive optimization (simulation- or experimentation-based optimization) where only a small number of solutions can be evaluated (e.g., 50, 100, 200).

History of Evolutionary Computation

Studies in 1960s:

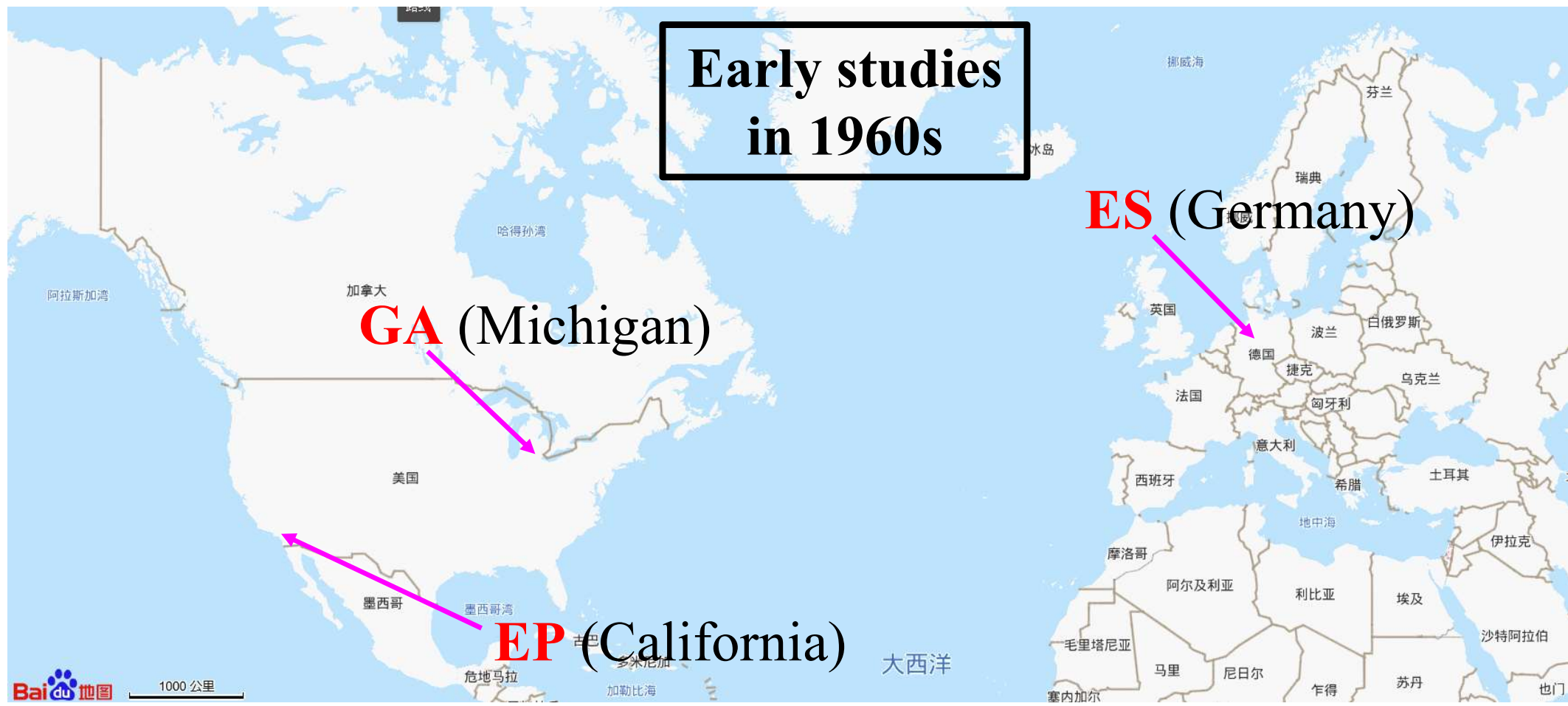
Very weak computer power, some clear advantages,
not well-known until 1990s, currently known as origins of EC.

- [6] J. H. Holland, “Outline for a logical theory of adaptive systems,” *J. Assoc. Comput. Mach.*, vol. 3, pp. 297–314, 1962. **GA**
- [7] I. Rechenberg, “Cybernetic solution path of an experimental problem,” Royal Aircraft Establishment, Library translation No. 1122, Farnborough, Hants., U.K., Aug. 1965. **ES (Evolution Strategy)**
- [8] H.-P. Schwefel, “Projekt MHD-Staustahlrohr: Experimentelle Optimierung einer Zweiphasendüse, Teil I,” Technischer Bericht 11.034/68, 35, AEG Forschungsinstitut, Berlin, Germany, Oct. 1968. **ES**
- [9] L. J. Fogel, “Autonomous automata,” *Ind. Res.*, vol. 4, pp. 14–19, 1962. **EP (Evolutionary Programming)**

ES (Evolution Strategies) I. Rechenberg, H. P. Schwefel
Mutation on real number strings: Function optimization

EP (Evolutionary Programming) J. L. Fogel
Mutation on finite automata (finite state machines)

GA (Genetic Algorithms) J. Holland
Mutation and crossover on binary strings (integer, permutation, tree)



ES (Evolution Strategies) I. Rechenberg, H. P. Schwefel
Mutation on real number strings: Function optimization

EP (Evolutionary Programming) J. L. Fogel
Mutation on finite automata (finite state machines)

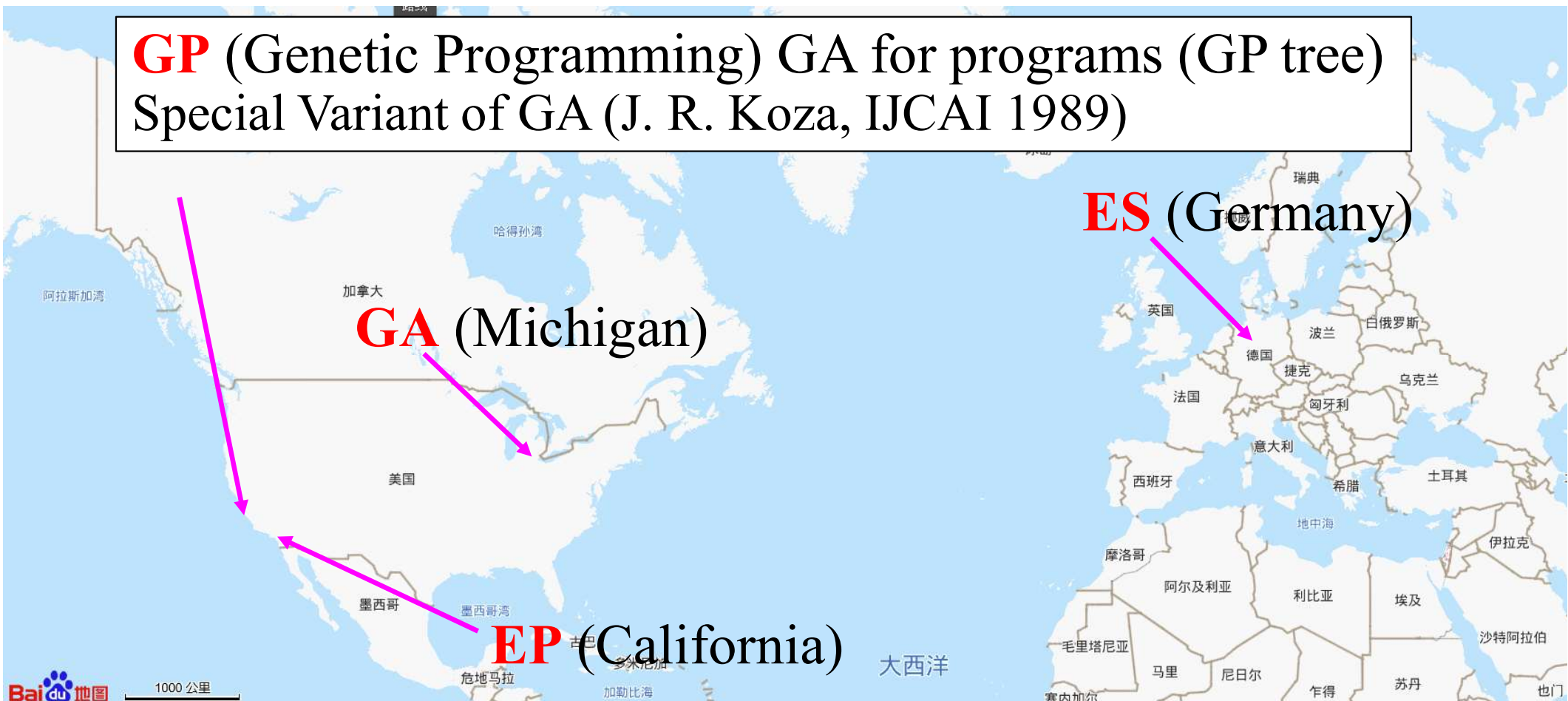
GA (Genetic Algorithms) J. Holland
Mutation and crossover on binary strings (integer, permutation, tree)

GP (Genetic Programming) GA for programs (GP tree)
Special Variant of GA (J. R. Koza, IJCAI 1989)

GA (Michigan)

ES (Germany)

EP (California)



ES (Evolution Strategies) I. Rechenberg, H. P. Schwefel

Mutation on real number strings: Function optimization

EP (Evolutionary Programming) J. L. Fogel

Mutation on finite automata (finite state machines)

GA (Genetic Algorithms) J. Holland

Mutation and crossover on binary strings (integer, permutation, tree)

GP (Genetic Programming) GA for programs (GP tree)
Special Variant of GA (J. R. Koza, IJCAI 1989)

GA (Michigan)

ES (Germany)

EP (Calif)

1990s: Swarm Intelligence

ACO: Ant colony optimization

PSO: Particle swarm optimization

Genetic Algorithms

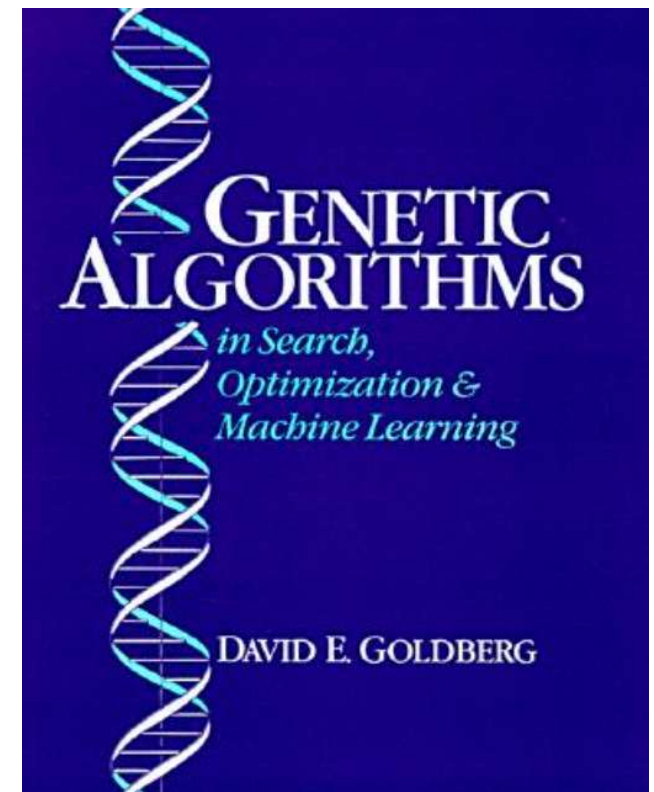


David E. Goldberg

Founder & President, Big Beacon & Professor Emeritus,
Verified email at threejoy.com

Genetic Algorithm Pioneer...

**Genetic Algorithms in Search,
Optimization & Machine Learning (1989)**



Genetic Algorithms

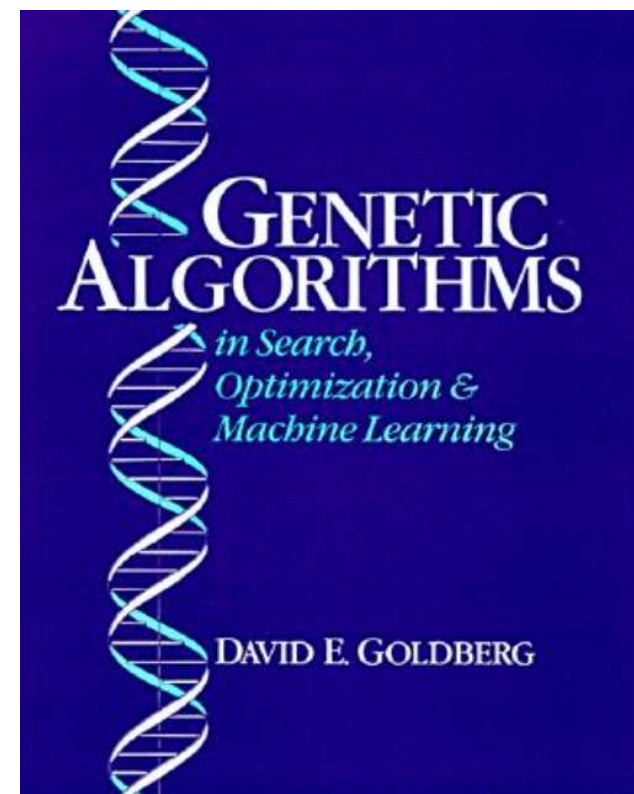


David E. Goldberg

Founder & President, Big Beacon & Professor Emeritus,
Verified email at threejoy.com

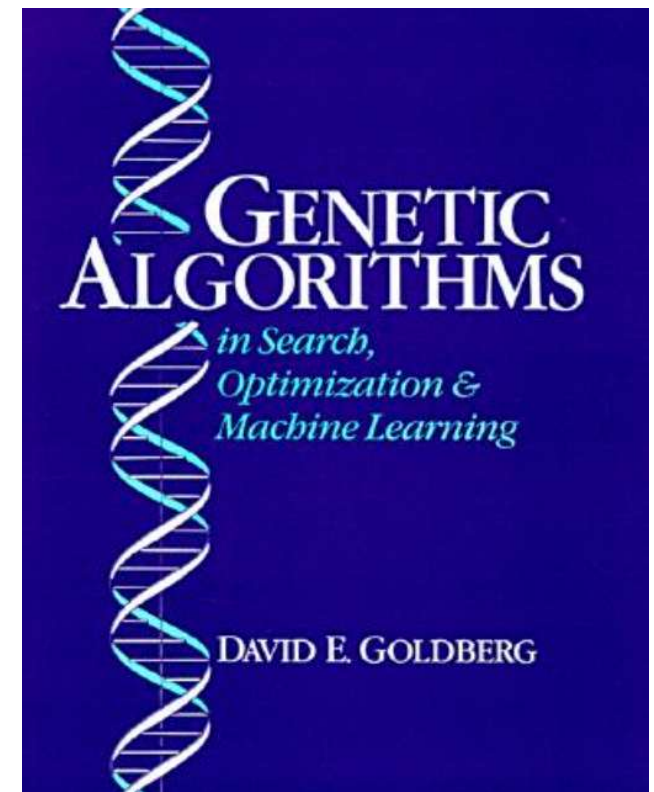
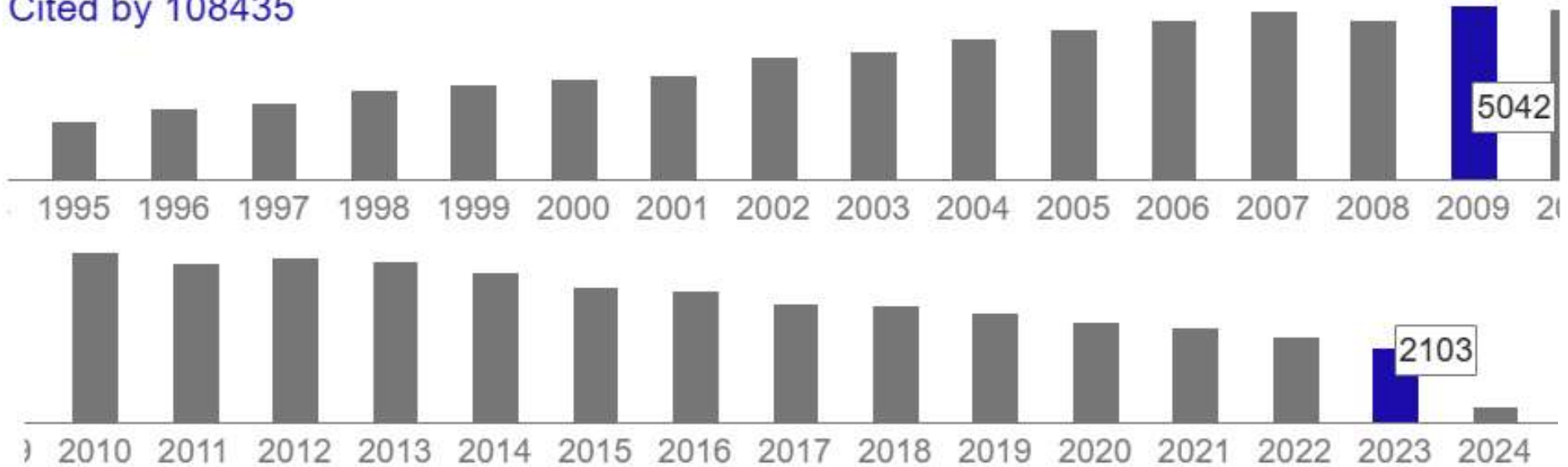
Genetic Algorithm Pioneer...

**Genetic Algorithms in Search,
Optimization & Machine Learning (1989)**



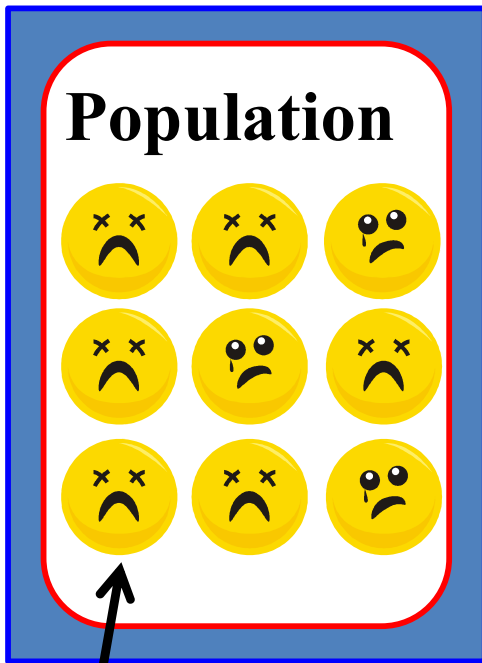
Genetic Algorithms

Cited by 108435



Basic Idea of Evolutionary Computation

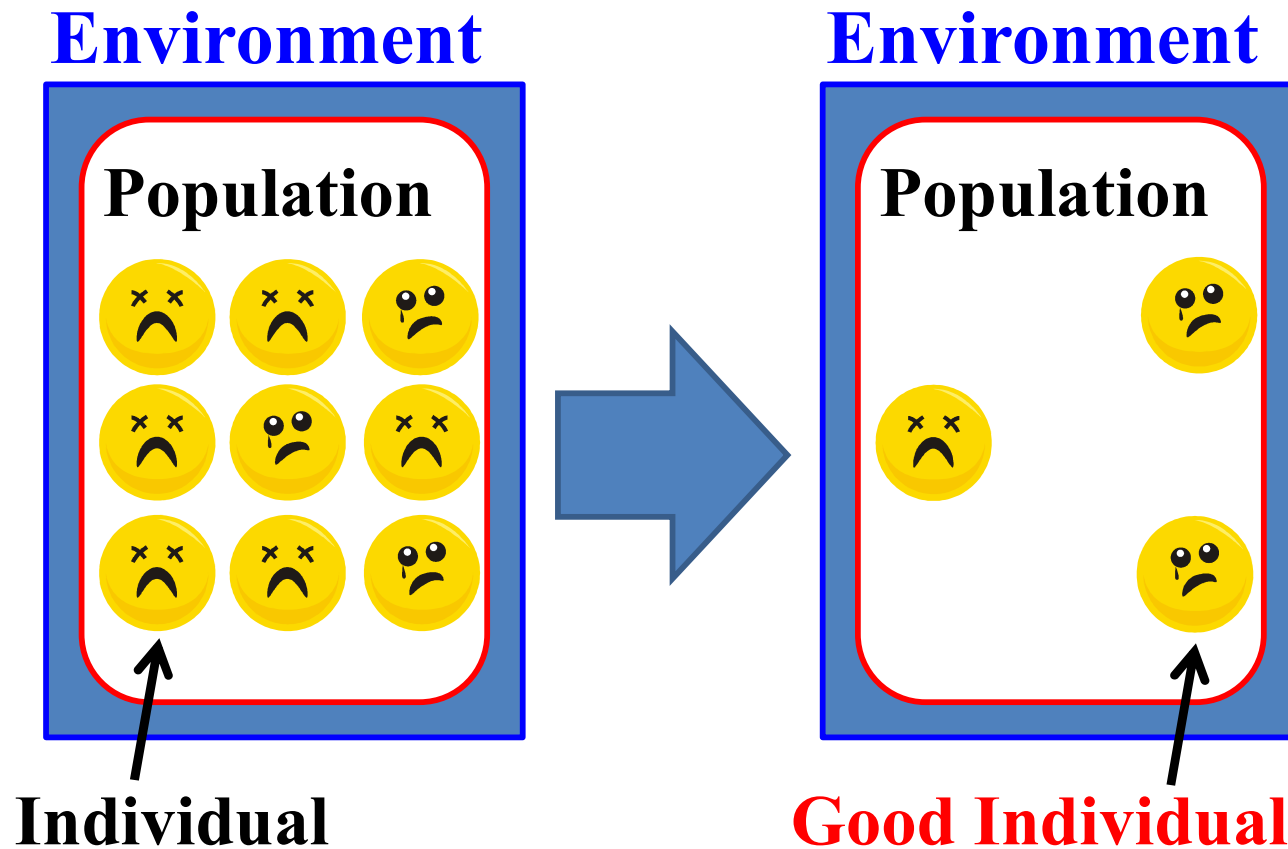
Environment



Individual

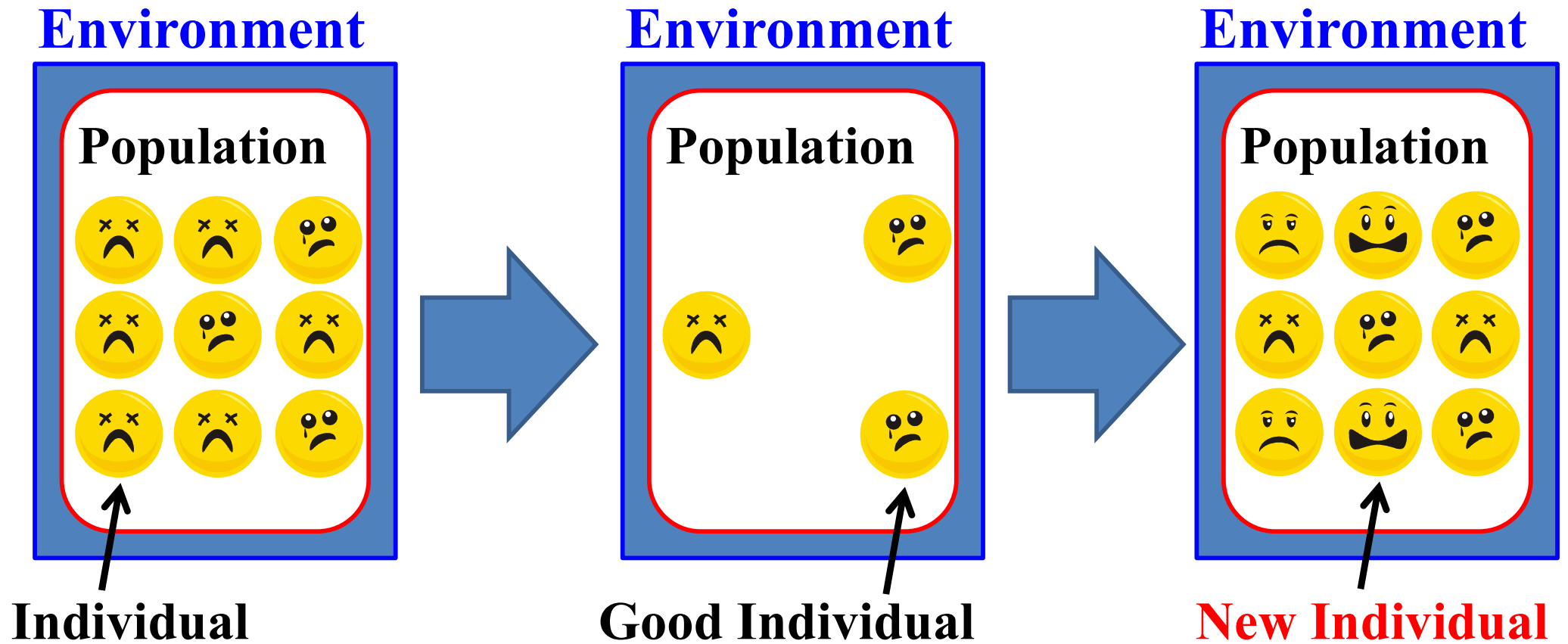
- (1) A population of individuals is randomly generated.
- (2) Each individual is evaluated in the environment.

Basic Idea of Evolutionary Computation



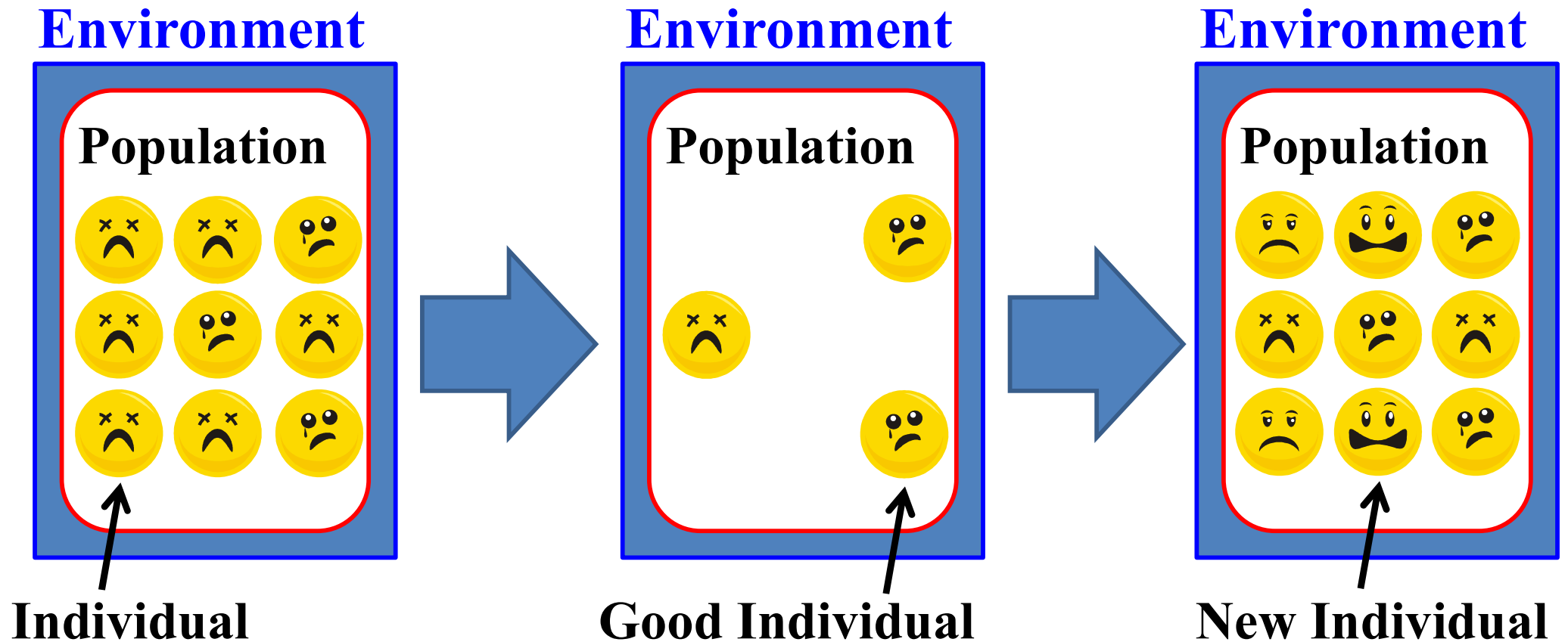
- (1) A population of individuals is randomly generated.
- (2) Each individual is evaluated in the environment.
- (3) **Good individuals survive probabilistically.**

Basic Idea of Evolutionary Computation



- (1) A population of individuals is randomly generated.
- (2) Each individual is evaluated in the environment.
- (3) Good individuals survive probabilistically.
- (4) **New individuals are generated from the good individuals.**

Basic Idea of Evolutionary Computation



These steps (2)-(4) are iterated many times.

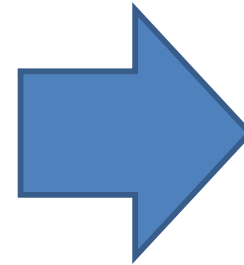
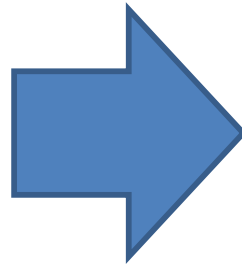
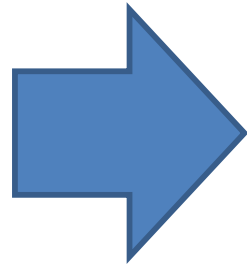
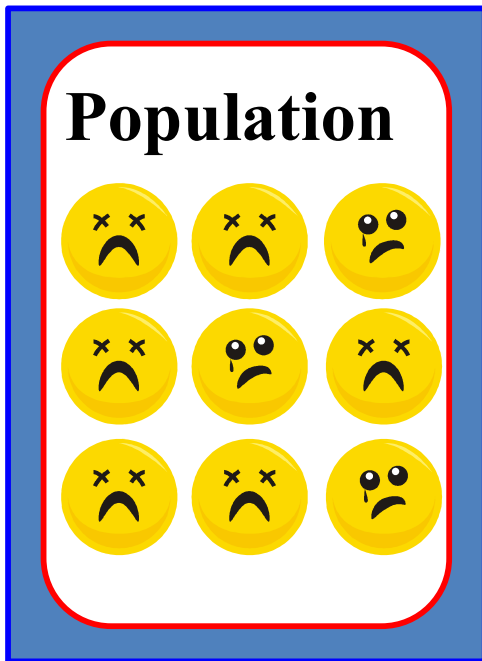
(2) Each individual is evaluated in the environment.

(3) Good individuals survive probabilistically.

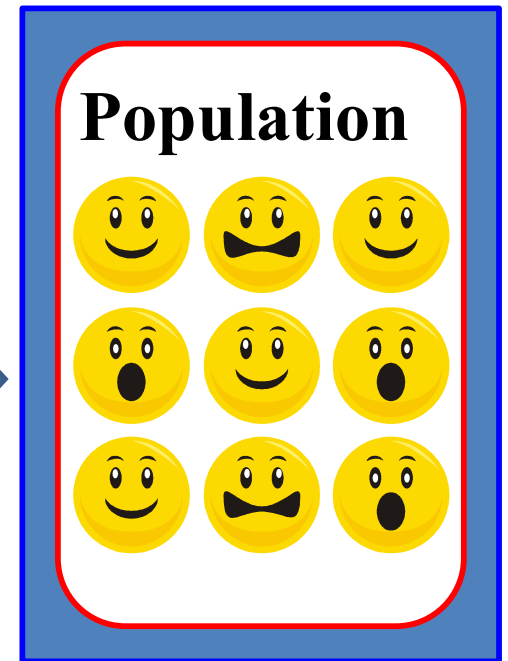
(4) New individuals are generated from the good individuals.

Basic Idea of Evolutionary Computation

Environment



Environment



These steps (2)-(4) are iterated many times.

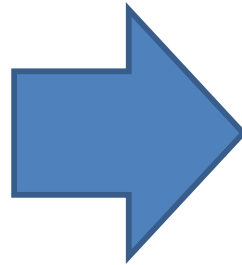
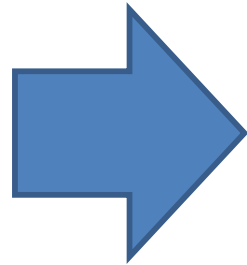
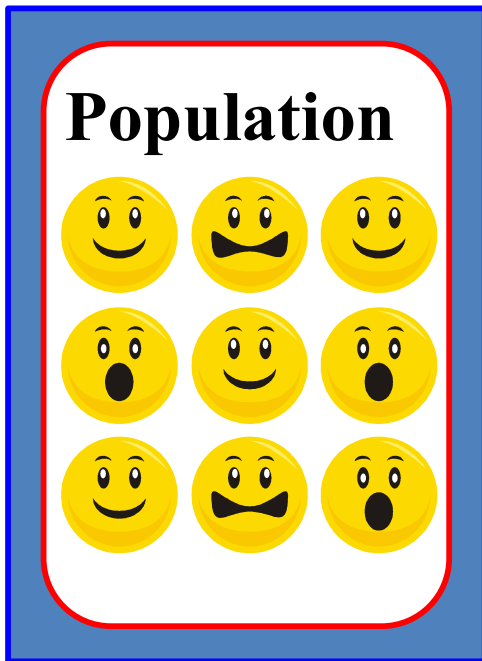
(2) Each individual is evaluated in the environment.

(3) Good individuals survive.

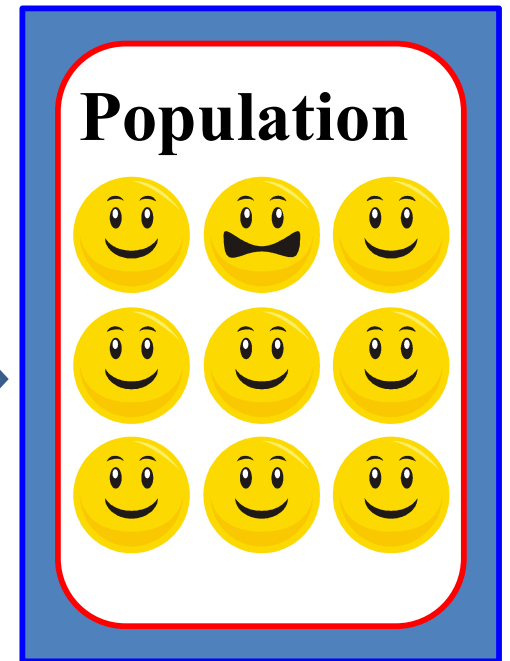
(4) New individuals are generated from the good individuals.

Basic Idea of Evolutionary Computation

Environment



Environment



These steps (2)-(4) are iterated many times.

(2) Each individual is evaluated in the environment.

(3) Good individuals survive.

(4) New individuals are generated from the good individuals.

After many generations, we may have good solutions.

Application of Evolutionary Computation

Design of High Speed Train

N700系フォトギャラリー



印刷



閉じる

東海旅客鉄道株式会社

©Central Japan Railway Company. All rights reserved.

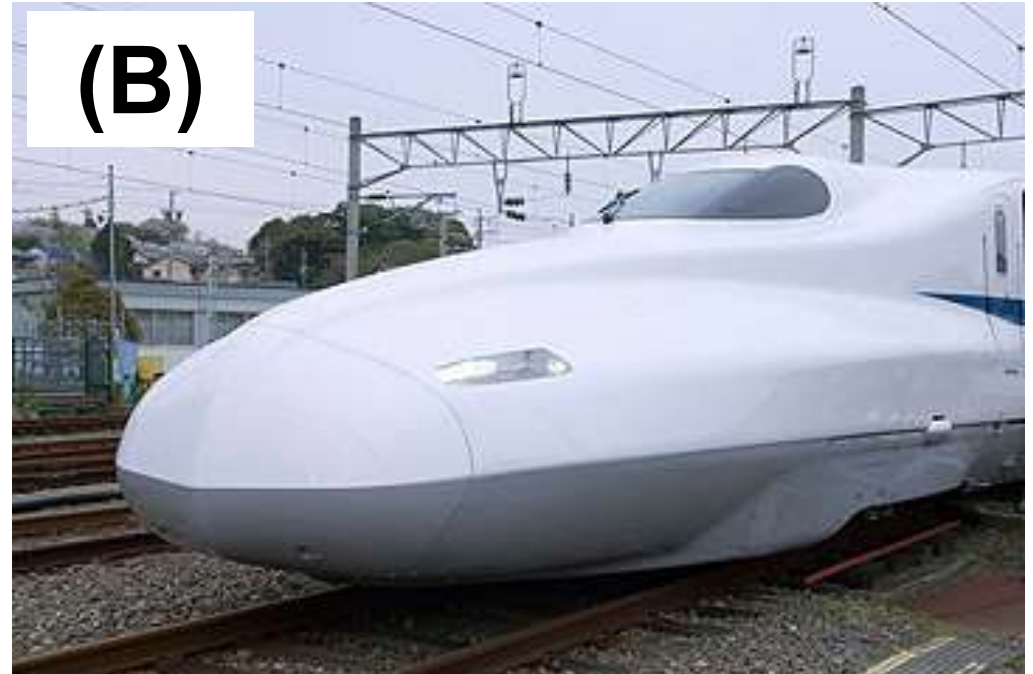
Two Types of High Speed Trains



(A)

**(A) Old Design by
Human Experts.**

Old design looks better.



(B)

**(B) New Design by
Genetic Algorithms.**

New design looks strange.

High Speed Trains in Japan

N700系フォトギャラリー



New design by GA looks strange

Old design by human experts looks nice !

東海旅客鉄道株式会社

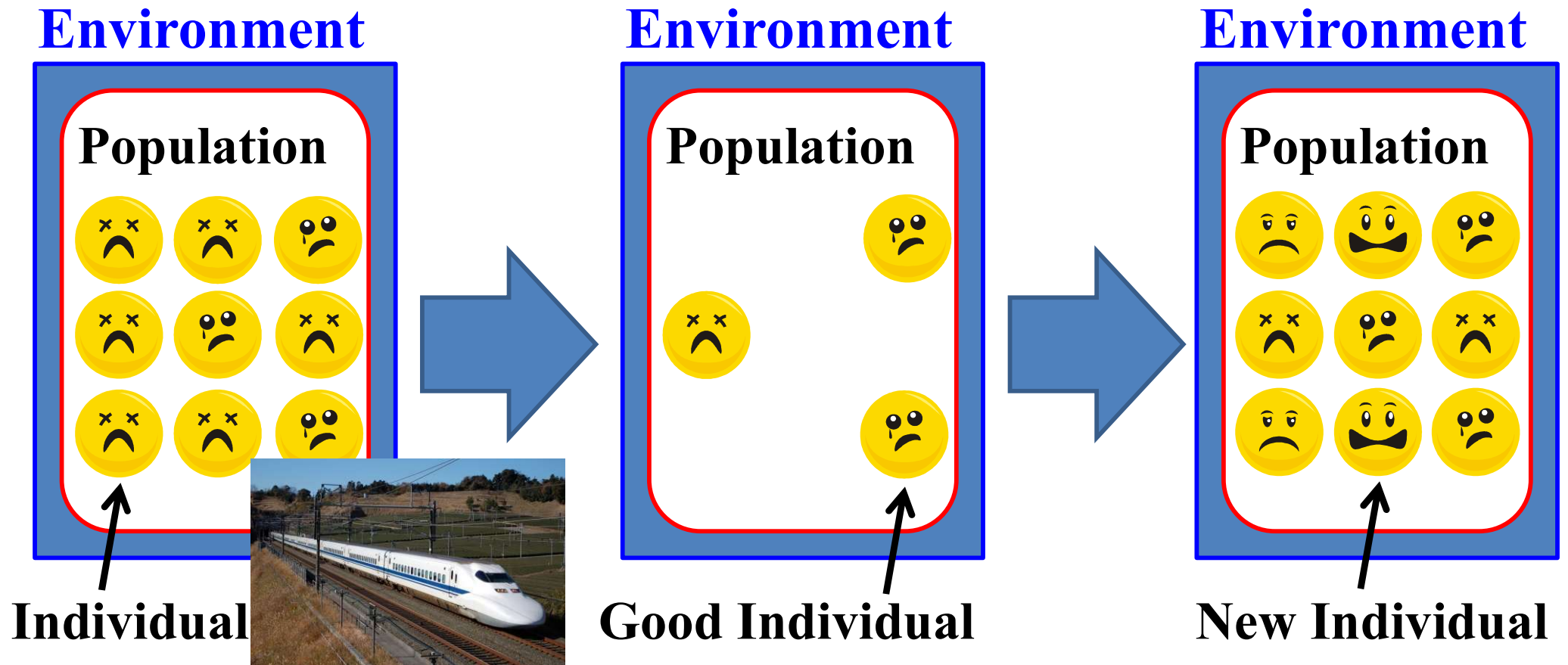
©Central Japan Railway Company. All rights reserved.

印刷



閉じる

Basic Idea of Evolutionary Computation



These steps (2)-(4) are iterated many times.

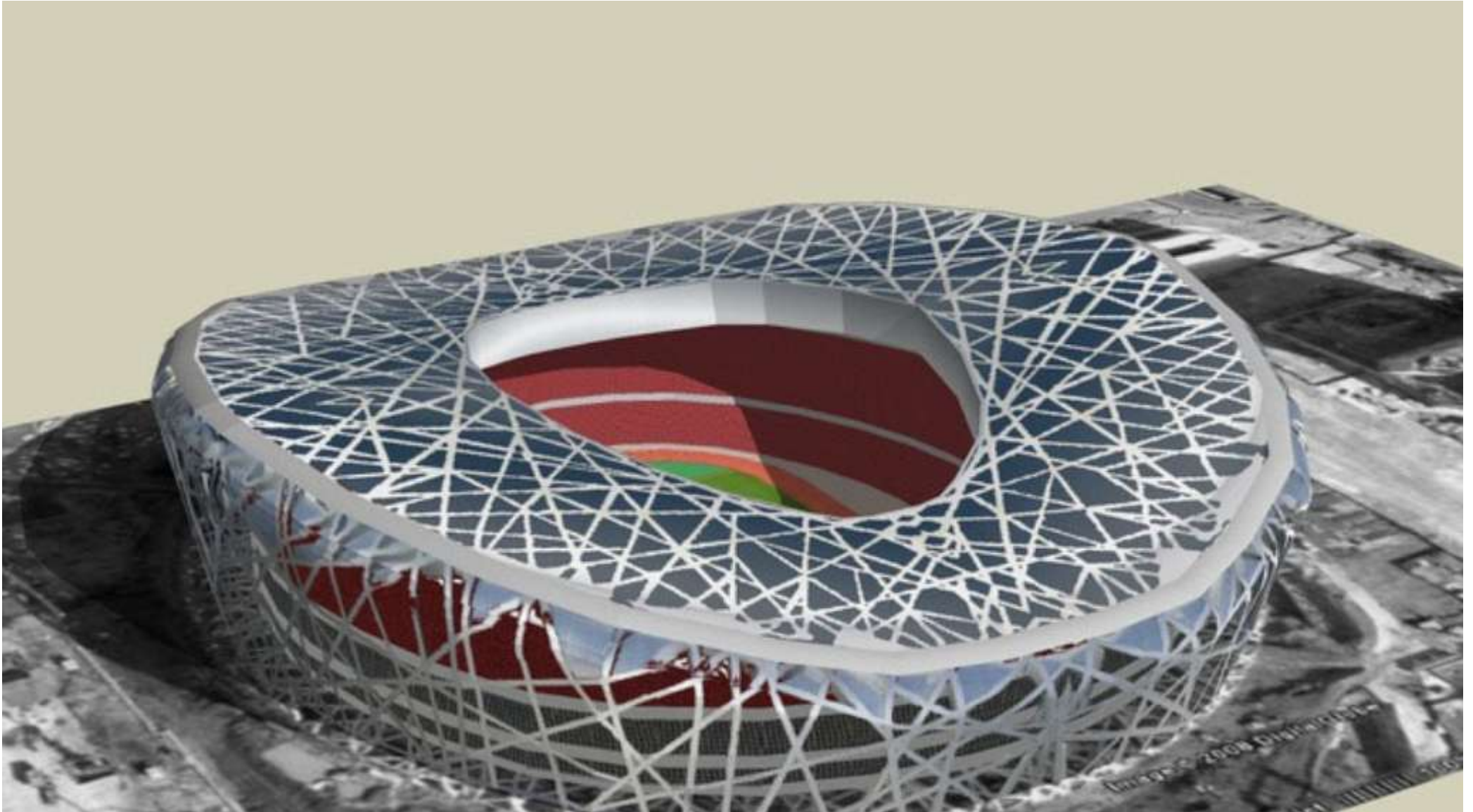
(2) Each individual is evaluated in the environment.

(3) Good individuals survive probabilistically.

(4) New individuals are generated from the good individuals.

Applications of Evolutionary Computation

Construction Planning



<http://sketchup3dconstruction.com/skp/warehouse/stadiums/beijing-national-stadium.html>

Applications of Evolutionary Computation

Construction Planning

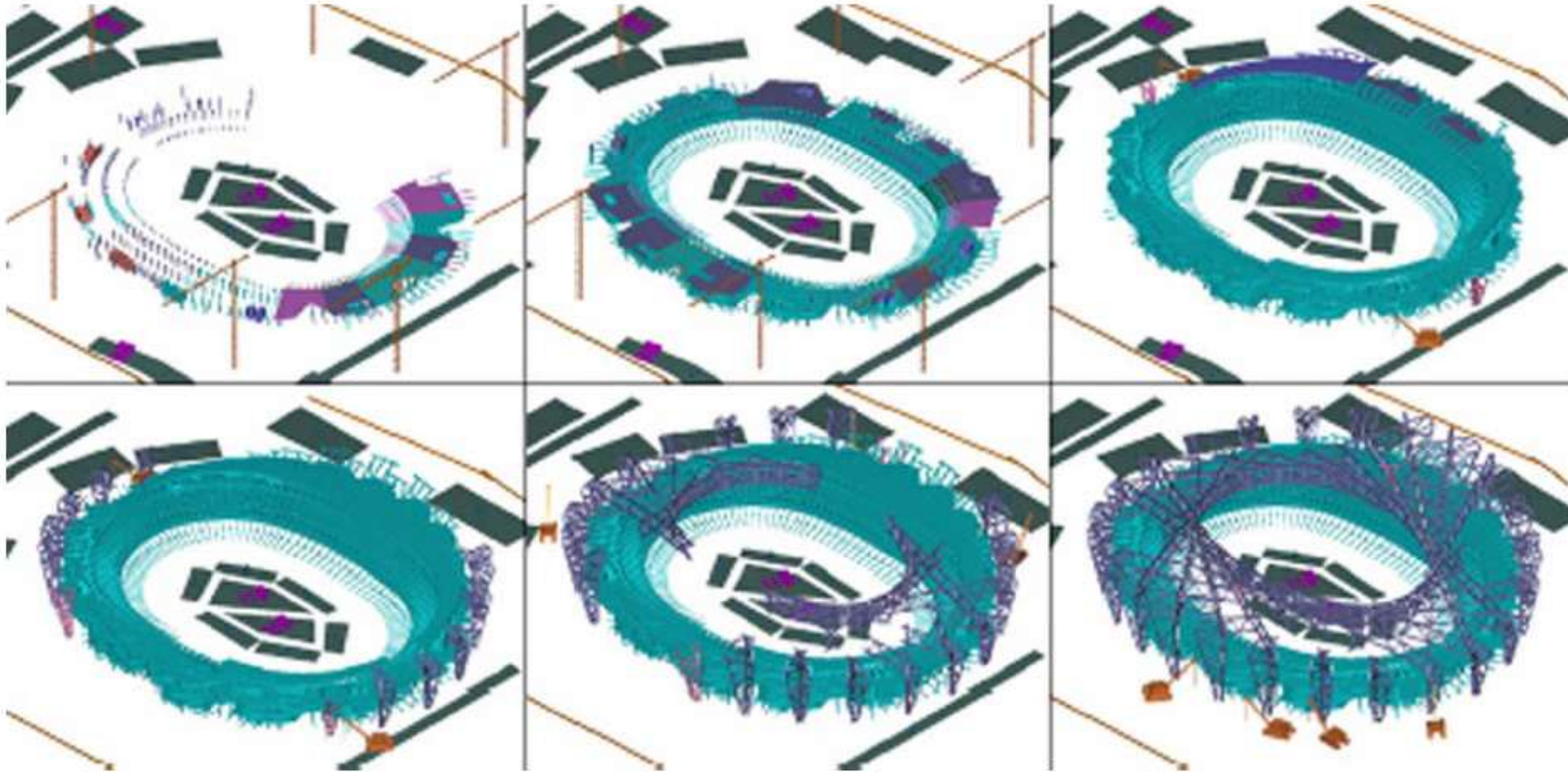


Fig. 4 4D screenshots of construction plan

J.ZHANG, Y. ZHANG, Z. HU, and M.LU: Construction Management Utilizing 4D CAD and Operations Simulation Methodologies, *TSINGHUA SCIENCE AND TECHNOLOGY*, Vol. 13, No. S1, pp. 241-247. Oct 2008.

Design of Rule-Based Systems

Environment

Population

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

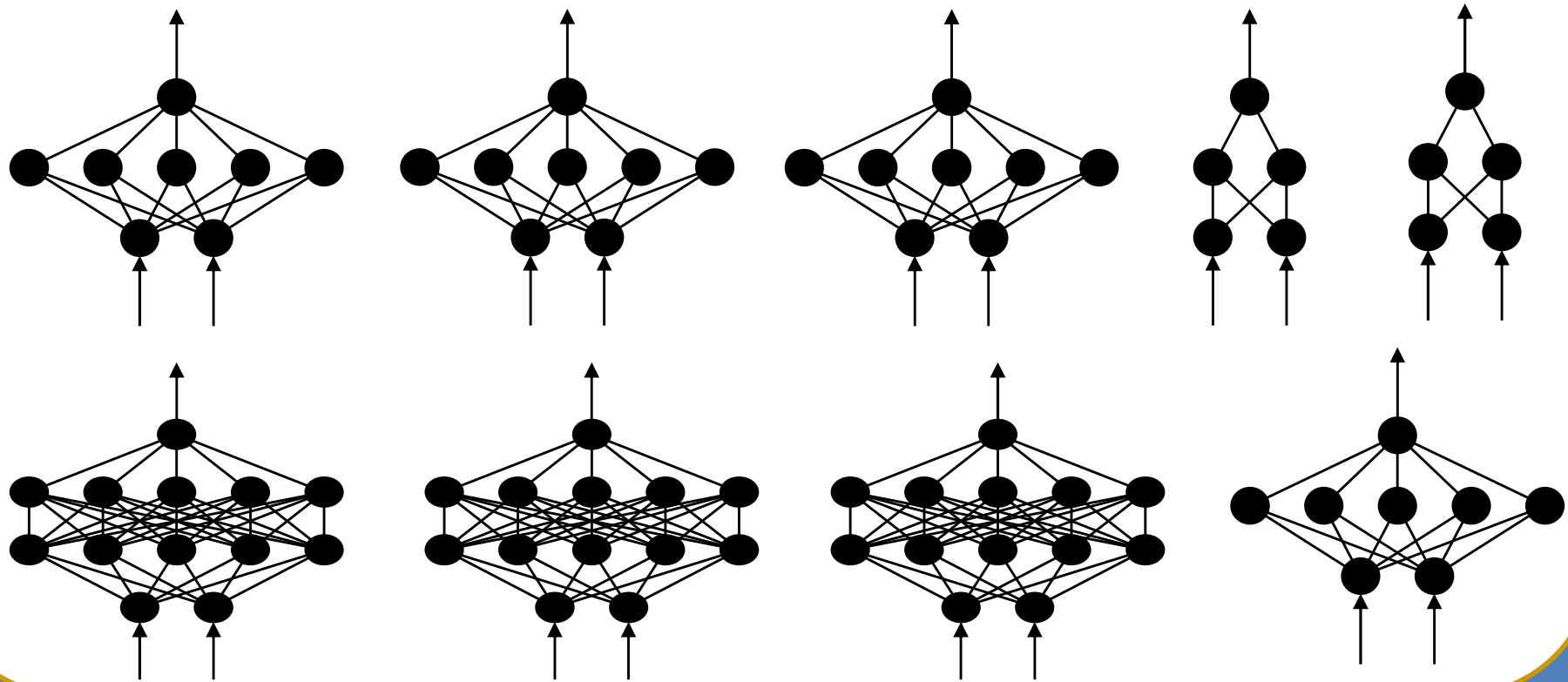
Individual = Rule-Based System ()

If ... Then ...
If ... Then ...
If ... Then ...
If ... Then ...

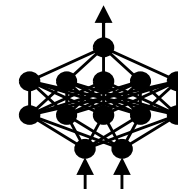
Design of Neural Networks

Environment

Population



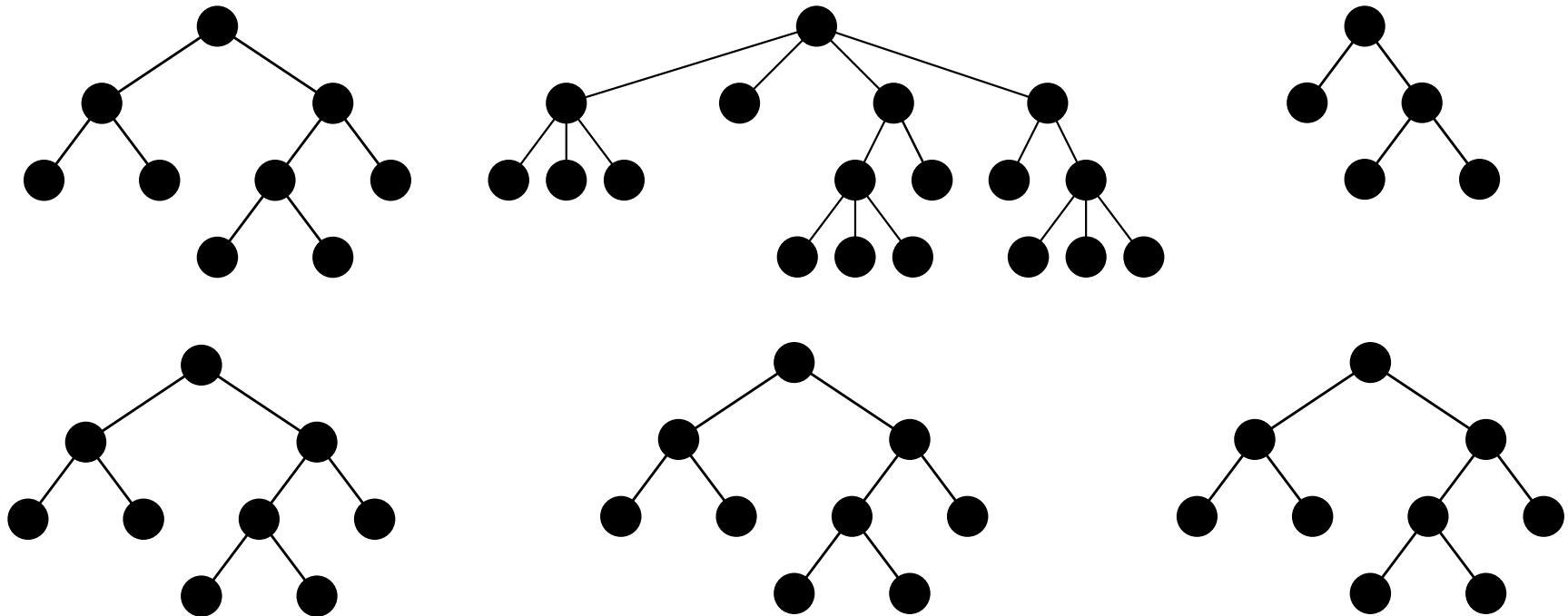
Individual = Neural Network (



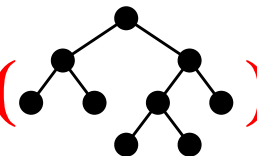
Design of Decision Trees

Environment

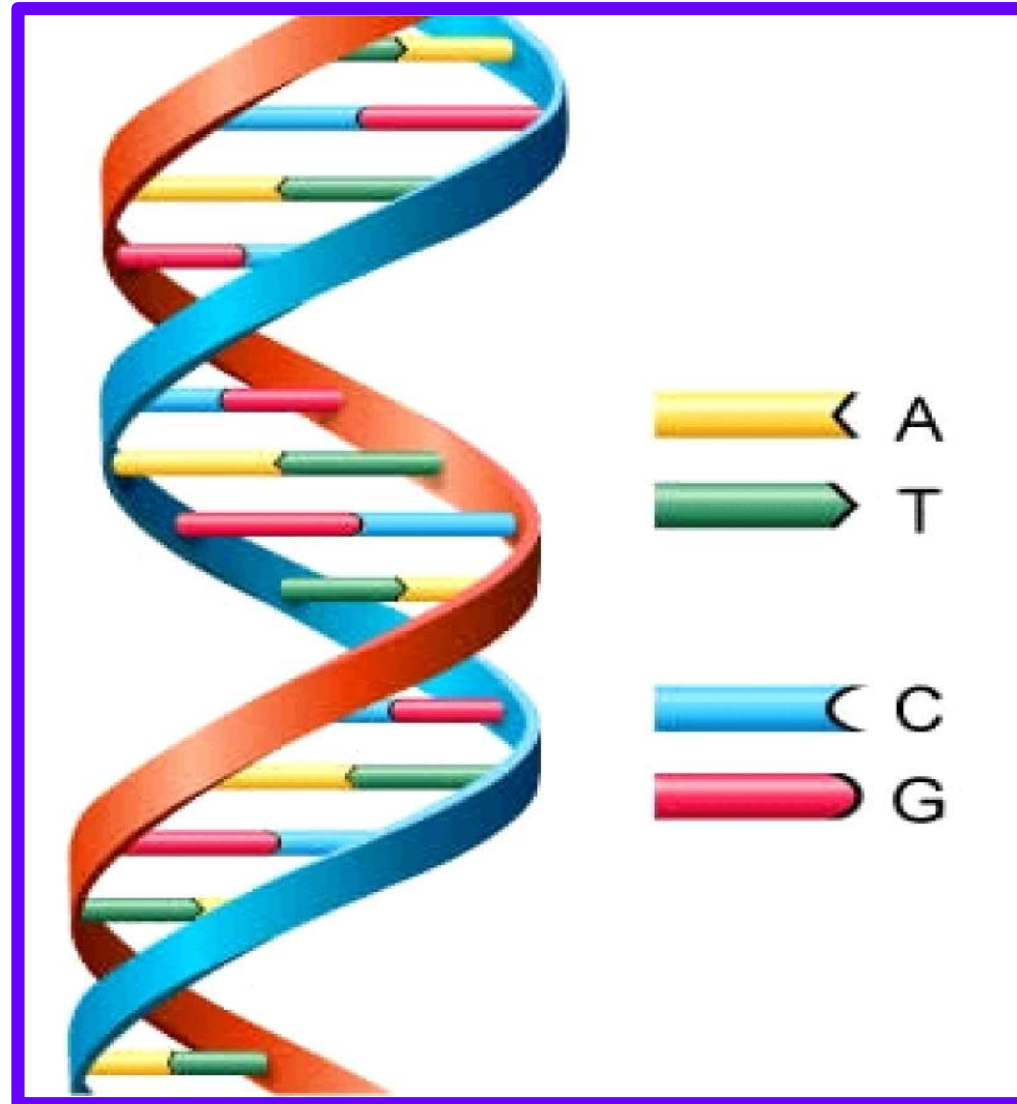
Population



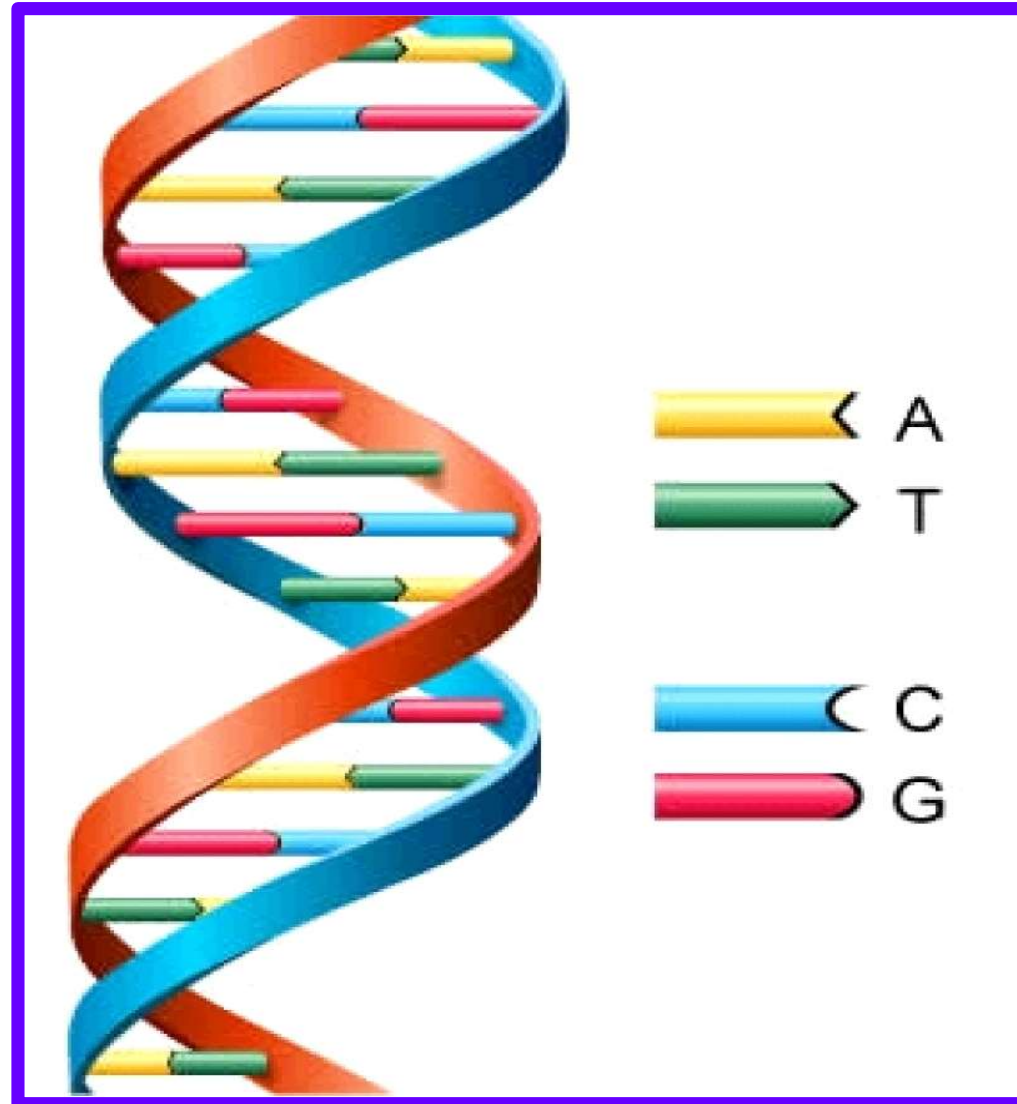
Individual = Decision Tree (



How to Represent Each Solution for Computer Simulation of Evolution



Any string can be used
depending on the problem at hand



Any string can be used!

Knapsack Problem:

Binary String

1	1	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---	---

Permutation

3	4	2	7	1	8	9	6	5
---	---	---	---	---	---	---	---	---

Random Key

0.23	0.29	0.19	0.59	0.13	0.79	0.91	0.53	0.38
------	------	------	------	------	------	------	------	------

TSP, Flowshop Scheduling:

Permutation

3	4	2	7	1	8	9	6	5
---	---	---	---	---	---	---	---	---

Random Key

0.23	0.29	0.19	0.59	0.13	0.79	0.91	0.53	0.38
------	------	------	------	------	------	------	------	------

Function Optimization:

Real number string

25.297	123.45	92.834
--------	--------	--------

Binary String

1 1 1 0 1 0 0 0 0 1 1 1	0 1 0 0 0 0 1 0 0 1 1 0	0 0 0 1 1 1 0 1 0 0 0 0
-------------------------	-------------------------	-------------------------

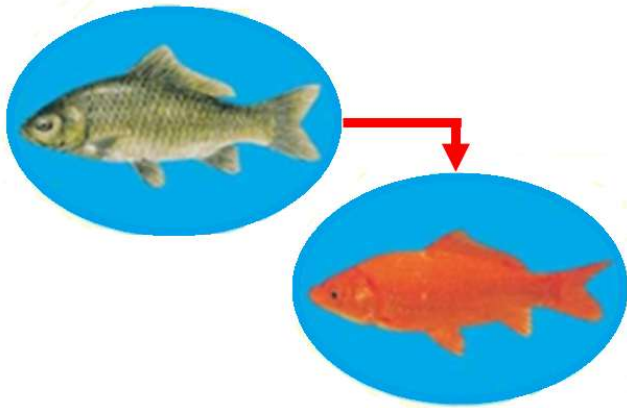
x_1

x_2

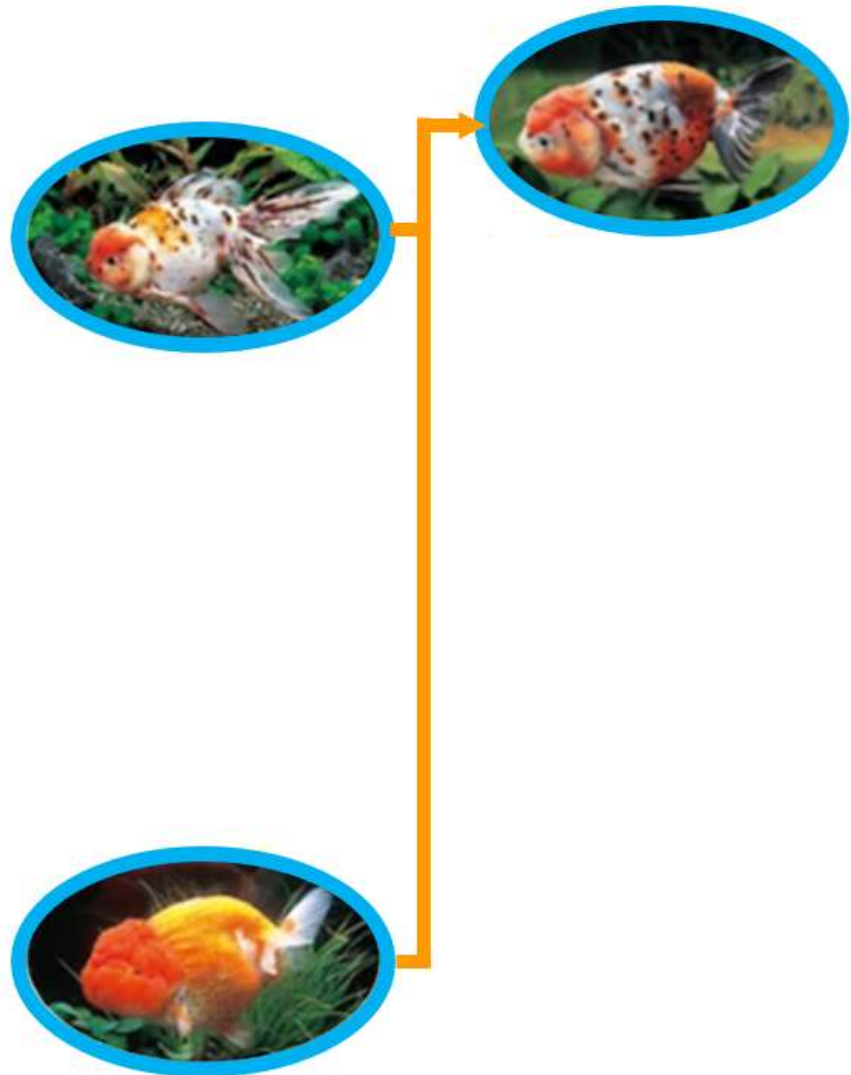
x_3

How to generate new solutions

Mutation

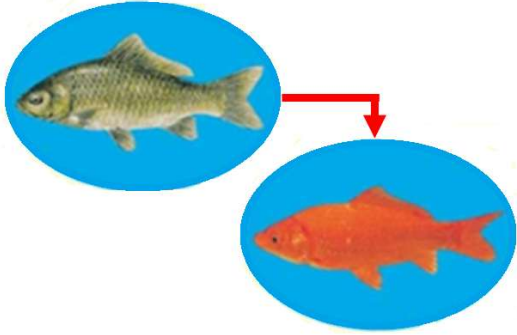


Crossover



Mutation

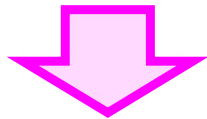
Random change of a part of a string



Random change of a part of a string

(Each value has the same mutation probability, e.g., $1/n$, $2/n$)

1	1	1	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---

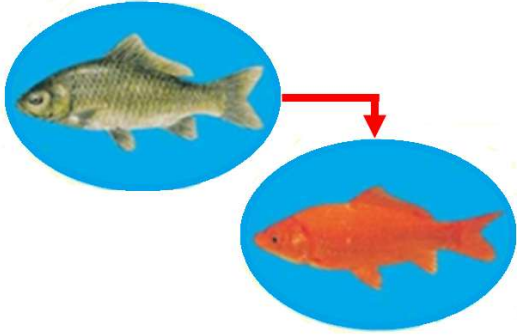


1	0	1	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---

Binary String

Mutation

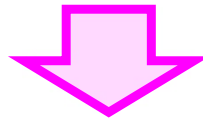
Random change of a part of a string



Random change of a part of a string

(Each value has the same mutation probability, e.g., $1/n$, $2/n$)

23	45	21	12	0	12	23	0	58	23
----	----	----	----	---	----	----	---	----	----

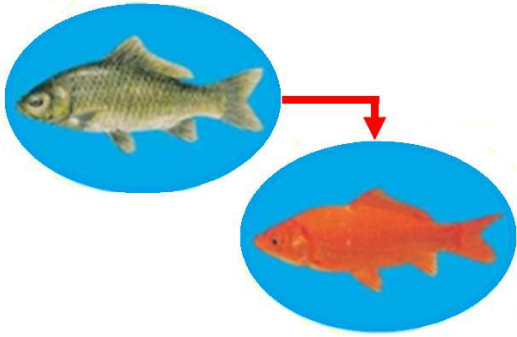


Integer String

23	45	21	12	0	12	23	0	26	23
----	----	----	----	---	----	----	---	----	----

Mutation

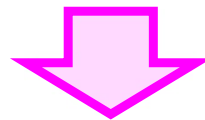
Random change of a part of a string



Random change of a part of a string

(Each value has the same mutation probability, e.g., $1/n$, $2/n$)

23.42	45.20	21.45	12.09	0.00	12.14	23.43	0.00	58.98	23.12
-------	-------	-------	-------	------	-------	-------	------	-------	-------



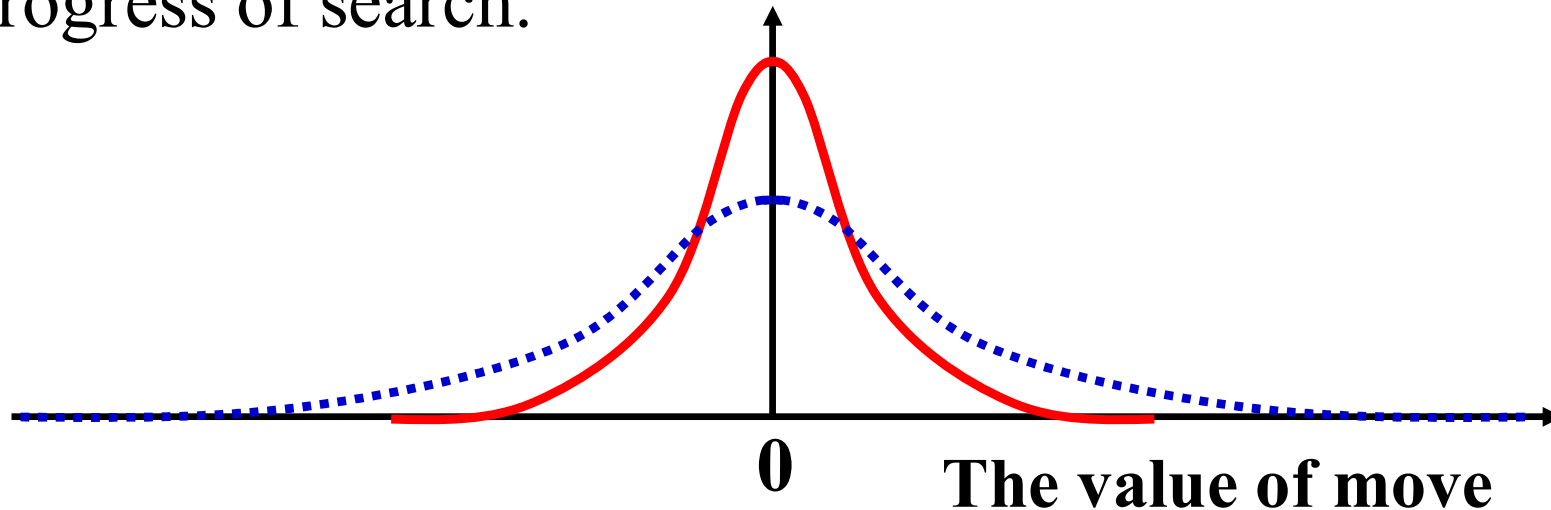
Real Number String

23.42	67.13	21.45	12.09	0.00	12.14	23.43	15.34	58.98	23.12
-------	-------	-------	-------	------	-------	-------	-------	-------	-------

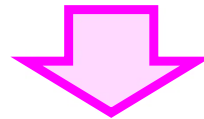
Mutation

For real number strings, a distribution can be used.

The spread of the distribution is a parameter, which can be a fixed parameter or an automatically adjustable parameter during the progress of search.



23.42	45.20	21.45	12.09	0.00	12.14	23.43	0.00	58.98	23.12
-------	-------	-------	-------	------	-------	-------	------	-------	-------

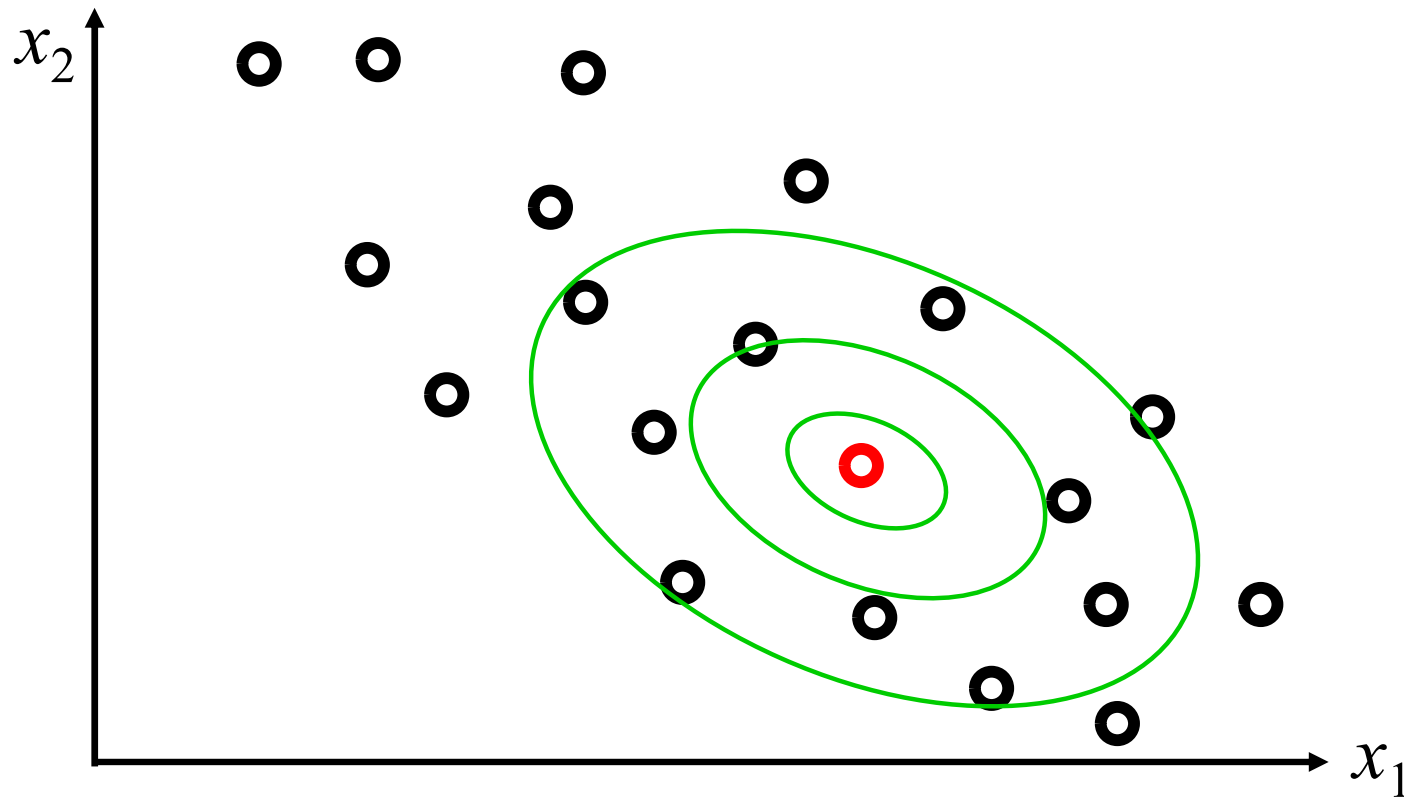


23.42	49.13	21.45	12.09	0.00	12.14	23.43	3.34	58.98	23.12
-------	-------	-------	-------	------	-------	-------	------	-------	-------

Mutation

A multi-dimensional distribution can be also used.

The distribution is usually automatically adjusted by the solutions in the current population (and the previous populations).



- All values of a solution are mutated.
- A mutation probability is assigned to each solution (not each value).

Mutation for Permutation Strings

Random change does not generate a permutation.

7	6	1	5	2	4	0	8	9	3
---	---	---	---	---	---	---	---	---	---



7	6	9	5	2	4	0	8	9	3
---	---	---	---	---	---	---	---	---	---

Two “9” and no “1”.

Mutation for Permutation Strings

A neighborhood structure is needed.

Adjacent two-position change

7	6	1	5	2	4	0	8	9	3
---	---	---	---	---	---	---	---	---	---



7	6	5	1	2	4	0	8	9	3
---	---	---	---	---	---	---	---	---	---

Two-position change

7	6	1	5	2	4	0	8	9	3
---	---	---	---	---	---	---	---	---	---

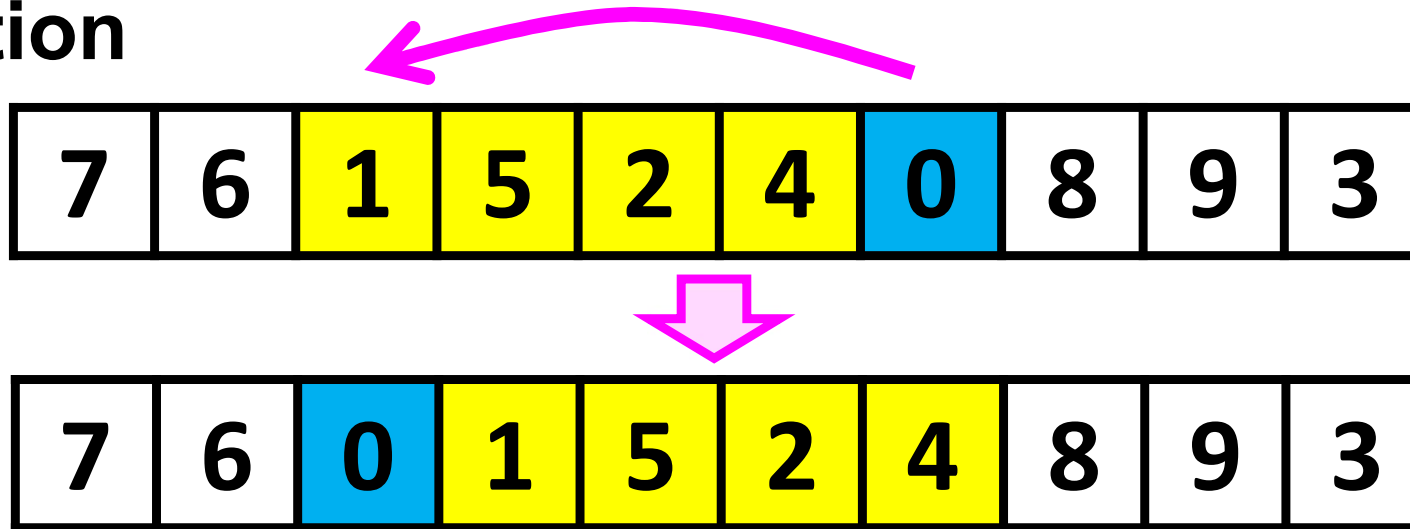


7	8	1	5	2	4	0	6	9	3
---	---	---	---	---	---	---	---	---	---

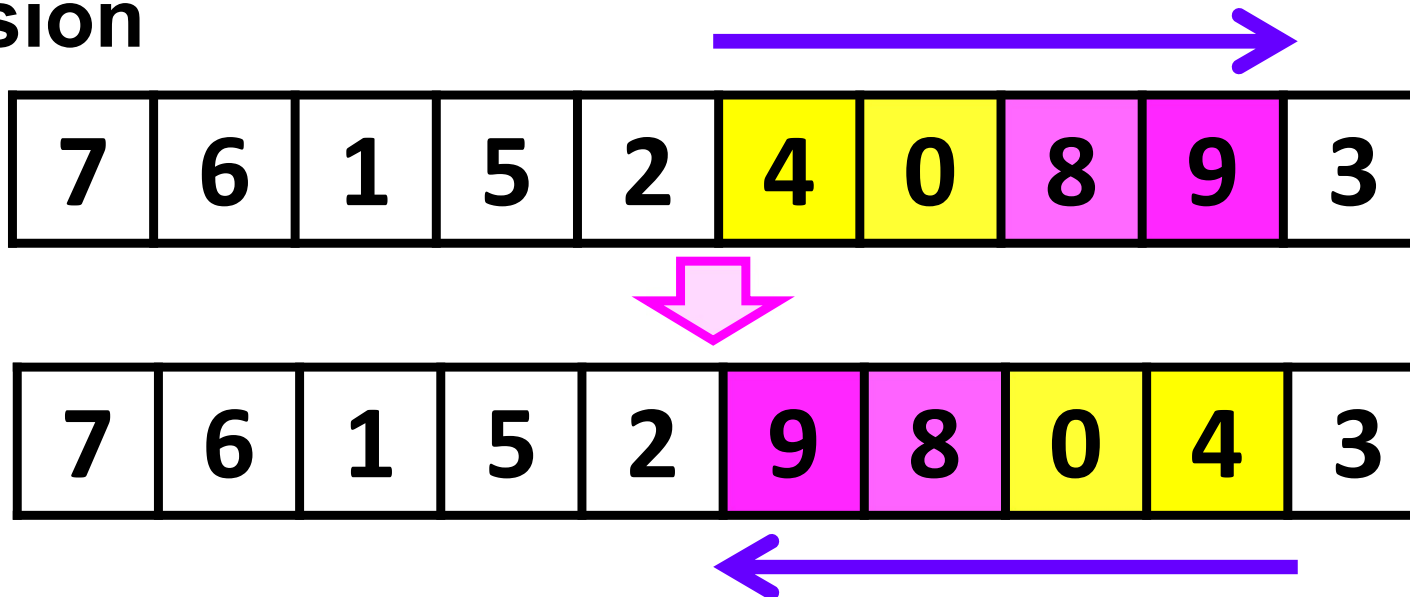
Mutation for Permutation Strings

A mutation probability is assigned to each string.

Insertion



Inversion

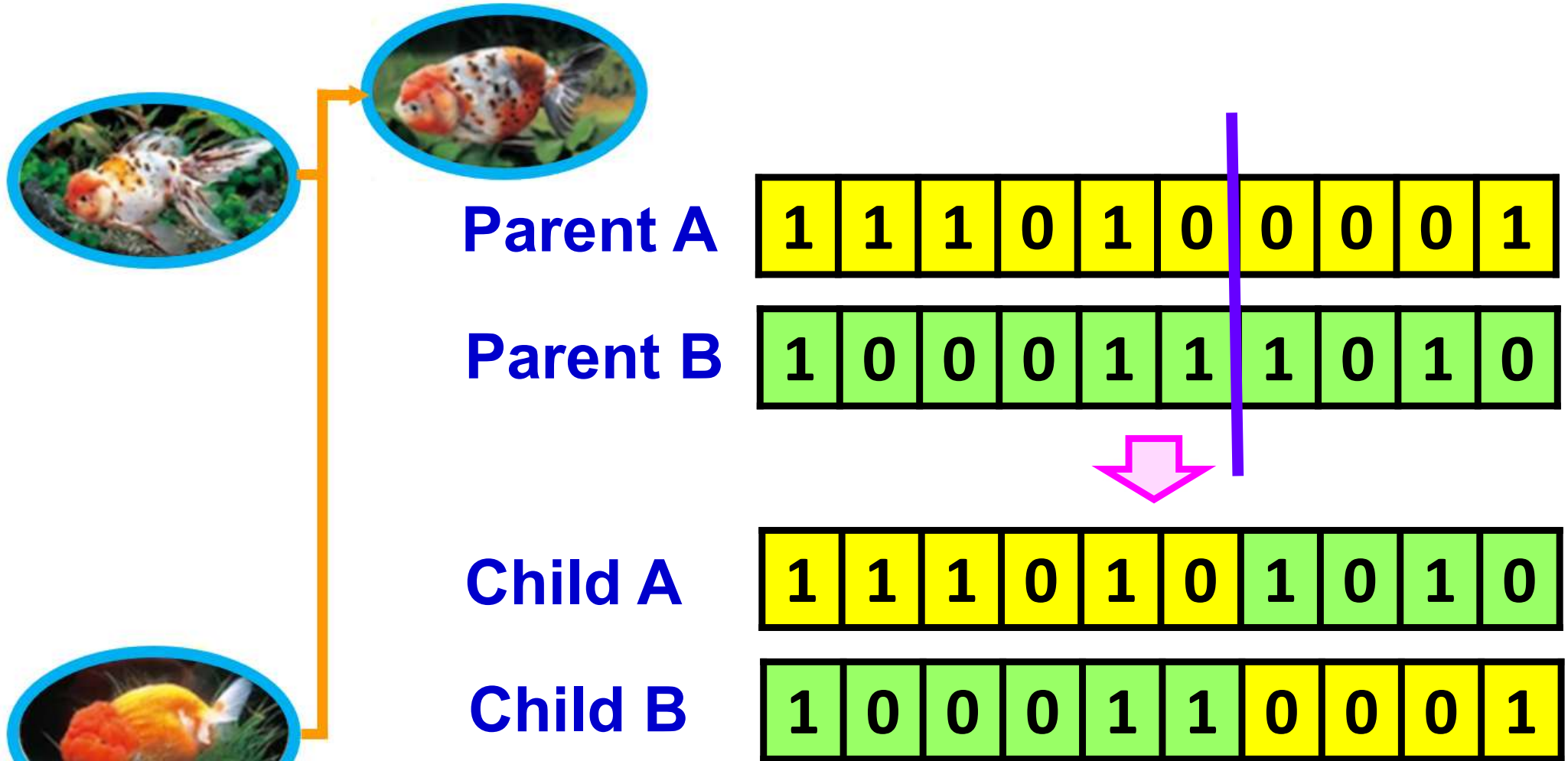


Crossover



Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

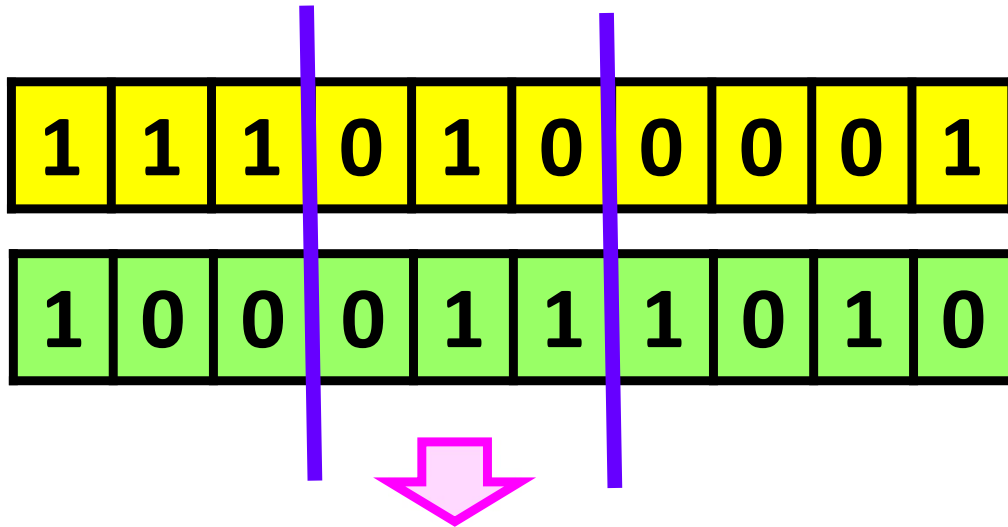


One-Point Crossover

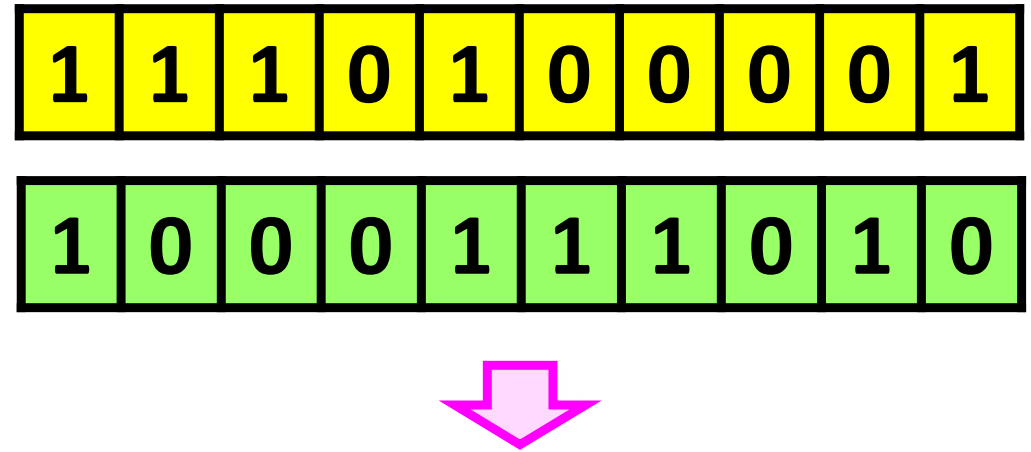
Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

Any exchanges are usually OK.



Two-Point Crossover



Uniform Crossover

Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

Multiple parents can be used.

Parent A

1	1	1	0	1	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---

Parent B

1	0	0	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---	---	---

Parent C

1	0	1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---



Child

1	0	0	0	1	0	0	1	0	1
---	---	---	---	---	---	---	---	---	---

Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

Calculation can be used.

$$\begin{array}{r} 0.2 \times \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 1.00 & 0.00 & 0.00 & 1.00 & 2.00 & 1.00 & 1.00 & 10.00 & 1.00 & 1.00 \\ \hline \end{array} \\ + \\ 0.8 \times \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 1.00 & 1.00 & 10.00 & 1.00 & 10.00 & 1.00 & 10.00 & 10.00 & 20.00 & 1.00 \\ \hline \end{array} \\ = \begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline 1.00 & 0.80 & 8.00 & 1.00 & 8.40 & 1.00 & 8.20 & 10.00 & 16.20 & 1.00 \\ \hline \end{array} \end{array}$$

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

1.0 x



0.6 x

100

0.6 x

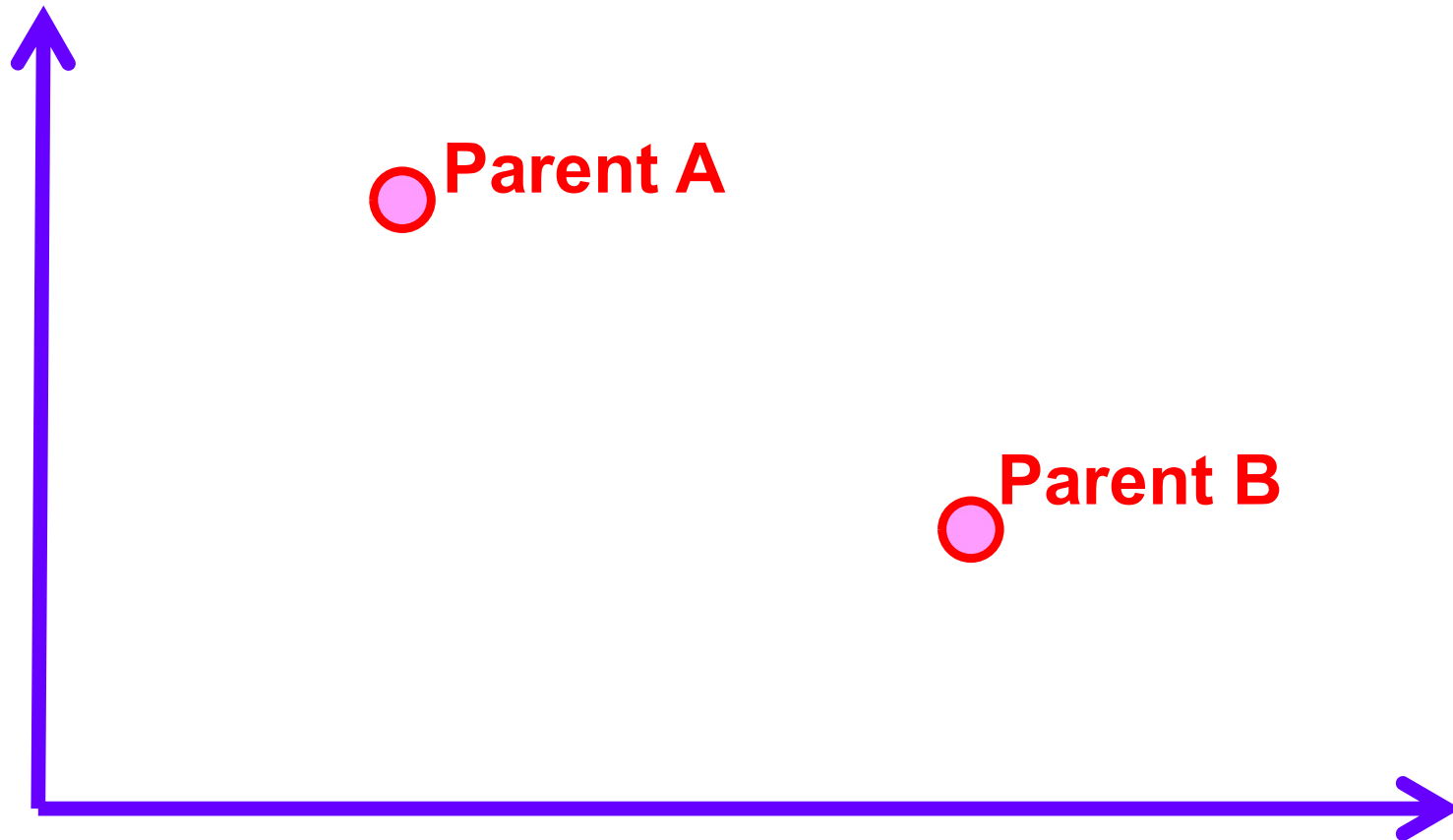
2

[illegible]

Crossover: **Exchange a part of strings**

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

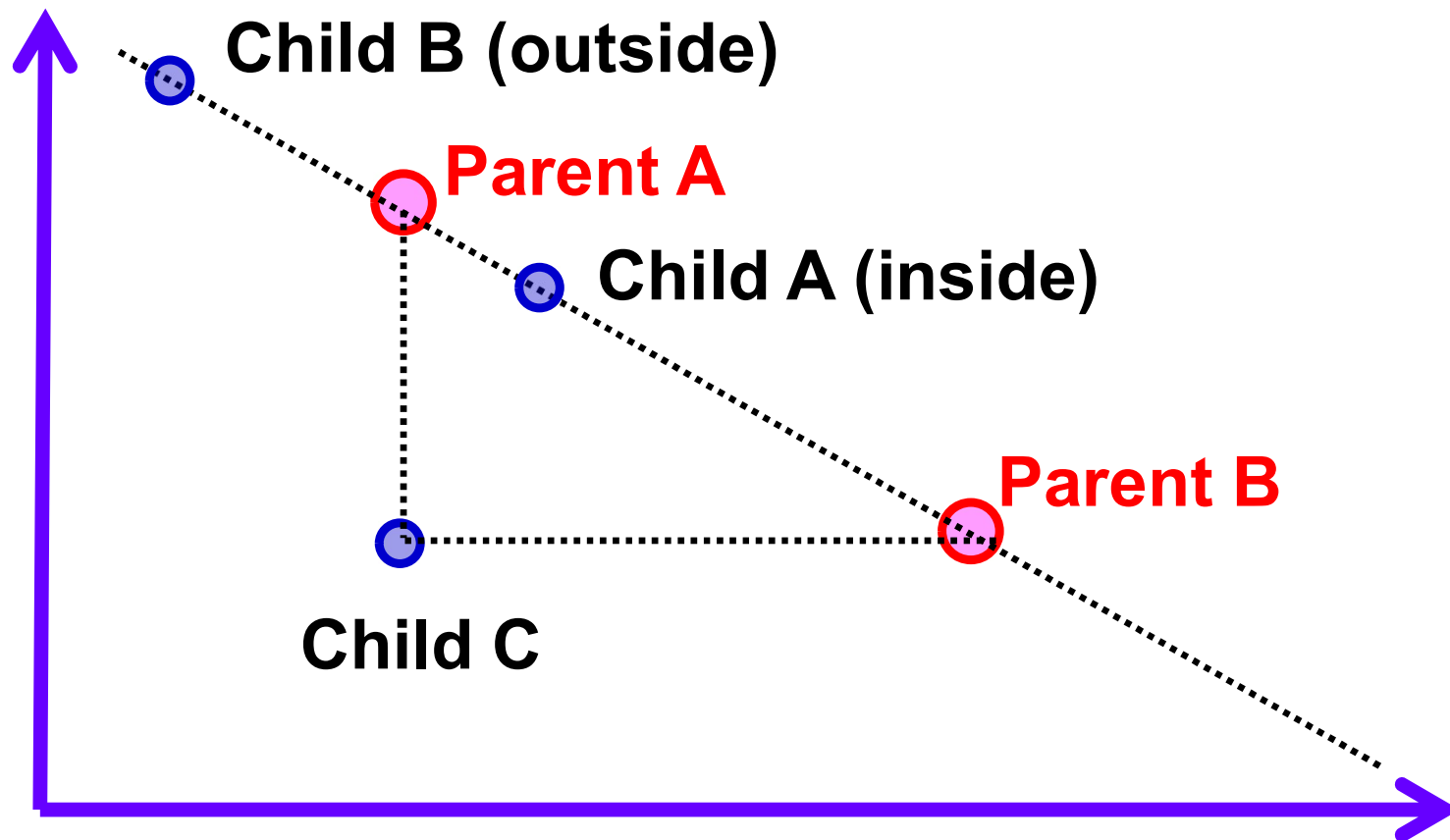
Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

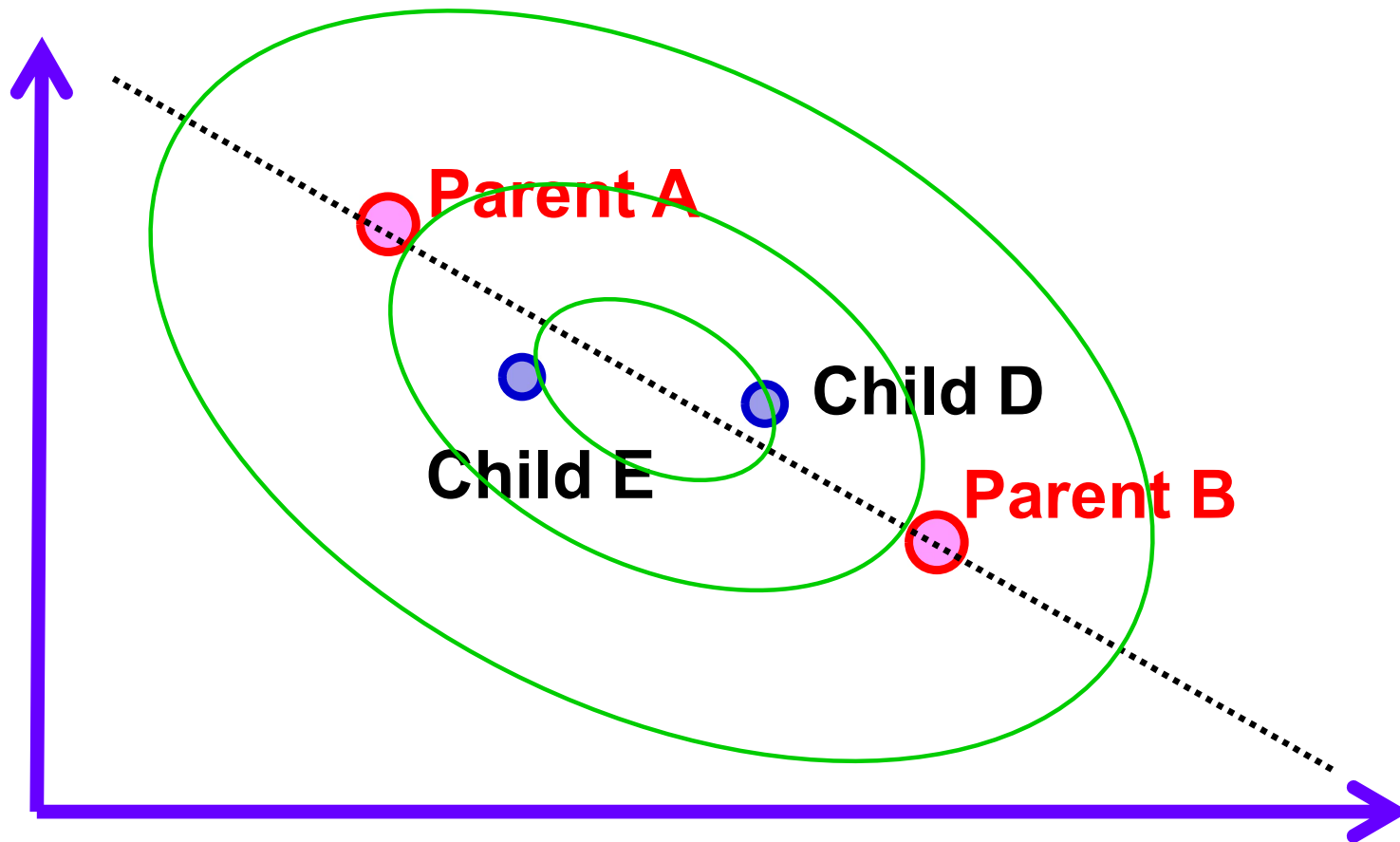
Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

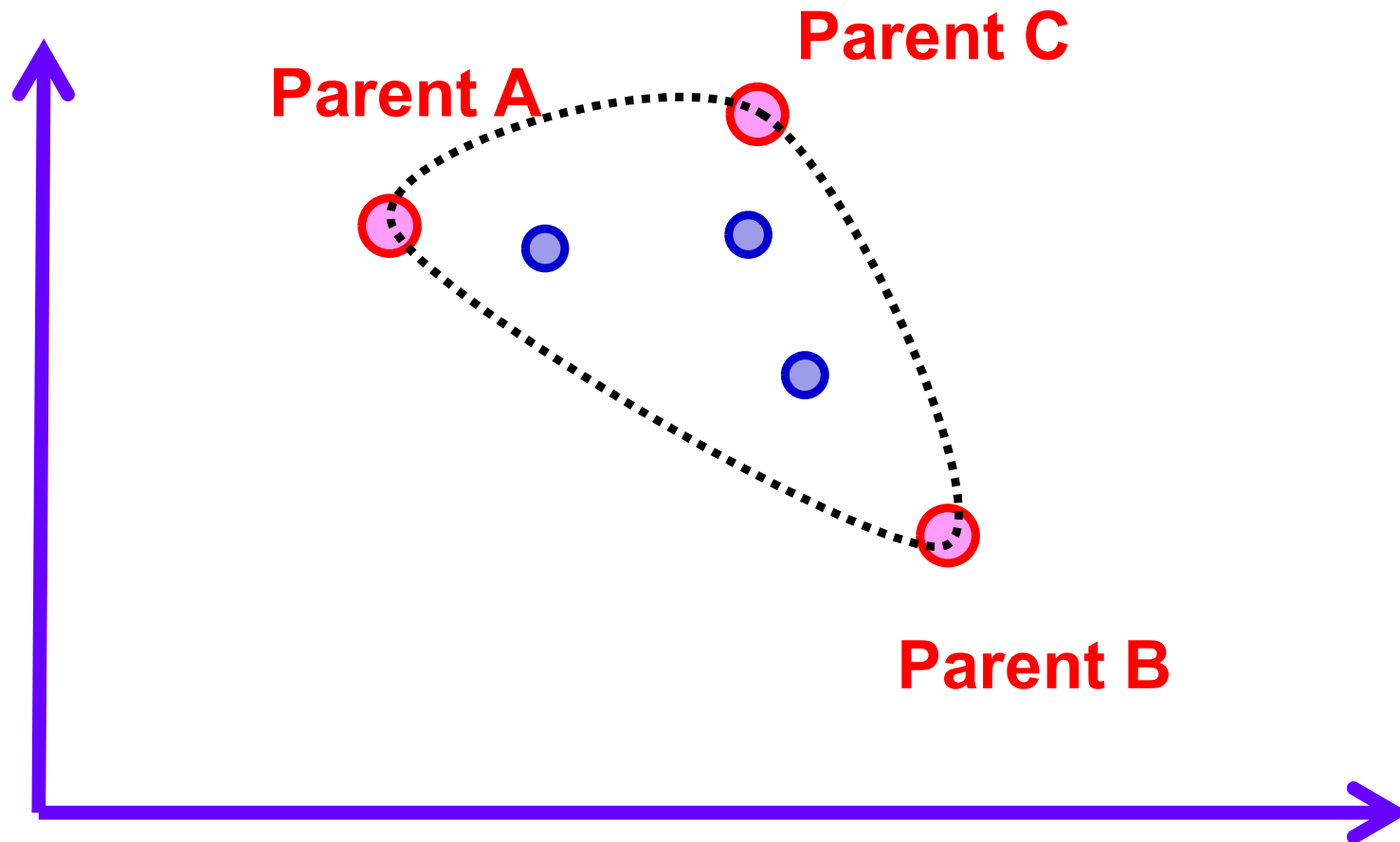
Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

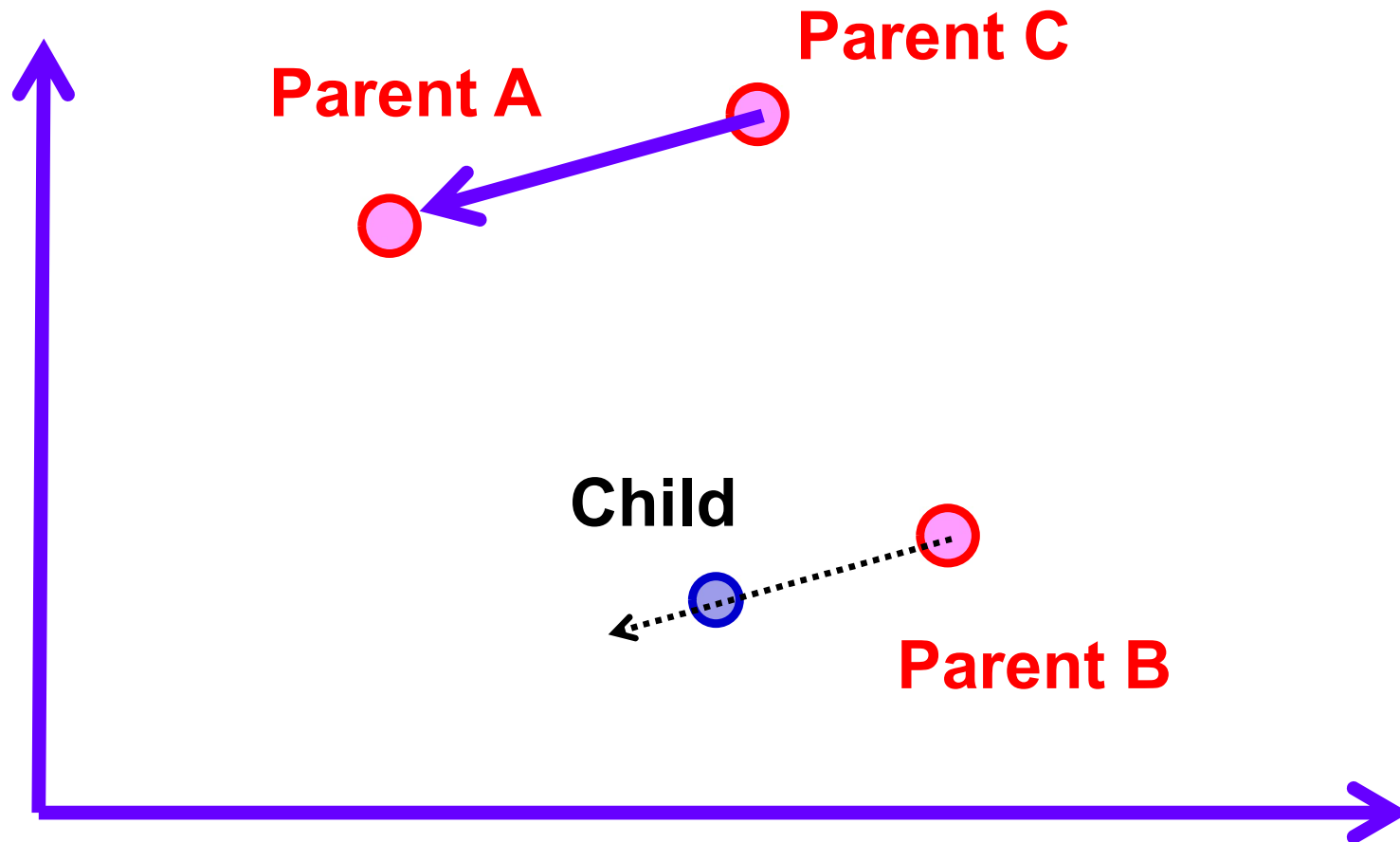
Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover: Exchange a part of strings

(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

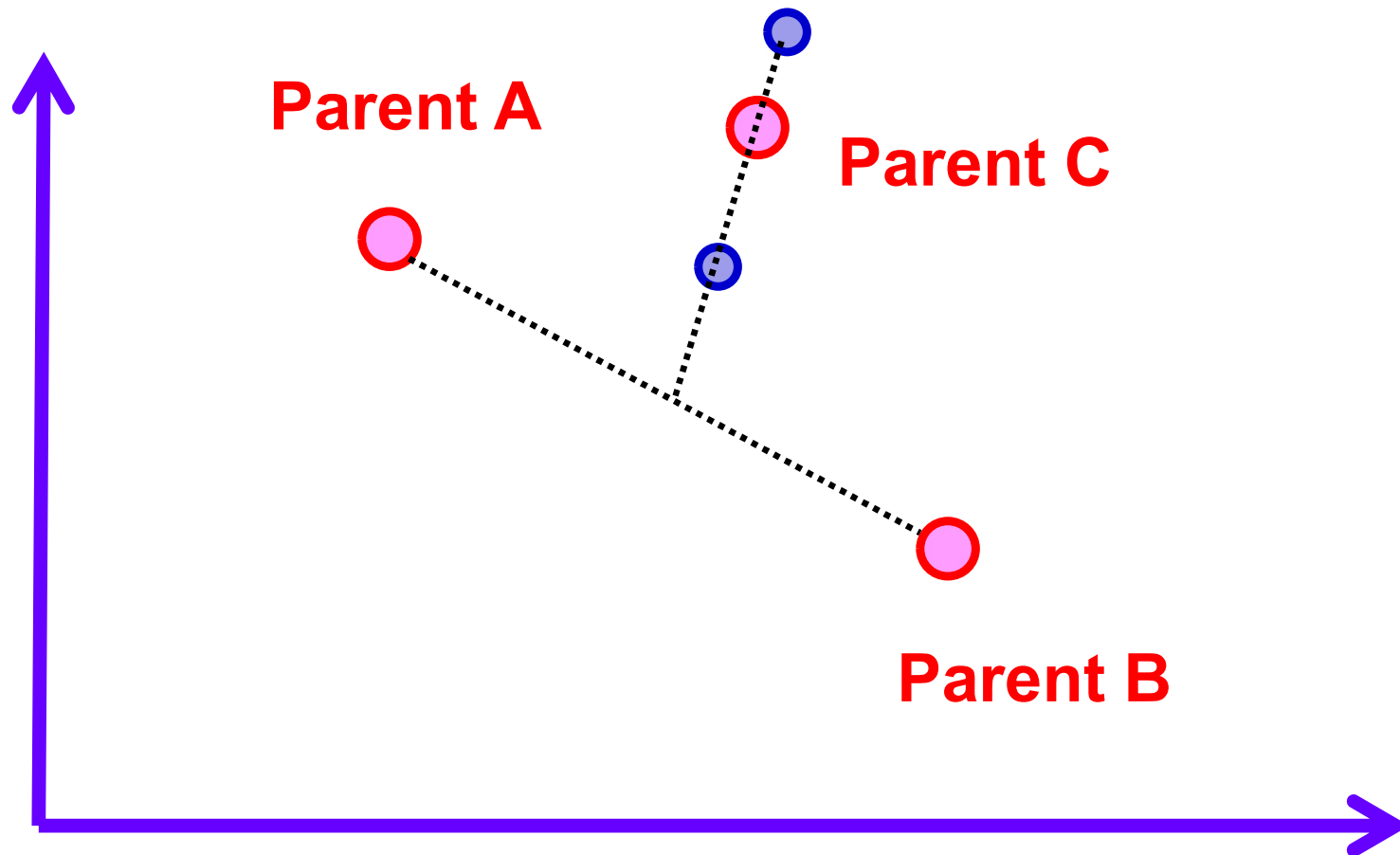
Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover: **Exchange a part of strings**

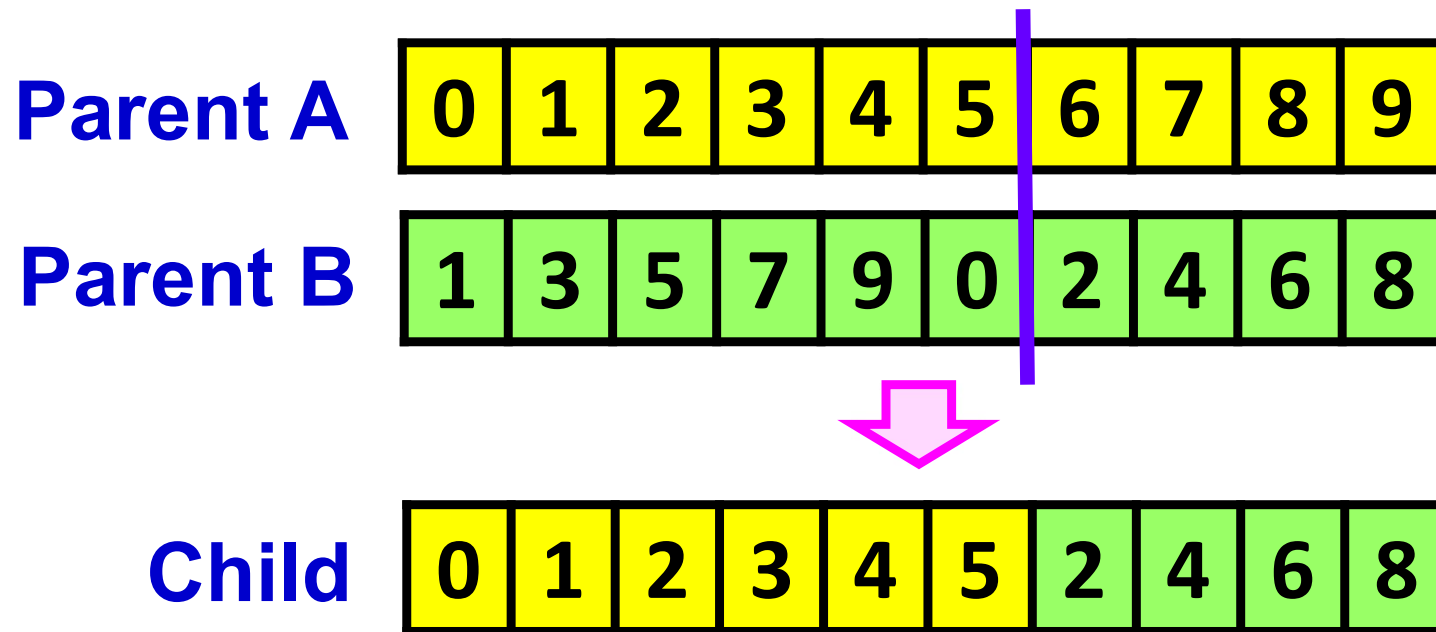
(with a crossover probability, e.g., 0.5, 0.8, 0.9, 1.0)

Crossover for real number strings of length 2 can be explained as a mechanism to generate a new point from the existing points in the 2D space.



Crossover for Permutation Strings

Simple crossover does not generate a permutation.

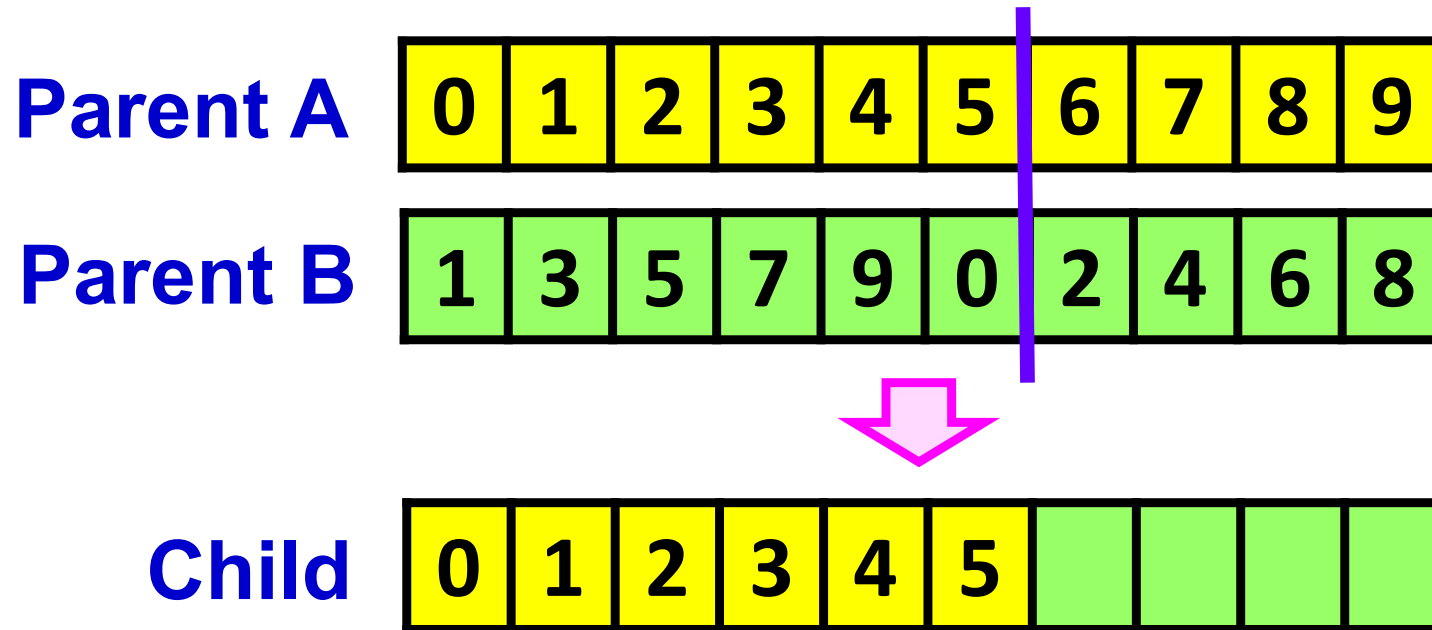


Two “2”, two “4”, no “7”, and no “9”.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

One-Point Order Crossover

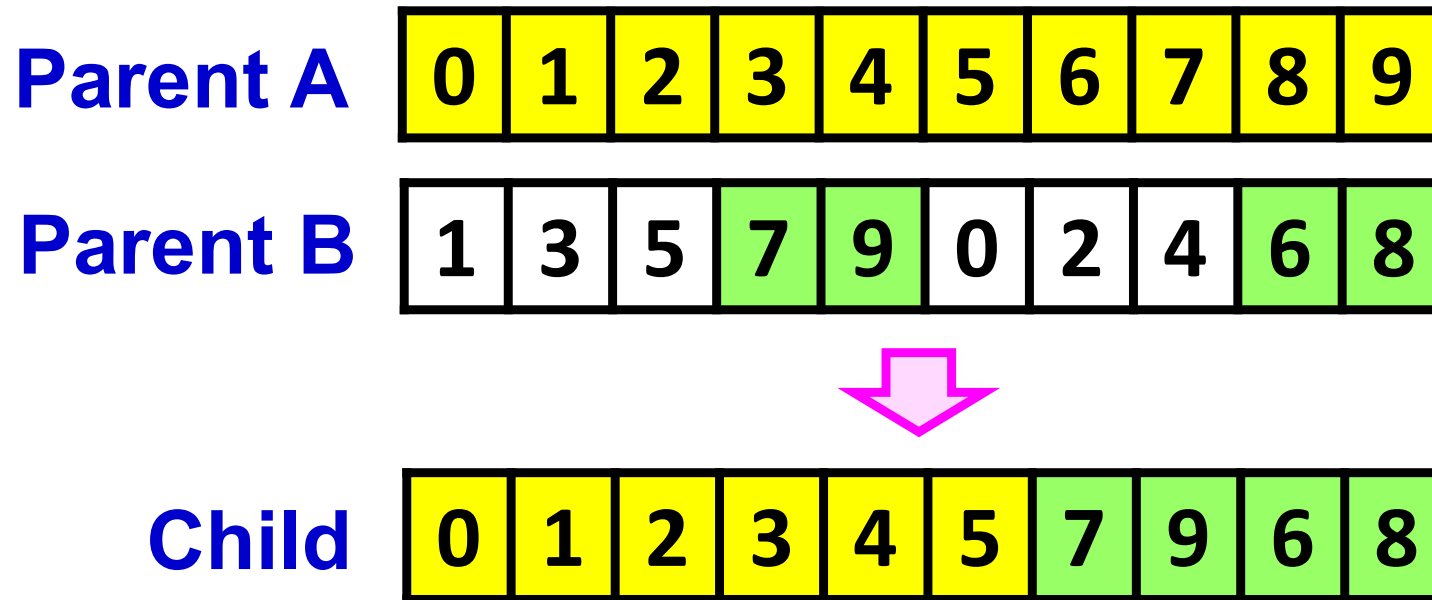


(1) A part of the child directly comes from Parent A.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

One-Point Order Crossover

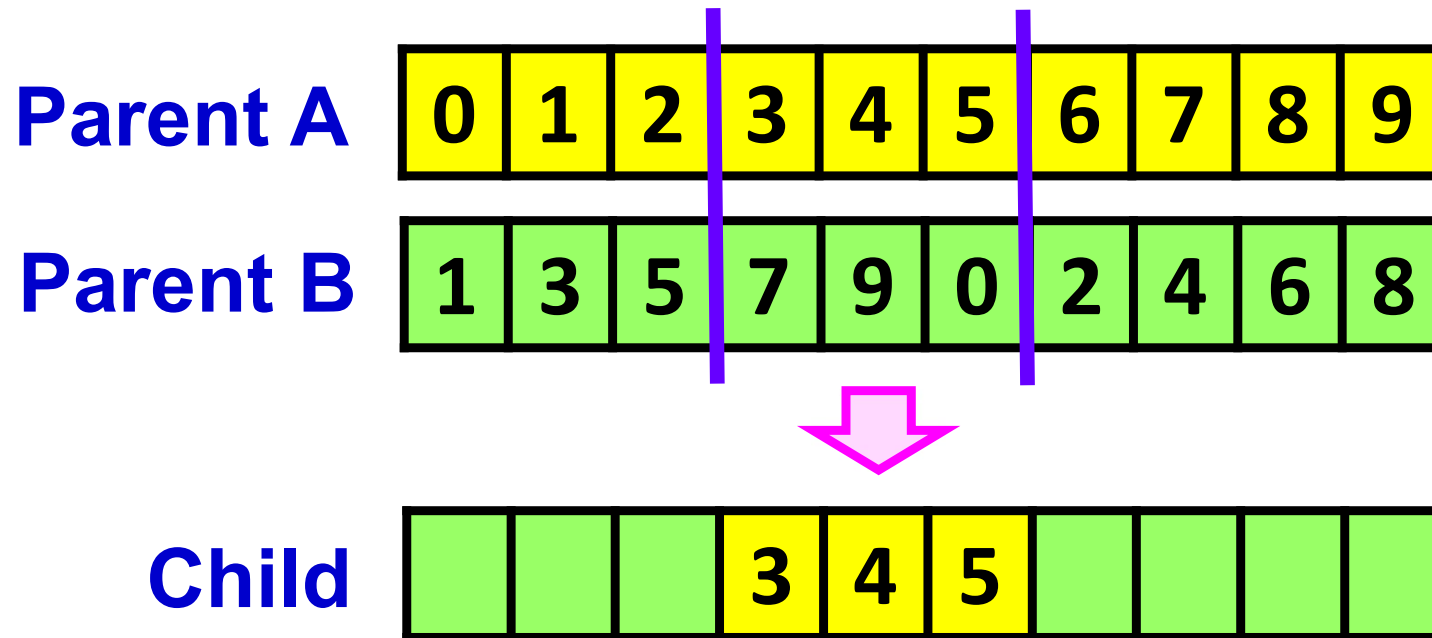


- (1) A part of the child directly comes from Parent A.
- (2) The other part of the child comes from Parent B in the order in Parent B.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

Two-Point Order Crossover

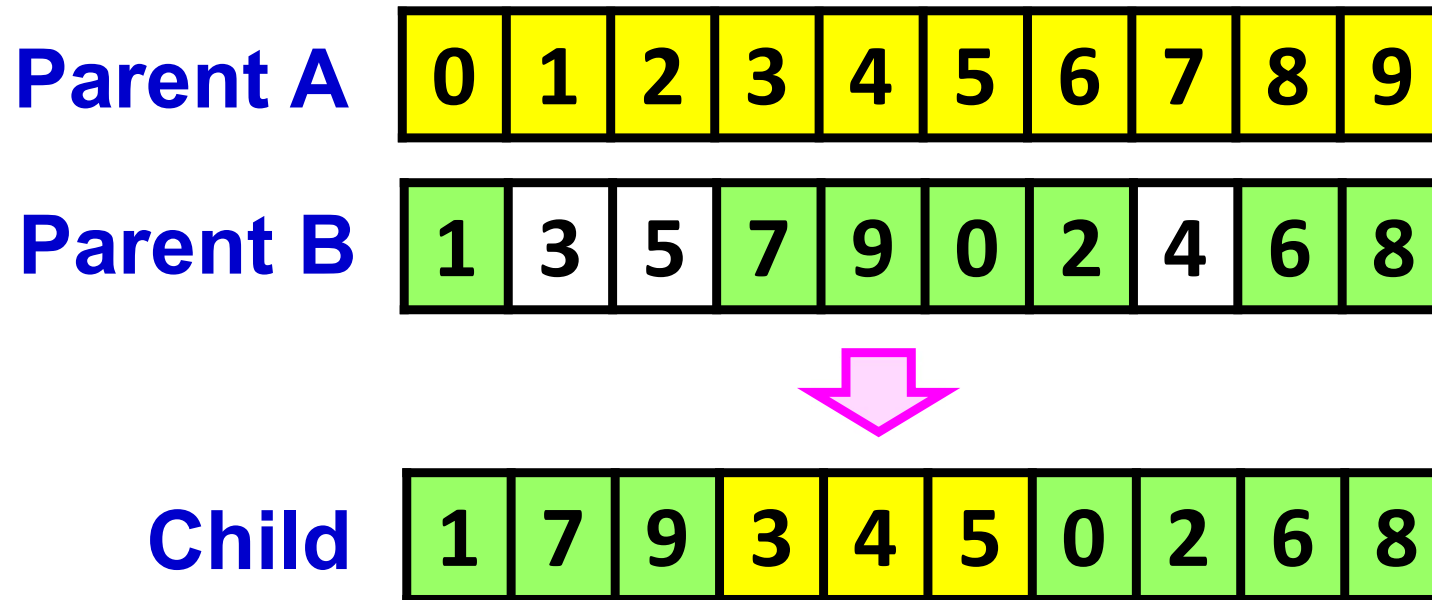


(1) A part of the child directly comes from Parent A.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

Two-Point Order Crossover

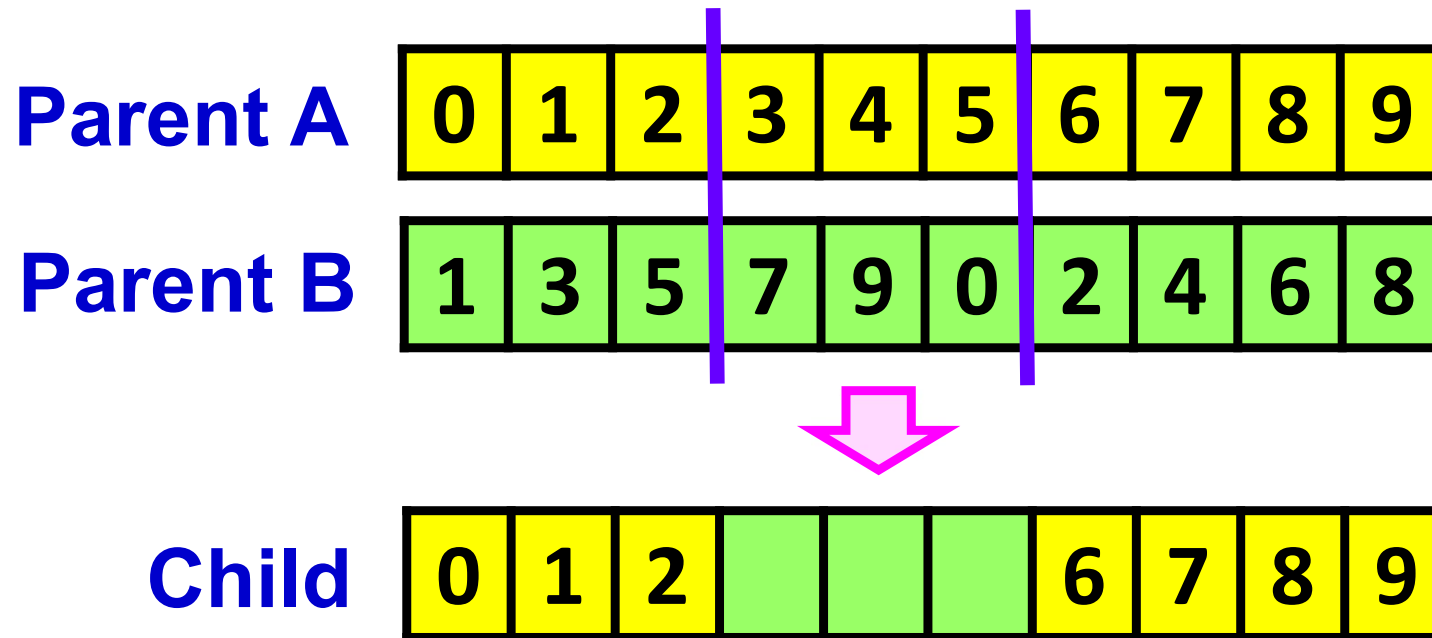


- (1) A part of the child directly comes from Parent A.
- (2) The other part of the child comes from Parent B in the order in Parent B.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

Two-Point Order Crossover

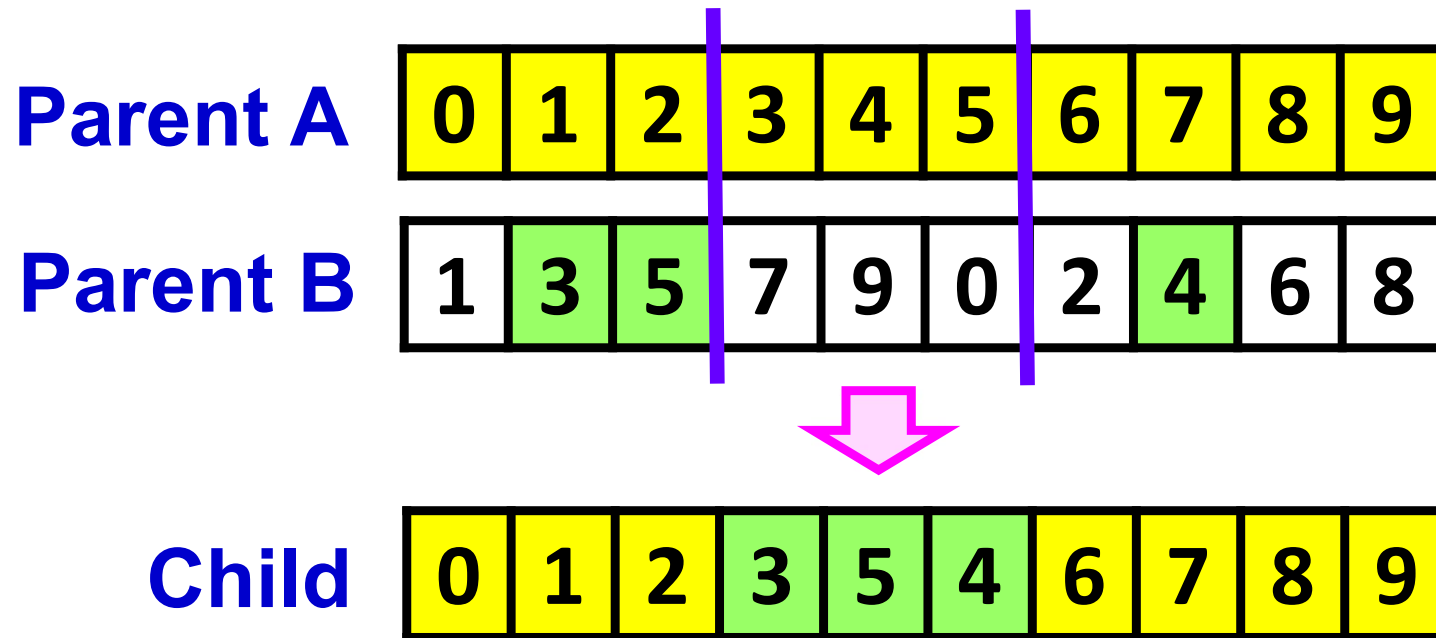


(1) A part of the child directly comes from Parent A.

Crossover for Permutation Strings

Special Crossover for Permutation Strings:

Two-Point Order Crossover



- (1) A part of the child directly comes from Parent A.
- (2) The other part of the child comes from Parent B in the order in Parent B.

How to evaluate each solution

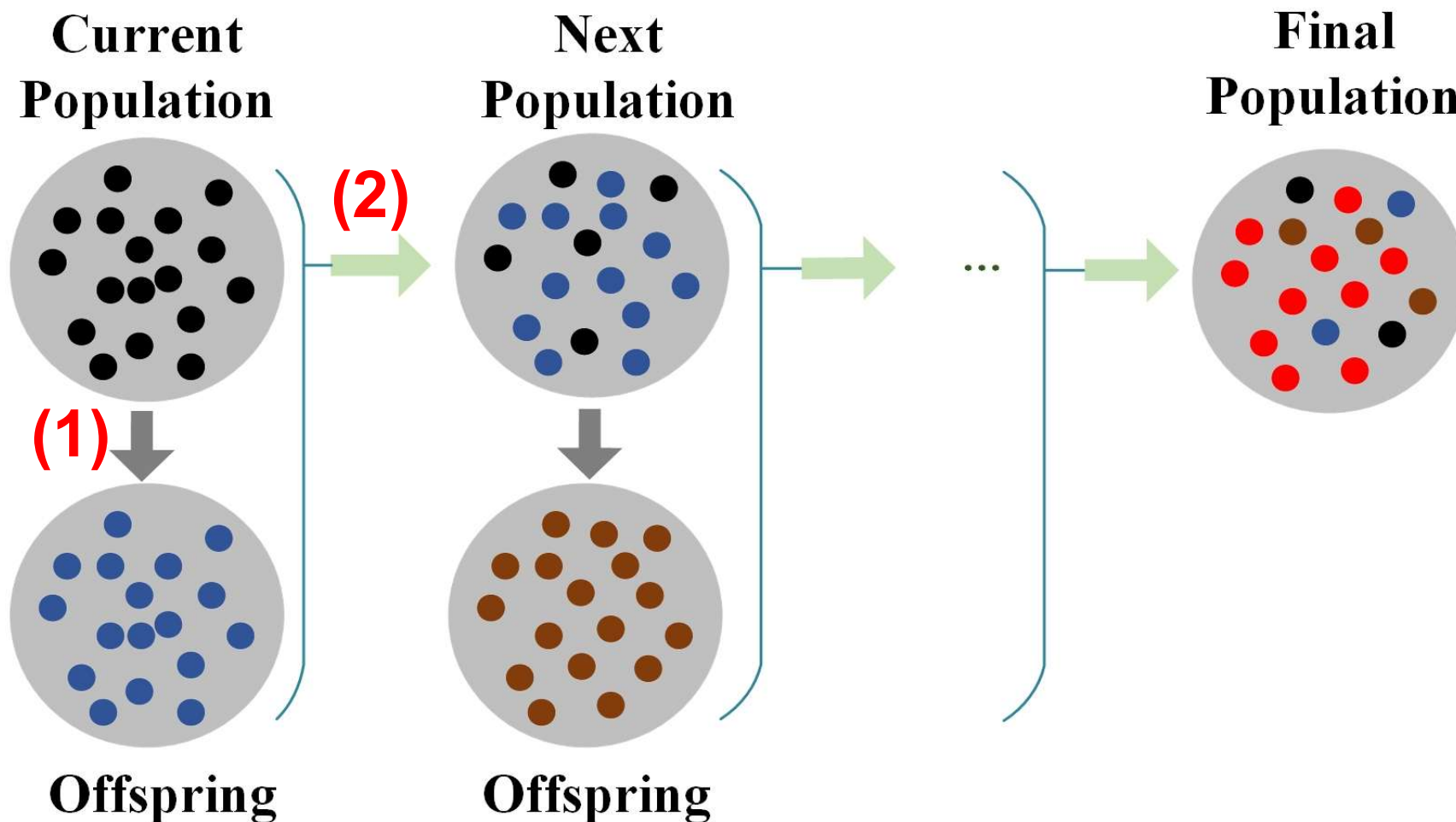
Solution evaluation depends on the problem.

- Profit maximization problem: **Profit**
- Cost minimization problem: **Cost**
- Distance minimization problem: **Distance**
- Error minimization problem: **Error**
- ...

Selection: How to select good solutions

Good solutions are selected through

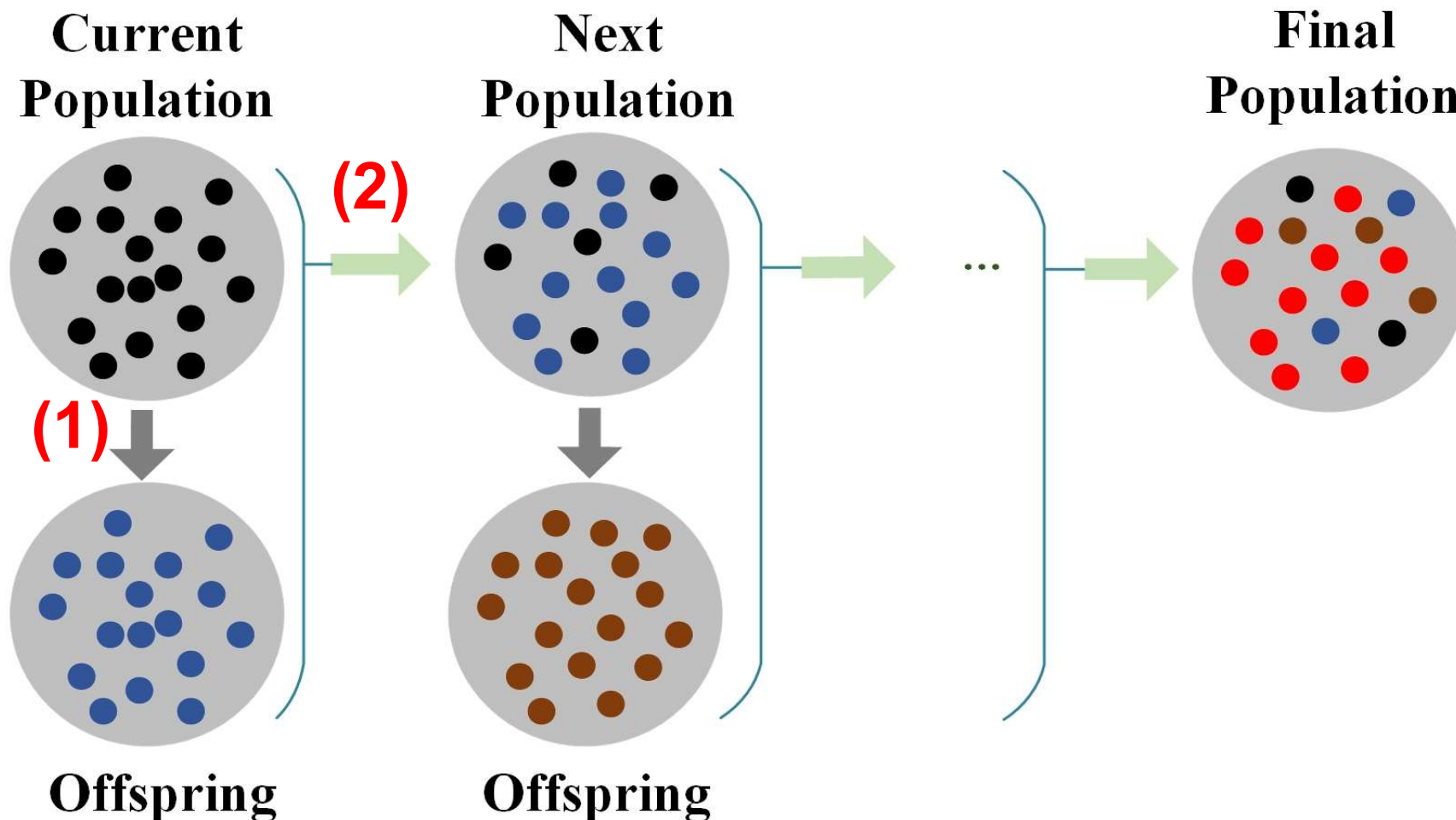
- (1) selection of parents from current population to generate new solutions
- (2) selection of solutions for the next population from the current population and the offspring population.



Two Phases of Selection:

(1) Parent Selection (Mating Selection)

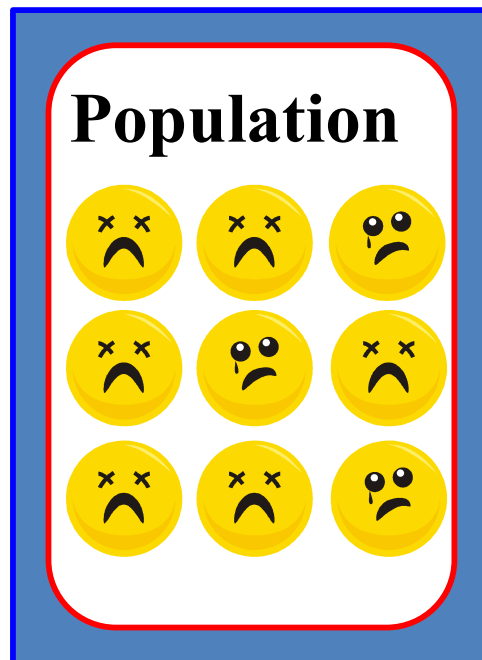
(2) Generation Update (Environmental Selection)



Parent Selection (Mating Selection)

Basic Idea of Parent Selection:

- Good children will be generated more likely from good parents than poor parents.
- So, let us choose good solutions as parents.



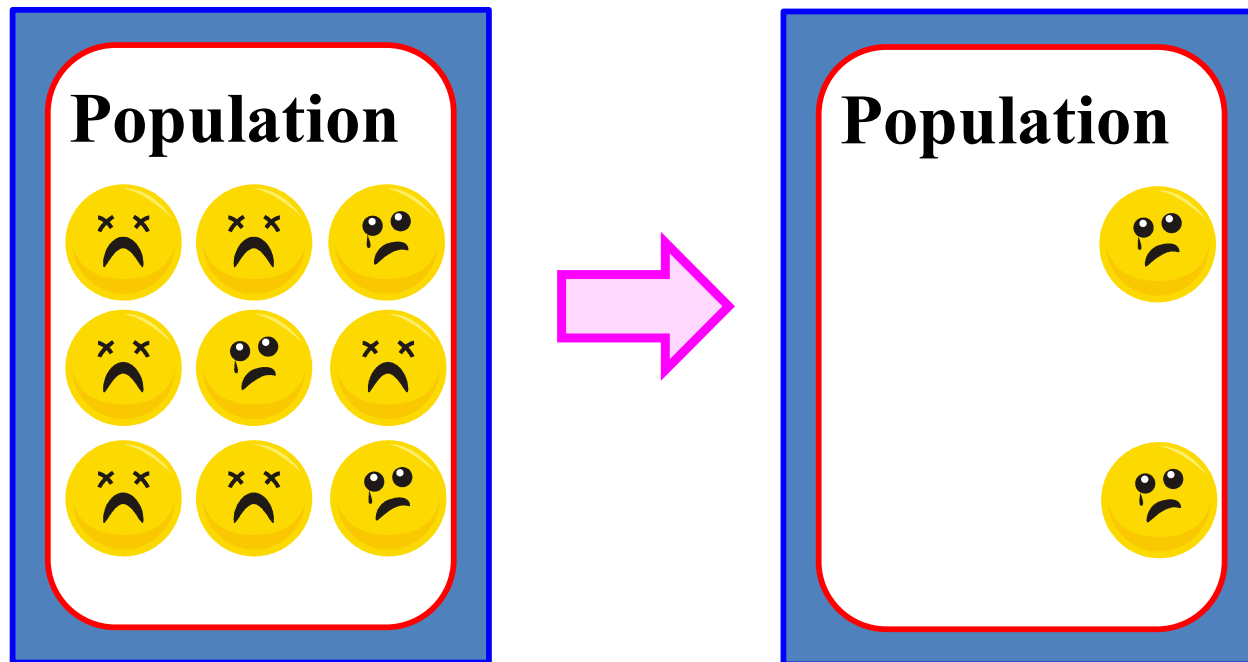
Parent Selection (Mating Selection)

Implementation

The basic idea is simple. However, there are a lot of possible implementations such as

- To choose the best two solutions as parents.

All children are generated from the two solutions.



Parent Selection (Mating Selection)

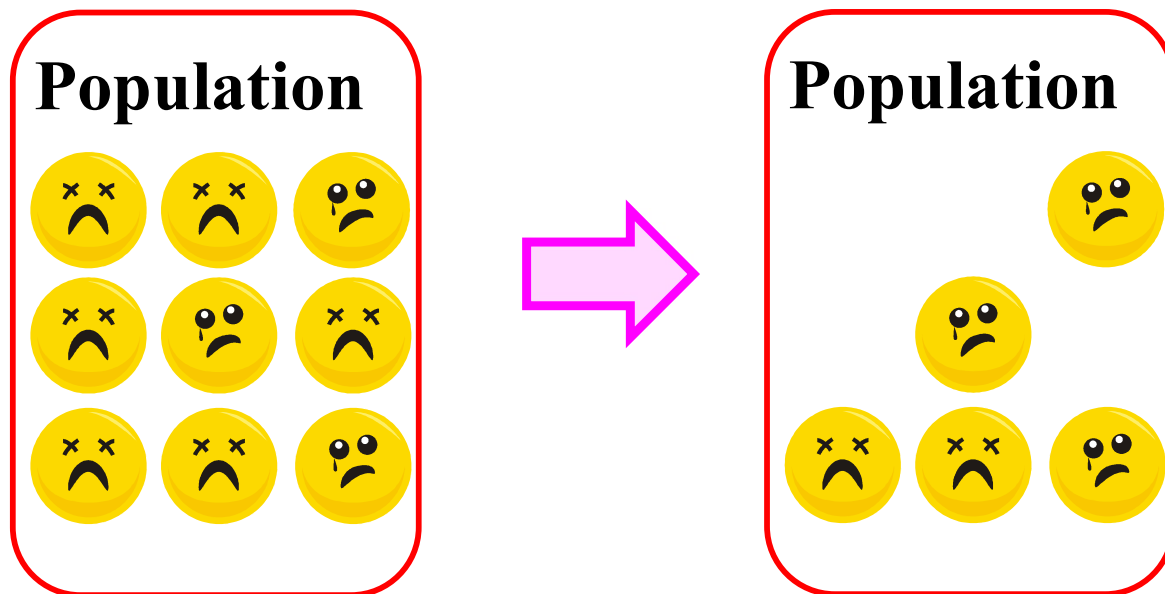
Implementation

The basic idea is simple. However, there are a lot of possible implementations such as

- To choose the best two solutions as parents.

All children are generated from the two solutions.

- To choose a half solutions as parents.



Parent Selection (Mating Selection)

Implementation

The basic idea is simple. However, there are a lot of possible implementations such as

- To choose the best two solutions as parents.

All children are generated from the two solutions.

- To choose a half solutions as parents.
- Assign a different selection probability to each solution depending on the fitness value.



Parent Selection (Mating Selection)

Roulette Wheel Selection (Fitness Proportional Selection)

Current Solutions: $x_1, x_2, \dots, x_N$ (N : Population Size)

Their Fitness Values: $fitness(x_1), fitness(x_2), \dots, fitness(x_N)$

Their Selection Probabilities: $p(x_1), p(x_2), \dots, p(x_N)$

$$p(x_i) = fitness(x_i) / (fitness(x_1) + fitness(x_2) + \dots + fitness(x_N))$$

Example ($N = 10$): **Exercise**

$$fitness(x_1) = fitness(x_2) = fitness(x_3) = fitness(x_4) = fitness(x_5) = fitness(x_6) = 1$$

$$fitness(x_7) = fitness(x_8) = fitness(x_9) = 2, \quad fitness(x_{10}) = 8$$

$$p(x_1) = p(x_2) = p(x_3) = p(x_4) = p(x_5) = p(x_6) = 0.05$$

$$p(x_7) = p(x_8) = p(x_9) = 0.1, \quad p(x_{10}) = 0.4$$

Rank Selection

Selection probability is based on the rank of each solution.

Example ($N = 10$)

$$\textit{fitness}(x_1) = 12$$

$$\textit{fitness}(x_2) = 23$$

$$\textit{fitness}(x_3) = 43$$

$$\textit{fitness}(x_4) = 99 \text{ (Best)}$$

$$\textit{fitness}(x_5) = 11$$

$$\textit{fitness}(x_6) = 12$$

$$\textit{fitness}(x_7) = 24$$

$$\textit{fitness}(x_8) = 56$$

$$\textit{fitness}(x_9) = 79 \text{ (3rd)}$$

$$\textit{fitness}(x_{10}) = 92 \text{ (2nd)}$$

Example of Selection Probabilities ($N = 10$)

Best: 0.19, 2nd: 0.17, 3rd: 0.15, 4th: 0.13, 5th: 0.11

6th: 0.09, 7th: 0.07, 8th: 0.05, 9th: 0.03, 10th: 0.01

Tournament Selection

**Choose the best from randomly selected K solutions.
(K : Tournament Size)**

Iterate the following steps:

Step 1: Randomly select K solutions from the current population.

Step 2: Select the best solution among the K solutions.

- Selection pressure (probabilities) can be adjusted by K .

Large $K \implies$ Only very good solutions can be parents.

Small $K \implies$ Average solutions can have some probabilities

$K = 1 \implies$ Random selection

Tournament Selection

**Choose the best from randomly selected K solutions.
(K : Tournament Size)**

Iterate the following steps:

Step 1: Randomly select K solutions from the current population.

Step 2: Select the best solution among the K solutions.

==> Selection probability of each solution depends on its rank (and the value of K).

==> Tournament selection with a specific value of K can be implemented as rank selection by specifying the corresponding selection probability for each rank.

Tournament Selection

Choose the best from randomly selected K solutions.
(K : Tournament Size)

Iterate the following steps:

Step 1: Randomly select K solutions from the current population
(with duplication or without duplication).

Step 2: Select the best solution among the K solutions.

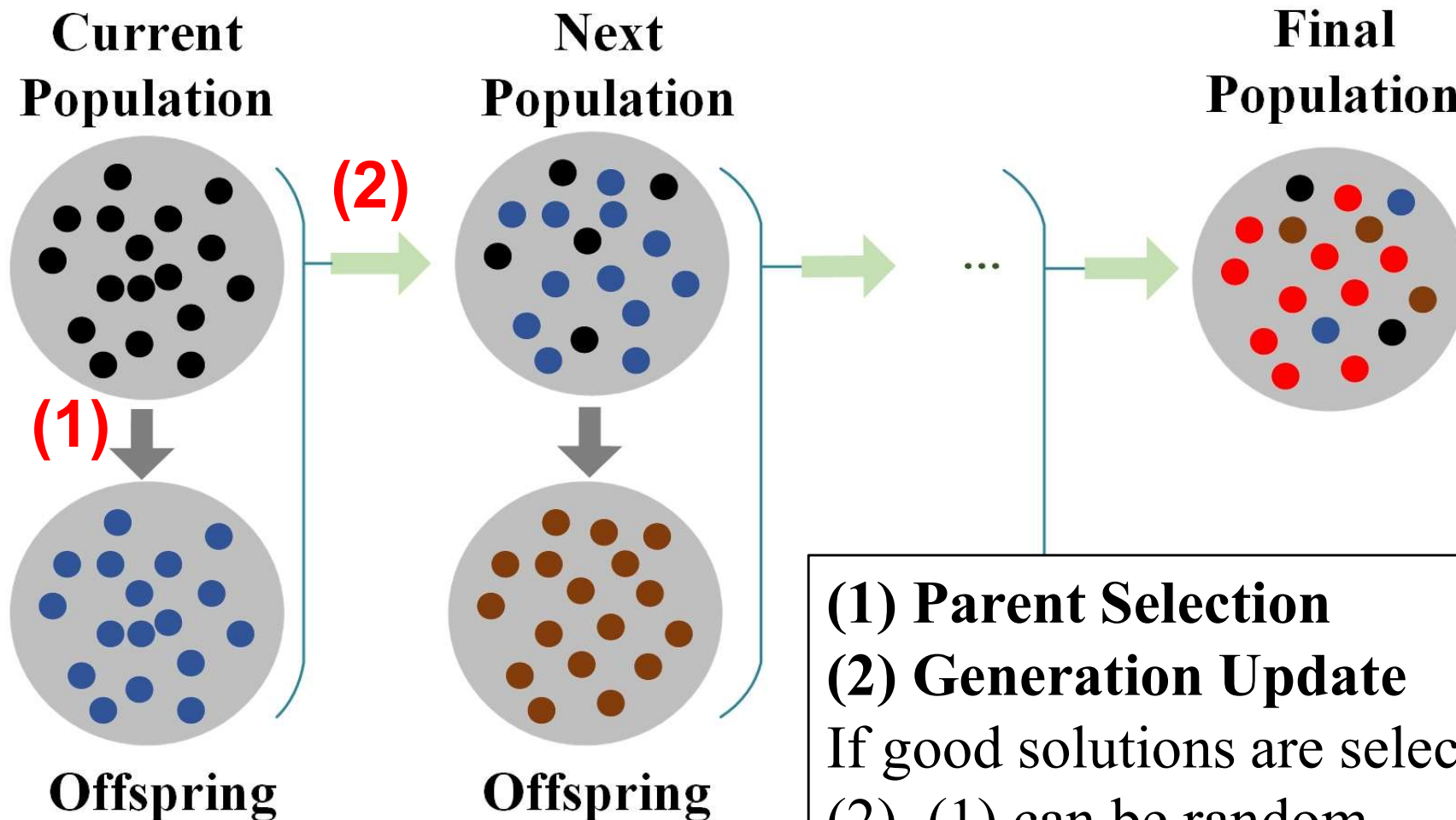
==> Selection probability of each solution depends on its rank (and the value of K).

Lab Session Task 1:

1. Calculate the selection probability of the best solution for the case of $N = 10$ (population size) and $K = 2$ (tournament size) “with duplication”.
2. Calculate the selection probability of the i -th solution for the case of $N = 10$ (population size) and $K = 2$ (tournament size) “with duplication”.
3. Calculate the selection probability of the i -th solution for the case of $N = 10$ (population size) and $K = 3$ (tournament size) “with duplication”.

Special Parent Selection Mechanism: Random Selection

Parent can be randomly selected from the current solution.

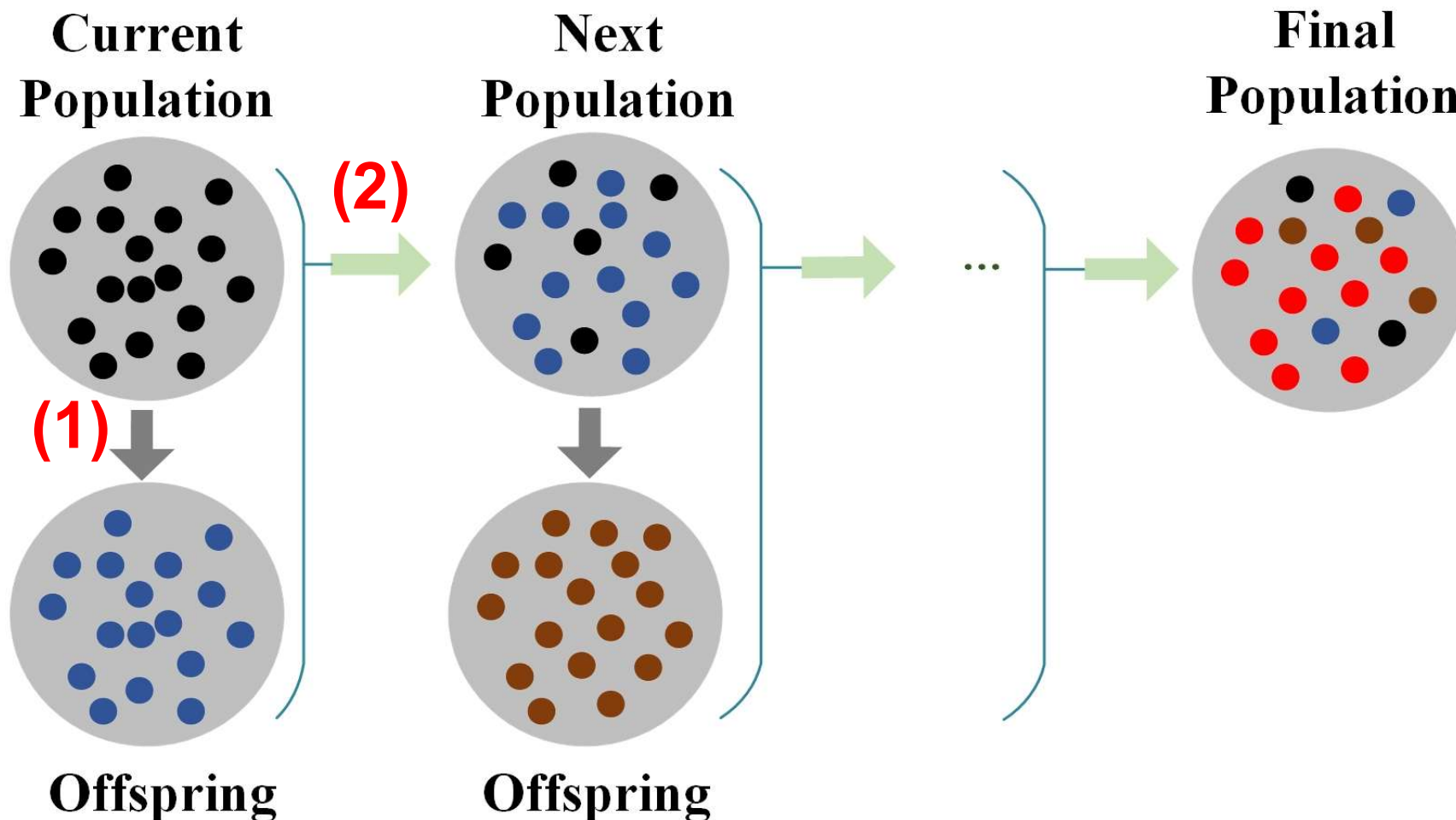


(1) Parent Selection
(2) Generation Update
If good solutions are selected in (2), (1) can be random.

Two Phases of Selection:

(1) Parent Selection (Mating Selection)

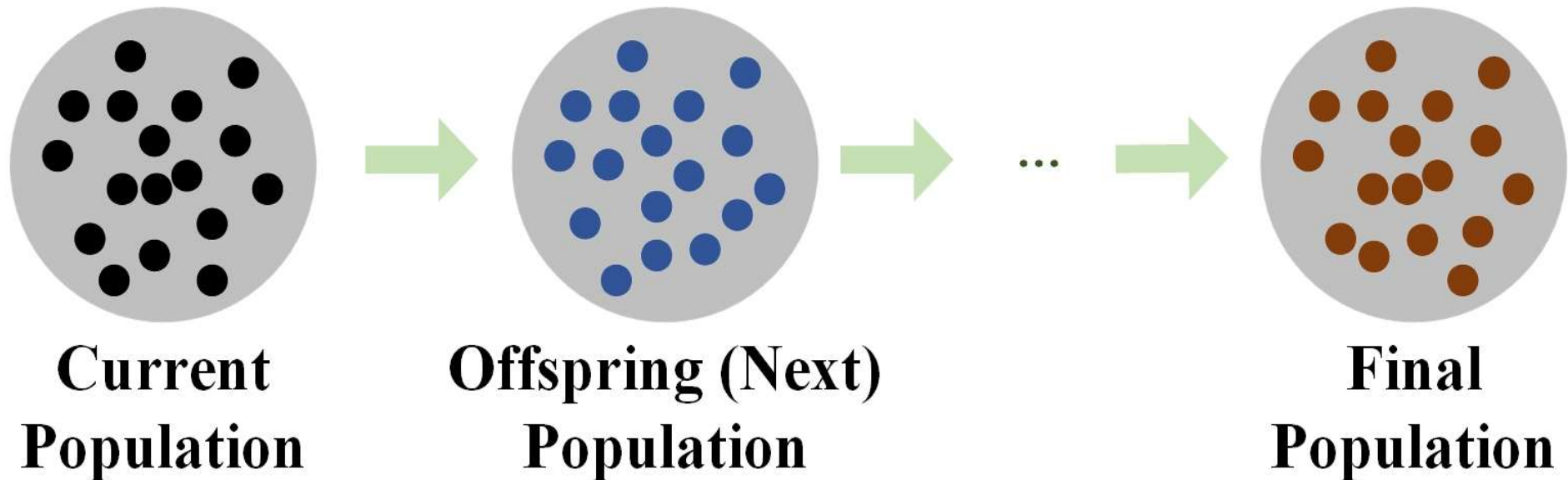
(2) Generation Update (Environmental Selection)



Generation Update (Environmental Selection)

Simplest Model: Next Population = Offspring Population

(This model is similar to evolution of many species in nature)



Advantage: Search can easily escape from local solutions.

Disadvantage: Good solutions cannot be efficiently utilized.

Strong selection pressure may be needed for parent selection.

Generation Update in ES (Evolution Strategies)

Plus Selection: $(\mu + \lambda)$ ES (Fast Convergence)

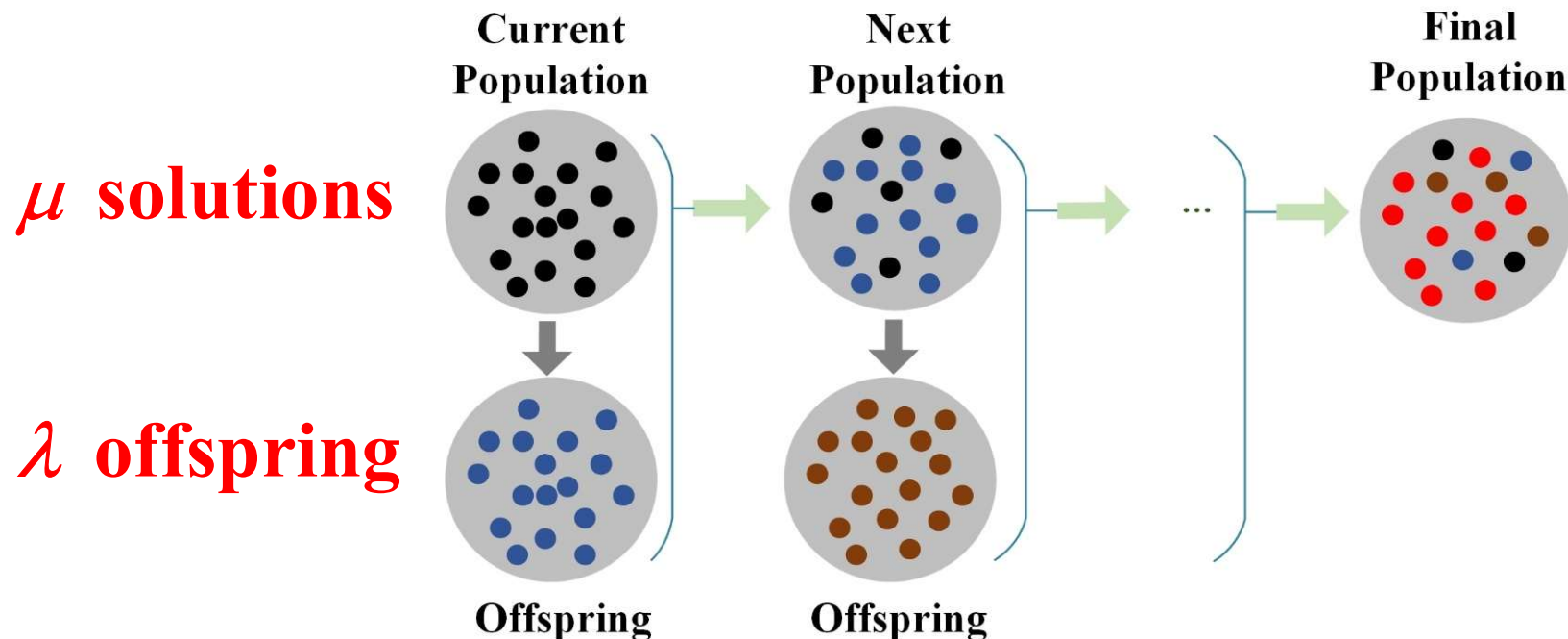
Select the best μ solutions from the $(\mu + \lambda)$ solutions

Comma Selection: (μ, λ) ES (Escape from Local Solutions)

Select the best μ solutions from the λ offspring

μ : Population size (Main population size)

λ : The number of offspring (Offspring population size)



Generation Update in ES (Evolution Strategies)

Special Cases

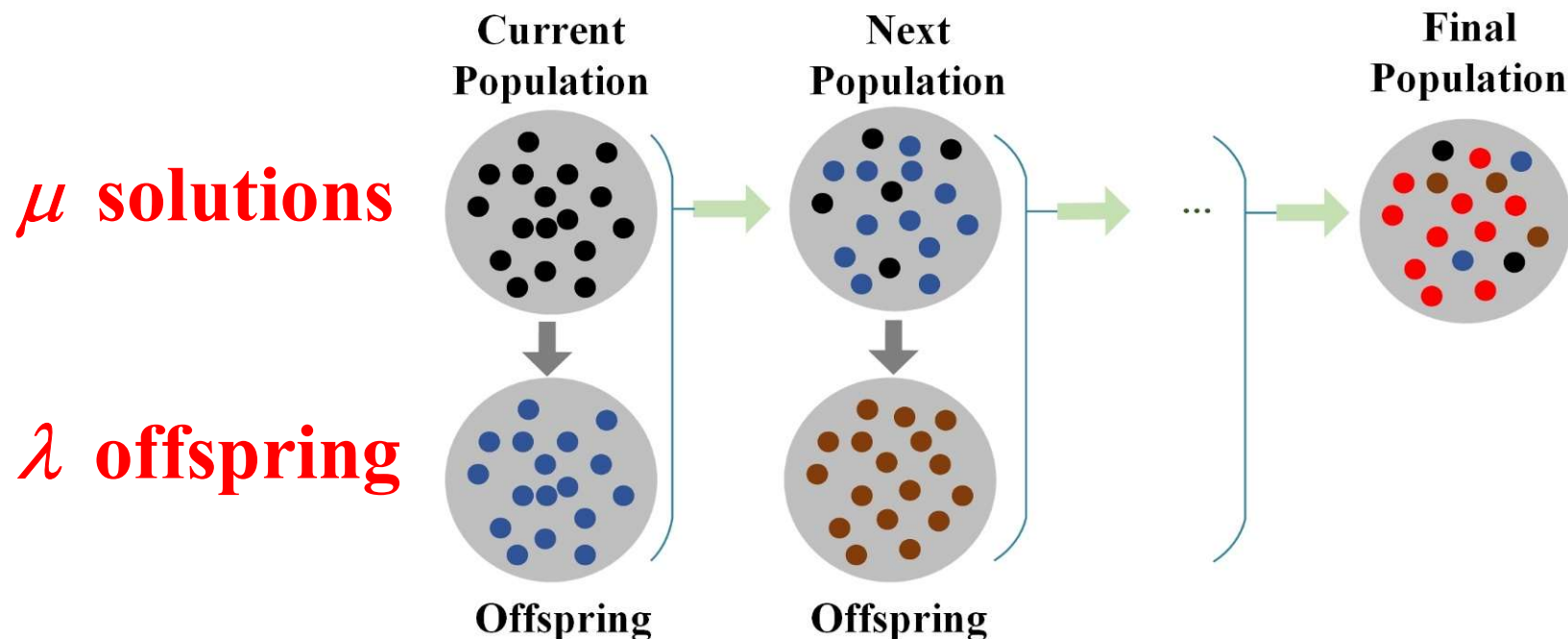
$(1, 1)$ ES: Random search

$(1 + 1)$ ES: Fast move local search

$(1 + \lambda)$ ES: Local search

$(\mu + 1)$ ES: Steady state algorithm

(μ, μ) ES: The simplest model of generation update

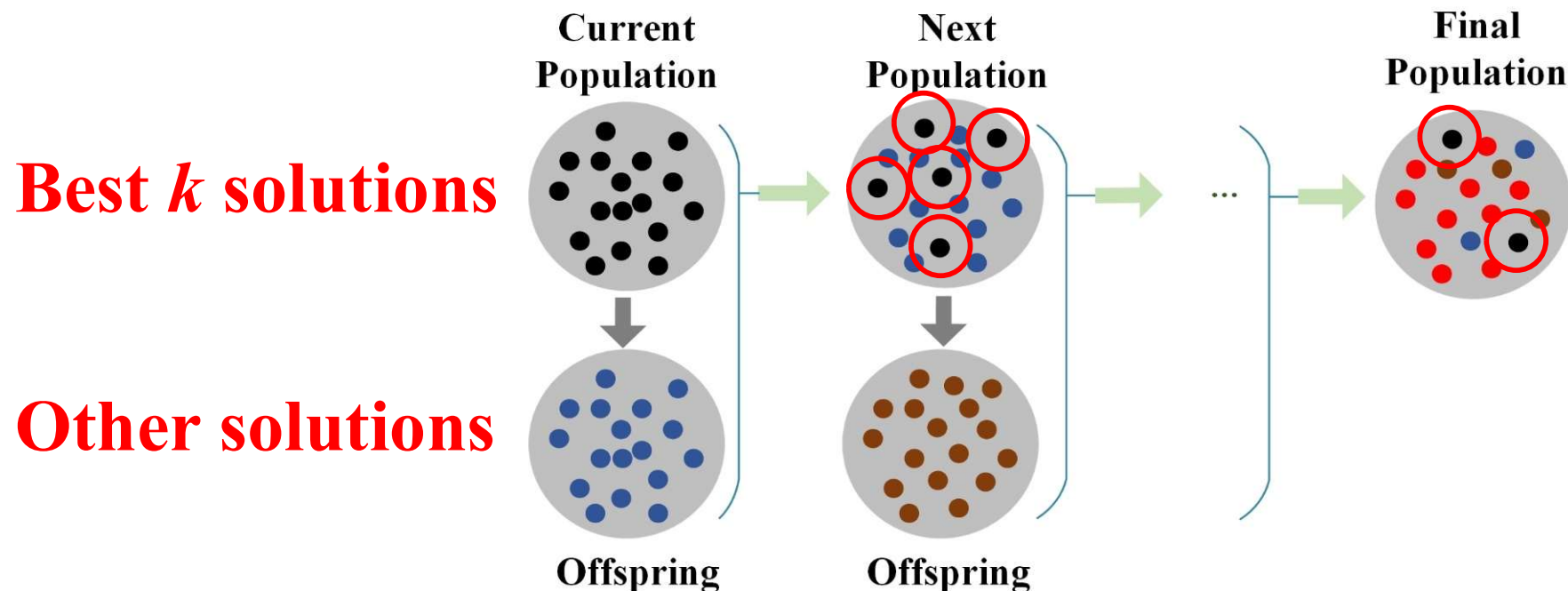


Generation Update in GA (Genetic Algorithms)

Elite Strategy (Elitist Strategy)

Next population with μ solutions:

- k best solutions in the current population (elite)
- $(\mu - k)$ new solutions (offspring)



Tournament Selection

Choose the best from randomly selected K solutions.
(K : Tournament Size)

Iterate the following steps:

Step 1: Randomly select K solutions from the current population
(with duplication or without duplication).

Step 2: Select the best solution among the K solutions.

==> Selection probability of each solution depends on its rank (and the value of K).

Lab Session Task 1:

1. Calculate the selection probability of the best solution for the case of $N = 10$ (population size) and $K = 2$ (tournament size) “with duplication”.
2. Calculate the selection probability of the i -th solution for the case of $N = 10$ (population size) and $K = 2$ (tournament size) “with duplication”.
3. Calculate the selection probability of the i -th solution for the case of $N = 10$ (population size) and $K = 3$ (tournament size) “with duplication”.

Lab Session Task 2:

(1) In Simulated Annealing, clearly explain the advantages and the disadvantages of each of the following two strategies: (i) the temperature is gradually decreased from a high temperature to a low temperature, and (ii) the temperature is fixed at a pre-specified temperature (i.e., the initial temperature and the final temperature are the same).

(2) In Simulated Annealing, which do you think is easier between the following two tasks: (i) to appropriately specify the initial temperature, the final temperature, and the cooling schedule between the initial and final temperatures, and (ii) to appropriately specify the fixed temperature. Please clearly explain the reason for your answer.

Lab Session Task 3:

In evolutionary computation, which do you think the best generation update mechanism among the following mechanisms? Please clearly explain the reason for your answer.

(1 , 1)ES: Random search

(1 + 1)ES: Fast move local search

(1 + λ)ES: Local search (e.g., $\lambda = 100$)

(μ + 1)ES: Steady state algorithm (e.g., $\mu = 100$)

(μ , μ)ES: The simplest model of generation update

(μ + μ)ES: The standard plus strategy

(μ , λ)ES: The standard comma strategy (e.g., $\mu = 100$, $\lambda = 500$)